

- [Cloud Storage with AWS](#)
- [Amazon EBS Volume Types](#)
- [Amazon EC2 Storage](#)
- [Amazon EFS: Amazon EFS Performance](#)
- [Amazon FSx for Lustre Performance](#)
- [Amazon FSx for Windows File Server Performance](#)
- [Amazon Glacier: Amazon Glacier Documentation](#)
- [Amazon S3: Request Rate and Performance Considerations](#)
- [Amazon EBS I/O Characteristics](#)
- [Cloud Databases with AWS](#)
- [AWS Database Caching](#)
- [DynamoDB Accelerator](#)
- [Amazon Aurora best practices](#)
- [Amazon Redshift performance](#)
- [Amazon Athena top 10 performance tips](#)
- [Amazon Redshift Spectrum best practices](#)
- [Amazon DynamoDB best practices](#)

Related videos:

- [AWS re:Invent 2023: Improve Amazon Elastic Block Store efficiency and be more cost-efficient](#)
- [AWS re:Invent 2023: Optimize storage price and performance with Amazon Simple Storage Service](#)
- [AWS re:Invent 2023: Building and optimizing a data lake on Amazon Simple Storage Service](#)
- [AWS re:Invent 2023: What's new with AWS file storage](#)
- [AWS re:Invent 2023: Dive deep into Amazon DynamoDB](#)

Related examples:

- [AWS Purpose Built Databases Workshop](#)
- [Databases for Developers](#)
- [AWS Modern Data Architecture Immersion Day](#)

- [Amazon EBS Autoscale](#)
- [Amazon S3 Examples](#)
- [Amazon DynamoDB Examples](#)
- [AWS Database migration samples](#)
- [Database Modernization Workshop](#)
- [Working with parameters on your Amazon RDS for Postgress DB](#)

PERF03-BP03 Collect and record data store performance metrics

Track and record relevant performance metrics for your data store to understand how your data management solutions are performing. These metrics can help you optimize your data store, verify that your workload requirements are met, and provide a clear overview on how the workload performs.

Common anti-patterns:

- You only use manual log file searching for metrics.
- You only publish metrics to internal tools used by your team and don't have a comprehensive picture of your workload.
- You only use the default metrics recorded by your selected monitoring software.
- You only review metrics when there is an issue.
- You only monitor system-level metrics and do not capture data access or usage metrics.

Benefits of establishing this best practice: Establishing a performance baseline helps you understand the normal behavior and requirements of workloads. Abnormal patterns can be identified and debugged faster, improving the performance and reliability of the data store.

Level of risk exposed if this best practice is not established: High

Implementation guidance

To monitor the performance of your data stores, you must record multiple performance metrics over a period of time. This allows you to detect anomalies, as well as measure performance against business metrics to verify you are meeting your workload needs.

Metrics should include both the underlying system that is supporting the data store and the database metrics. The underlying system metrics might include CPU utilization, memory, available

disk storage, disk I/O, cache hit ratio, and network inbound and outbound metrics, while the data store metrics might include transactions per second, top queries, average queries rates, response times, index usage, table locks, query timeouts, and number of connections open. This data is crucial to understand how the workload is performing and how the data management solution is used. Use these metrics as part of a data-driven approach to tune and optimize your workload's resources.

Use tools, libraries, and systems that record performance measurements related to database performance.

Implementation steps

- Identify the key performance metrics for your data store to track.
 - [Amazon S3 Metrics and dimensions](#)
 - [Monitoring metrics for in an Amazon RDS instance](#)
 - [Monitoring DB load with Performance Insights on Amazon RDS](#)
 - [Overview of Enhanced Monitoring](#)
 - [DynamoDB Metrics and dimensions](#)
 - [Monitoring DynamoDB Accelerator](#)
 - [Monitoring Amazon MemoryDB with Amazon CloudWatch](#)
 - [Which Metrics Should I Monitor?](#)
 - [Monitoring Amazon Redshift cluster performance](#)
 - [Timestream metrics and dimensions](#)
 - [Amazon CloudWatch metrics for Amazon Aurora](#)
 - [Logging and monitoring in Amazon Keyspaces \(for Apache Cassandra\)](#)
 - [Monitoring Amazon Neptune Resources](#)
- Use an approved logging and monitoring solution to collect these metrics. [Amazon CloudWatch](#) can collect metrics across the resources in your architecture. You can also collect and publish custom metrics to surface business or derived metrics. Use CloudWatch or third-party solutions to set alarms that indicate when thresholds are breached.
- Check if data store monitoring can benefit from a machine learning solution that detects performance anomalies.
 - [Amazon DevOps Guru for Amazon RDS](#) provides visibility into performance issues and makes recommendations for corrective actions.

- Configure data retention in your monitoring and logging solution to match your security and operational goals.
 - [Default data retention for CloudWatch metrics](#)
 - [Default data retention for CloudWatch Logs](#)

Resources

Related documents:

- [AWS Database Caching](#)
- [Amazon Athena top 10 performance tips](#)
- [Amazon Aurora best practices](#)
- [DynamoDB Accelerator](#)
- [Amazon DynamoDB best practices](#)
- [Amazon Redshift Spectrum best practices](#)
- [Amazon Redshift performance](#)
- [Cloud Databases with AWS](#)
- [Amazon RDS Performance Insights](#)

Related videos:

- [AWS re:Invent 2022 - Performance monitoring with Amazon RDS and Aurora, featuring Autodesk](#)
- [Database Performance Monitoring and Tuning with Amazon DevOps Guru for Amazon RDS](#)
- [AWS re:Invent 2023 - What's new with AWS file storage](#)
- [AWS re:Invent 2023 - Dive deep into Amazon DynamoDB](#)
- [AWS re:Invent 2023 - Building and optimizing a data lake on Amazon S3](#)
- [AWS re:Invent 2023 - What's new with AWS file storage](#)
- [AWS re:Invent 2023 - Dive deep into Amazon DynamoDB](#)
- [Best Practices for Monitoring Redis Workloads on Amazon ElastiCache](#)

Related examples:

- [AWS Dataset Ingestion Metrics Collection Framework](#)

- [Amazon RDS Monitoring Workshop](#)
- [AWS Purpose Built Databases Workshop](#)

PERF03-BP04 Implement strategies to improve query performance in data store

Implement strategies to optimize data and improve data query to enable more scalability and efficient performance for your workload.

Common anti-patterns:

- You do not partition data in your data store.
- You store data in only one file format in your data store.
- You do not use indexes in your data store.

Benefits of establishing this best practice: Optimizing data and query performance results in more efficiency, lower cost, and improved user experience.

Level of risk exposed if this best practice is not established: Medium

Implementation guidance

Data optimization and query tuning are critical aspects of performance efficiency in a data store, as they impact the performance and responsiveness of the entire cloud workload. Unoptimized queries can result in greater resource usage and bottlenecks, which reduce the overall efficiency of a data store.

Data optimization includes several techniques to ensure efficient data storage and access. This also help to improve the query performance in a data store. Key strategies include data partitioning, data compression, and data denormalization, which help data to be optimized for both storage and access.

Implementation steps

- Understand and analyze the critical data queries which are performed in your data store.
- Identify the slow-running queries in your data store and use query plans to understand their current state.
 - [Analyzing the query plan in Amazon Redshift](#)
 - [Using EXPLAIN and EXPLAIN ANALYZE in Athena](#)

- Implement strategies to improve the query performance. Some of the key strategies include:
 - Using a [columnar file format](#) (like Parquet or ORC).
 - Compressing data in the data store to reduce storage space and I/O operation.
 - Data partitioning to split data into smaller parts and reduce data scanning time.
 - [Partitioning data in Athena](#)
 - [Partitions and data distribution](#)
 - Data indexing on the common columns in the query.
 - Use materialized views for frequent queries.
 - [Understanding materialized views](#)
 - [Creating materialized views in Amazon Redshift](#)
 - Choose the right join operation for the query. When you join two tables, specify the larger table on the left side of join and the smaller table on the right side of the join.
 - Distributed caching solution to improve latency and reduce the number of database I/O operation.
 - Regular maintenance such as [vacuuming](#), reindexing, and [running statistics](#).
- Experiment and test strategies in a non-production environment.

Resources

Related documents:

- [Amazon Aurora best practices](#)
- [Amazon Redshift performance](#)
- [Amazon Athena top 10 performance tips](#)
- [AWS Database Caching](#)
- [Best Practices for Implementing Amazon ElastiCache](#)
- [Partitioning data in Athena](#)

Related videos:

- [AWS re:Invent 2023 - AWS storage cost-optimization best practices](#)
- [AWS re:Invent 2022 - Performance monitoring with Amazon RDS and Aurora, featuring Autodesk](#)
- [Optimize Amazon Athena Queries with New Query Analysis Tools](#)

Related examples:

- [AWS Purpose Built Databases Workshop](#)

PERF03-BP05 Implement data access patterns that utilize caching

Implement access patterns that can benefit from caching data for fast retrieval of frequently accessed data.

Common anti-patterns:

- You cache data that changes frequently.
- You rely on cached data as if it is durably stored and always available.
- You don't consider the consistency of your cached data.
- You don't monitor the efficiency of your caching implementation.

Benefits of establishing this best practice: Storing data in a cache can improve read latency, read throughput, user experience, and overall efficiency, as well as reduce costs.

Level of risk exposed if this best practice is not established: Medium

Implementation guidance

A cache is a software or hardware component aimed at storing data so that future requests for the same data can be served faster or more efficiently. The data stored in a cache can be reconstructed if lost by repeating an earlier calculation or fetching it from another data store.

Data caching can be one of the most effective strategies to improve your overall application performance and reduce burden on your underlying primary data sources. Data can be cached at multiple levels in the application, such as within the application making remote calls, known as *client-side caching*, or by using a fast secondary service for storing the data, known as *remote caching*.

Client-side caching

With client-side caching, each client (an application or service that queries the backend datastore) can store the results of their unique queries locally for a specified amount of time. This can reduce the number of requests across the network to a datastore by checking the local client cache first. If the results are not present, the application can then query the datastore and store those results locally. This pattern allows each client to store data in the closest location possible (the client

itself), resulting in the lowest possible latency. Clients can also continue to serve some queries when the backend datastore is unavailable, increasing the availability of the overall system.

One disadvantage of this approach is that when multiple clients are involved, they may store the same cached data locally. This results in both duplicate storage usage and data inconsistency between those clients. One client might cache the results of a query, and one minute later another client can run the same query and get a different result.

Remote caching

To solve the issue of duplicate data between clients, a fast external service, or *remote cache*, can be used to store the queried data. Instead of checking a local data store, each client will check the remote cache before querying the backend datastore. This strategy allows for more consistent responses between clients, better efficiency in stored data, and a higher volume of cached data because the storage space scales independently of clients.

The disadvantage of a remote cache is that the overall system may see a higher latency, as an additional network hop is required to check the remote cache. Client-side caching can be used alongside remote caching for multi-level caching to improve the latency.

Implementation steps

- Identify databases, APIs and network services that could benefit from caching. Services that have heavy read workloads, have a high read-to-write ratio, or are expensive to scale are candidates for caching.
 - [Database Caching](#)
 - [Enabling API caching to enhance responsiveness](#)
- Identify the appropriate type of caching strategy that best fits your access pattern.
 - [Caching strategies](#)
 - [AWS Caching Solutions](#)
- Follow [Caching Best Practices](#) for your data store.
- Configure a cache invalidation strategy, such as a time-to-live (TTL), for all data that balances freshness of data and reducing pressure on backend datastore.
- Enable features such as automatic connection retries, exponential backoff, client-side timeouts, and connection pooling in the client, if available, as they can improve performance and reliability.
 - [Best practices: Redis clients and Amazon ElastiCache \(Redis OSS\)](#)

- Monitor cache hit rate with a goal of 80% or higher. Lower values may indicate insufficient cache size or an access pattern that does not benefit from caching.
 - [Which metrics should I monitor?](#)
 - [Best practices for monitoring Redis workloads on Amazon ElastiCache](#)
 - [Monitoring best practices with Amazon ElastiCache \(Redis OSS\) using Amazon CloudWatch](#)
- Implement [data replication](#) to offload reads to multiple instances and improve data read performance and availability.

Resources

Related documents:

- [Using the Amazon ElastiCache Well-Architected Lens](#)
- [Monitoring best practices with Amazon ElastiCache \(Redis OSS\) using Amazon CloudWatch](#)
- [Which Metrics Should I Monitor?](#)
- [Performance at Scale with Amazon ElastiCache whitepaper](#)
- [Caching challenges and strategies](#)

Related videos:

- [Amazon ElastiCache Learning Path](#)
- [Design for success with Amazon ElastiCache best practices](#)
- [AWS re:Invent 2020 - Design for success with Amazon ElastiCache best practices](#)
- [AWS re:Invent 2023 - \[LAUNCH\] Introducing Amazon ElastiCache Serverless](#)
- [AWS re:Invent 2022 - 5 great ways to reimagine your data layer with Redis](#)
- [AWS re:Invent 2021 - Deep dive on Amazon ElastiCache \(Redis OSS\)](#)

Related examples:

- [Boosting MySQL database performance with Amazon ElastiCache \(Redis OSS\)](#)

Networking and content delivery

Questions

- [PERF 4. How do you select and configure networking resources in your workload?](#)

PERF 4. How do you select and configure networking resources in your workload?

The optimal networking solution for a workload varies based on latency, throughput requirements, jitter, and bandwidth. Physical constraints, such as user or on-premises resources, determine location options. These constraints can be offset with edge locations or resource placement.

Best practices

- [PERF04-BP01 Understand how networking impacts performance](#)
- [PERF04-BP02 Evaluate available networking features](#)
- [PERF04-BP03 Choose appropriate dedicated connectivity or VPN for your workload](#)
- [PERF04-BP04 Use load balancing to distribute traffic across multiple resources](#)
- [PERF04-BP05 Choose network protocols to improve performance](#)
- [PERF04-BP06 Choose your workload's location based on network requirements](#)
- [PERF04-BP07 Optimize network configuration based on metrics](#)

PERF04-BP01 Understand how networking impacts performance

Analyze and understand how network-related decisions impact your workload to provide efficient performance and improved user experience.

Common anti-patterns:

- All traffic flows through your existing data centers.
- You route all traffic through central firewalls instead of using cloud-native network security tools.
- You provision AWS Direct Connect connections without understanding actual usage requirements.
- You don't consider workload characteristics and encryption overhead when defining your networking solutions.
- You use on-premises concepts and strategies for networking solutions in the cloud.

Benefits of establishing this best practice: Understanding how networking impacts workload performance helps you identify potential bottlenecks, improve user experience, increase reliability, and lower operational maintenance as the workload changes.

Level of risk exposed if this best practice is not established: High

Implementation guidance

The network is responsible for the connectivity between application components, cloud services, edge networks, and on-premises data, and therefore it can heavily impact workload performance. In addition to workload performance, user experience can be also impacted by network latency, bandwidth, protocols, location, network congestion, jitter, throughput, and routing rules.

Have a documented list of networking requirements from the workload including latency, packet size, routing rules, protocols, and supporting traffic patterns. Review the available networking solutions and identify which service meets your workload networking characteristics. Cloud-based networks can be quickly rebuilt, so evolving your network architecture over time is necessary to improve performance efficiency.

Implementation steps:

- Define and document networking performance requirements, including metrics such as network latency, bandwidth, protocols, locations, traffic patterns (spikes and frequency), throughput, encryption, inspection, and routing rules.
- Learn about key AWS networking services like [VPCs](#), [AWS Direct Connect](#), [Elastic Load Balancing \(ELB\)](#), and [Amazon Route 53](#).
- Capture the following key networking characteristics:

Characteristics	Tools and metrics
Foundational networking characteristics	• VPC Flow Logs • AWS Transit Gateway Flow Logs • AWS Transit Gateway metrics • AWS PrivateLink metrics
Application networking characteristics	• Elastic Fabric Adapter • AWS App Mesh metrics • Amazon API Gateway metrics

Characteristics	Tools and metrics
Edge networking characteristics	• Amazon CloudFront metrics • Amazon Route 53 metrics • AWS Global Accelerator metrics
Hybrid networking characteristics	• AWS Direct Connect metrics • AWS Site-to-Site VPN metrics • AWS Client VPN metrics • AWS Cloud WAN metrics
Security networking characteristics	• AWS Shield, AWS WAF, and AWS Network Firewall metrics
Tracing characteristics	• AWS X-Ray • VPC Reachability Analyzer • Network Access Analyzer • Amazon Inspector • Amazon CloudWatch RUM

- Benchmark and test network performance:
 - [Benchmark](#) network throughput, as some factors can affect Amazon EC2 network performance when instances are in the same VPC. Measure the network bandwidth between Amazon EC2 Linux instances in the same VPC.
 - Perform [load tests](#) to experiment with networking solutions and options.

Resources

Related documents:

- [Application Load Balancer](#)
- [EC2 Enhanced Networking on Linux](#)
- [EC2 Enhanced Networking on Windows](#)
- [EC2 Placement Groups](#)
- [Enabling Enhanced Networking with the Elastic Network Adapter \(ENA\) on Linux Instances](#)

- [Network Load Balancer](#)
- [Networking Products with AWS](#)
- [Transit Gateway](#)
- [Transitioning to latency-based routing in Amazon Route 53](#)
- [VPC Endpoints](#)

Related videos:

- [AWS re:Invent 2023 - AWS networking foundations](#)
- [AWS re:Invent 2023 - What can networking do for your application?](#)
- [AWS re:Invent 2023 - Advanced VPC designs and new capabilities](#)
- [AWS re:Invent 2023 - A developer's guide to cloud networking](#)
- [AWS re:Invent 2019 - Connectivity to AWS and hybrid AWS network architectures](#)
- [AWS re:Invent 2019 - Optimizing Network Performance for Amazon EC2 Instances](#)
- [AWS Summit Online - Improve Global Network Performance for Applications](#)
- [AWS re:Invent 2020 - Networking best practices and tips with the Well-Architected Framework](#)
- [AWS re:Invent 2020 - AWS networking best practices in large-scale migrations](#)

Related examples:

- [AWS Transit Gateway and Scalable Security Solutions](#)
- [AWS Networking Workshops](#)
- [Hands-on Network Firewall Workshop](#)
- [Observing and Diagnosing your Network on AWS](#)
- [Finding and addressing Network Misconfigurations on AWS](#)

PERF04-BP02 Evaluate available networking features

Evaluate networking features in the cloud that may increase performance. Measure the impact of these features through testing, metrics, and analysis. For example, take advantage of network-level features that are available to reduce latency, network distance, or jitter.

Common anti-patterns:

- You stay within one Region because that is where your headquarters is physically located.
- You use firewalls instead of security groups for filtering traffic.
- You break TLS for traffic inspection rather than relying on security groups, endpoint policies, and other cloud-native functionality.
- You only use subnet-based segmentation instead of security groups.

Benefits of establishing this best practice: Evaluating all service features and options can increase your workload performance, reduce the cost of infrastructure, decrease the effort required to maintain your workload, and increase your overall security posture. You can use the global AWS backbone to provide the optimal networking experience for your customers.

Level of risk exposed if this best practice is not established: High

Implementation guidance

AWS offers services like [AWS Global Accelerator](#) and [Amazon CloudFront](#) that can help improve network performance, while most AWS services have product features (such as the [Amazon S3 Transfer Acceleration](#) feature) to optimize network traffic.

Review which network-related configuration options are available to you and how they could impact your workload. Performance optimization depends on understanding how these options interact with your architecture and the impact that they will have on both measured performance and user experience.

Implementation steps

- Create a list of workload components.
 - Consider using [AWS Cloud WAN](#) to build, manage and monitor your organization's network when building a unified global network.
 - Monitor your global and core networks with [Amazon CloudWatch Logs metrics](#). Leverage [Amazon CloudWatch RUM](#), which provides insights to help to identify, understand, and enhance users' digital experience.
 - View aggregate network latency between AWS Regions and Availability Zones, as well as within each Availability Zone, using [AWS Network Manager](#) to gain insight into how your application performance relates to the performance of the underlying AWS network.
 - Use an existing configuration management database (CMDB) tool or a service such as [AWS Config](#) to create an inventory of your workload and how it's configured.

- If this is an existing workload, identify and document the benchmark for your performance metrics, focusing on the bottlenecks and areas to improve. Performance-related networking metrics will differ per workload based on business requirements and workload characteristics. As a start, these metrics might be important to review for your workload: bandwidth, latency, packet loss, jitter, and retransmits.
- If this is a new workload, perform [load tests](#) to identify performance bottlenecks.
- For the performance bottlenecks you identify, review the configuration options for your solutions to identify performance improvement opportunities. Check out the following key networking options and features:

Improvement opportunity	Solution
Network path or routes	Use Network Access Analyzer to identify paths or routes.
Network protocols	See PERF04-BP05 Choose network protocols to improve performance
Network topology	Evaluate your operational and performance tradeoffs between VPC Peering and AWS Transit Gateway when connecting multiple accounts. AWS Transit Gateway simplifies how you interconnect all of your VPCs, which can span across thousands of AWS accounts and into on-premises networks. Share your AWS Transit Gateway between multiple accounts using AWS Resource Access Manager. See PERF04-BP03 Choose appropriate dedicated connectivity or VPN for your workload
Network services	AWS Global Accelerator is a networking service that improves the performance of your users' traffic by up to 60% using the AWS global network infrastructure.

Improvement opportunity	Solution
	Amazon CloudFront can improve the performance of your workload content delivery and latency globally. Use Lambda@edge to run functions that customize the content that CloudFront delivers closer to the users, reduce latency, and improve performance. Amazon Route 53 offers latency-based routing, geolocation routing, geoproximity routing, and IP-based routing options to help you improve your workload's performance for a global audience. Identify which routing option would optimize your workload performance by reviewing your workload traffic and user location when your workload is distributed globally.
Storage resource features	Amazon S3 Transfer Acceleration is a feature that lets external users benefit from the networking optimizations of CloudFront to upload data to Amazon S3. This improves the ability to transfer large amounts of data from remote locations that don't have dedicated connectivity to the AWS Cloud. Amazon S3 Multi-Region Access Points replicates content to multiple Regions and simplifies the workload by providing one access point. When a Multi-Region Access Point is used, you can request or write data to Amazon S3 with the service identifying the lowest latency bucket.

Improvement opportunity	Solution
Compute resource features	Elastic Network Interfaces (ENI) used by Amazon EC2 instances, containers, and Lambda functions are limited on a per-flow basis. Review your placement groups to optimize your EC2 networking throughput. To avoid a bottleneck on a per flow-basis, design your application to use multiple flows. To monitor and get visibility into your compute related networking metrics, use CloudWatch Metrics and ethtool. The <code>ethtool</code> command is included in the ENA driver and exposes additional network-related metrics that can be published as a custom metric to CloudWatch. Amazon Elastic Network Adapters (ENA) provide further optimization by delivering better throughput for your instances within a cluster placement group. Elastic Fabric Adapter (EFA) is a network interface for Amazon EC2 instances that allows you to run workloads requiring high levels of internode communications at scale on AWS. Amazon EBS-optimized instances use an optimized configuration stack and provide additional, dedicated capacity to increase the Amazon EBS I/O.

Resources

Related documents:

- [Application Load Balancer](#)

- [EC2 Enhanced Networking on Linux](#)
- [EC2 Enhanced Networking on Windows](#)
- [EC2 Placement Groups](#)
- [Enabling Enhanced Networking with the Elastic Network Adapter \(ENA\) on Linux Instances](#)
- [Network Load Balancer](#)
- [Networking Products with AWS](#)
- [Transitioning to Latency-Based Routing in Amazon Route 53](#)
- [VPC Endpoints](#)
- [VPC Flow Logs](#)

Related videos:

- [AWS re:Invent 2023 – Ready for what's next? Designing networks for growth and flexibility](#)
- [AWS re:Invent 2023 – Advanced VPC designs and new capabilities](#)
- [AWS re:Invent 2023 – A developer's guide to cloud networking](#)
- [AWS re:Invent 2022 – Dive deep on AWS networking infrastructure](#)
- [AWS re:Invent 2019 – Connectivity to AWS and hybrid AWS network architectures](#)
- [AWS re:Invent 2018 – Optimizing Network Performance for Amazon EC2 Instances](#)
- [AWS Global Accelerator](#)

Related examples:

- [AWS Transit Gateway and Scalable Security Solutions](#)
- [AWS Networking Workshops](#)
- [Observing and diagnosing your network](#)
- [Finding and addressing network misconfigurations on AWS](#)

PERF04-BP03 Choose appropriate dedicated connectivity or VPN for your workload

When hybrid connectivity is required to connect on-premises and cloud resources, provision adequate bandwidth to meet your performance requirements. Estimate the bandwidth and latency requirements for your hybrid workload. These numbers will drive your sizing requirements.

Common anti-patterns:

- You only evaluate VPN solutions for your network encryption requirements.
- You do not evaluate backup or redundant connectivity options.
- You do not identify all workload requirements (encryption, protocol, bandwidth, and traffic needs).

Benefits of establishing this best practice: Selecting and configuring appropriate connectivity solutions will increase the reliability of your workload and maximize performance. By identifying workload requirements, planning ahead, and evaluating hybrid solutions, you can minimize expensive physical network changes and operational overhead while increasing your time-to-value.

Level of risk exposed if this best practice is not established: High

Implementation guidance

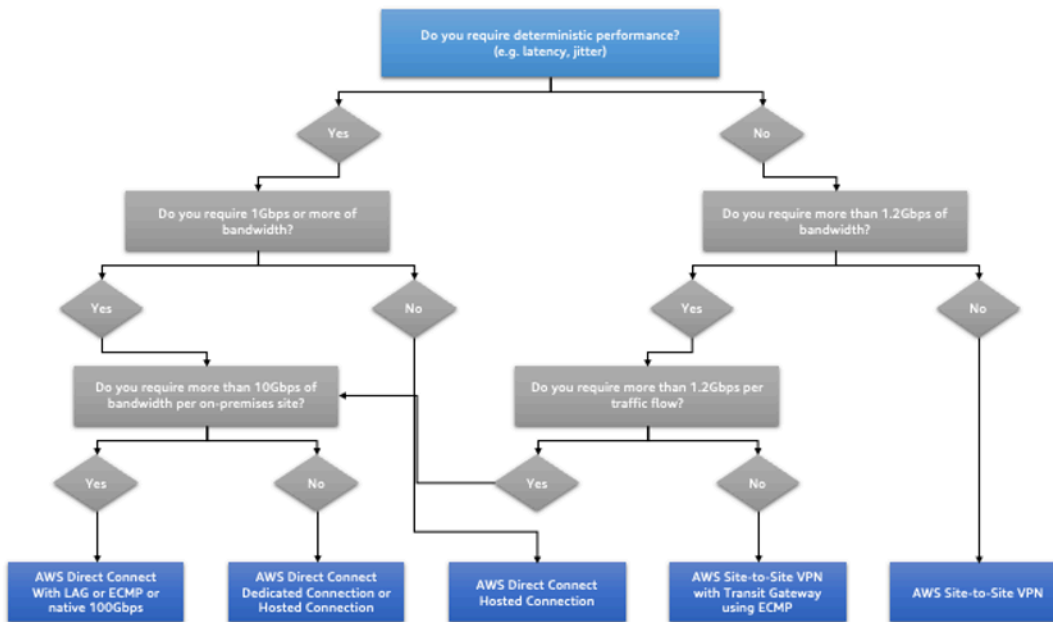
Develop a hybrid networking architecture based on your bandwidth requirements. [AWS Direct Connect](#) allows you to connect your on-premises network privately with AWS. It is suitable when you need high-bandwidth and low-latency while achieving consistent performance. A VPN connection establishes secure connection over the internet. It is used when only a temporary connection is required, when cost is a factor, or as a contingency while waiting for resilient physical network connectivity to be established when using AWS Direct Connect.

If your bandwidth requirements are high, you might consider multiple AWS Direct Connect or VPN services. Traffic can be load balanced across services, although we don't recommend load balancing between AWS Direct Connect and VPN because of the latency and bandwidth differences.

Implementation steps

- Estimate the bandwidth and latency requirements of your existing applications.
 - For existing workloads that are moving to AWS, leverage the data from your internal network monitoring systems.
 - For new or existing workloads for which you don't have monitoring data, consult with the product owners to determine adequate performance metrics and provide a good user experience.
- Select dedicated connection or VPN as your connectivity option. Based on all workload requirements (encryption, bandwidth, and traffic needs), you can either choose AWS Direct Connect or [AWS VPN](#) (or both). The following diagram can help you choose the appropriate connection type.

- [AWS Direct Connect](#) provides dedicated connectivity to the AWS environment, from 50 Mbps up to 100 Gbps, using either dedicated connections or hosted connections. This gives you managed and controlled latency and provisioned bandwidth so your workload can connect efficiently to other environments. Using AWS Direct Connect partners, you can have end-to-end connectivity from multiple environments, providing an extended network with consistent performance. AWS offers scaling direct connect connection bandwidth using either native 100 Gbps, link aggregation group (LAG), or BGP equal-cost multipath (ECMP).
- The AWS [Site-to-Site VPN](#) provides a managed VPN service supporting internet protocol security (IPsec). When a VPN connection is created, each VPN connection includes two tunnels for high availability.
- Follow AWS documentation to choose an appropriate connectivity option:
 - If you decide to use AWS Direct Connect, select the appropriate bandwidth for your connectivity.
 - If you are using an AWS Site-to-Site VPN across multiple locations to connect to an AWS Region, use an [accelerated Site-to-Site VPN connection](#) for the opportunity to improve network performance.
 - If your network design consists of IPsec VPN connection over [AWS Direct Connect](#), consider using Private IP VPN to improve security and achieve segmentation. [AWS Site-to-Site Private IP VPN](#) is deployed on top of transit virtual interface (VIF).
 - [AWS Direct Connect SiteLink](#) allows creating low-latency and redundant connections between your data centers worldwide by sending data over the fastest path between [AWS Direct Connect locations](#), bypassing AWS Regions.
- Validate your connectivity setup before deploying to production. Perform security and performance testing to assure it meets your bandwidth, reliability, latency, and compliance requirements.
- Regularly monitor your connectivity performance and usage and optimize if required.



Deterministic performance flowchart

Resources

Related documents:

- [Networking Products with AWS](#)
- [AWS Transit Gateway](#)
- [VPC Endpoints](#)
- [Building a Scalable and Secure Multi-VPC AWS Network Infrastructure](#)
- [Client VPN](#)

Related videos:

- [AWS re:Invent 2023 – Building hybrid network connectivity with AWS](#)
- [AWS re:Invent 2023 – Secure remote connectivity to AWS](#)
- [AWS re:Invent 2022 – Optimizing performance with Amazon CloudFront](#)
- [AWS re:Invent 2019 – Connectivity to AWS and hybrid AWS network architectures](#)
- [AWS re:Invent 2020 – AWS Transit Gateway Connect](#)

Related examples:

- [AWS Transit Gateway and Scalable Security Solutions](#)
- [AWS Networking Workshops](#)

PERF04-BP04 Use load balancing to distribute traffic across multiple resources

Distribute traffic across multiple resources or services to allow your workload to take advantage of the elasticity that the cloud provides. You can also use load balancing for offloading encryption termination to improve performance, reliability and manage and route traffic effectively.

Common anti-patterns:

- You don't consider your workload requirements when choosing the load balancer type.
- You don't leverage the load balancer features for performance optimization.
- The workload is exposed directly to the internet without a load balancer.
- You route all internet traffic through existing load balancers.
- You use generic TCP load balancing and making each compute node handle SSL encryption.

Benefits of establishing this best practice: A load balancer handles the varying load of your application traffic in a single Availability Zone or across multiple Availability Zones and enables high availability, automatic scaling, and better utilization for your workload.

Level of risk exposed if this best practice is not established: High

Implementation guidance

Load balancers act as the entry point for your workload, from which point they distribute the traffic to your backend targets, such as compute instances or containers, to improve utilization.

Choosing the right load balancer type is the first step to optimize your architecture. Start by listing your workload characteristics, such as protocol (like TCP, HTTP, TLS, or WebSockets), target type (like instances, containers, or serverless), application requirements (like long running connections, user authentication, or stickiness), and placement (like Region, Local Zone, Outpost, or zonal isolation).

AWS provides multiple models for your applications to use load balancing. [Application Load Balancer](#) is best suited for load balancing of HTTP and HTTPS traffic and provides advanced request routing targeted at the delivery of modern application architectures, including microservices and containers.

[Network Load Balancer](#) is best suited for load balancing of TCP traffic where extreme performance is required. It is capable of handling millions of requests per second while maintaining ultra-low latencies, and it is optimized to handle sudden and volatile traffic patterns.

[Elastic Load Balancing](#) provides integrated certificate management and SSL/TLS decryption, allowing you the flexibility to centrally manage the SSL settings of the load balancer and offload CPU intensive work from your workload.

After choosing the right load balancer, you can start leveraging its features to reduce the amount of effort your backend has to do to serve the traffic.

For example, using both Application Load Balancer (ALB) and Network Load Balancer (NLB), you can perform SSL/TLS encryption offloading, which is an opportunity to avoid the CPU-intensive TLS handshake from being completed by your targets and also to improve certificate management.

When you configure SSL/TLS offloading in your load balancer, it becomes responsible for the encryption of the traffic from and to clients while delivering the traffic unencrypted to your backends, freeing up your backend resources and improving the response time for the clients.

Application Load Balancer can also serve HTTP/2 traffic without needing to support it on your targets. This simple decision can improve your application response time, as HTTP/2 uses TCP connections more efficiently.

Your workload latency requirements should be considered when defining the architecture. As an example, if you have a latency-sensitive application, you may decide to use Network Load Balancer, which offers extremely low latencies. Alternatively, you may decide to bring your workload closer to your customers by leveraging Application Load Balancer in [AWS Local Zones](#) or even [AWS Outposts](#).

Another consideration for latency-sensitive workloads is cross-zone load balancing. With cross-zone load balancing, each load balancer node distributes traffic across the registered targets in all allowed Availability Zones.

Use Auto Scaling integrated with your load balancer. One of the key aspects of a performance efficient system has to do with right-sizing your backend resources. To do this, you can leverage load balancer integrations for backend target resources. Using the load balancer integration with Auto Scaling groups, targets will be added or removed from the load balancer as required in response to incoming traffic. Load balancers can also integrate with [Amazon ECS](#) and [Amazon EKS](#) for containerized workloads.

- [Amazon ECS - Service load balancing](#)

- [Application load balancing on Amazon EKS](#)
- [Network load balancing on Amazon EKS](#)

Implementation steps

- Define your load balancing requirements including traffic volume, availability and application scalability.
- Choose the right load balancer type for your application.
 - Use Application Load Balancer for HTTP/HTTPS workloads.
 - Use Network Load Balancer for non-HTTP workloads that run on TCP or UDP.
 - Use a combination of both ([ALB as a target of NLB](#)) if you want to leverage features of both products. For example, you can do this if you want to use the static IPs of NLB together with HTTP header based routing from ALB, or if you want to expose your HTTP workload to an [AWS PrivateLink](#).
- For a full comparison of load balancers, see [ELB product comparison](#).
- Use SSL/TLS offloading if possible.
 - Configure HTTPS/TLS listeners with both [Application Load Balancer](#) and [Network Load Balancer](#) integrated with [AWS Certificate Manager](#).
 - Note that some workloads may require end-to-end encryption for compliance reasons. In this case, it is a requirement to allow encryption at the targets.
 - For security best practices, see [SEC09-BP02 Enforce encryption in transit](#).
- Select the right routing algorithm (only ALB).
 - The routing algorithm can make a difference in how well-used your backend targets are and therefore how they impact performance. For example, ALB provides [two options for routing algorithms](#):
 - **Least outstanding requests:** Use to achieve a better load distribution to your backend targets for cases when the requests for your application vary in complexity or your targets vary in processing capability.
 - **Round robin:** Use when the requests and targets are similar, or if you need to distribute requests equally among targets.
- Consider cross-zone or zonal isolation.
 - Use cross-zone turned off (zonal isolation) for latency improvements and zonal failure domains. It is turned off by default in NLB and in [ALB you can turn it off per target group](#).

- Use cross-zone turned on for increased availability and flexibility. By default, cross-zone is turned on for ALB and in [NLB you can turn it on per target group](#).
- Turn on HTTP keep-alives for your HTTP workloads (only ALB). With this feature, the load balancer can reuse backend connections until the keep-alive timeout expires, improving your HTTP request and response time and also reducing resource utilization on your backend targets. For detail on how to do this for Apache and Nginx, see [What are the optimal settings for using Apache or NGINX as a backend server for ELB?](#)
- Turn on monitoring for your load balancer.
 - Turn on access logs for your [Application Load Balancer](#) and [Network Load Balancer](#).
 - The main fields to consider for ALB are `request_processing_time`, `request_processing_time`, and `response_processing_time`.
 - The main fields to consider for NLB are `connection_time` and `tls_handshake_time`.
 - Be ready to query the logs when you need them. You can use Amazon Athena to query both [ALB logs](#) and [NLB logs](#).
 - Create alarms for performance related metrics such as [TargetResponseTime for ALB](#).

Resources

Related documents:

- [ELB product comparison](#)
- [AWS Global Infrastructure](#)
- [Improving Performance and Reducing Cost Using Availability Zone Affinity](#)
- [Step by step for Log Analysis with Amazon Athena](#)
- [Querying Application Load Balancer logs](#)
- [Monitor your Application Load Balancers](#)
- [Monitor your Network Load Balancer](#)
- [Use Elastic Load Balancing to distribute traffic across the instances in your Auto Scaling group](#)

Related videos:

- [AWS re:Invent 2023: What can networking do for your application?](#)
- [AWS re:Inforce 20: How to use Elastic Load Balancing to enhance your security posture at scale](#)

- [AWS re:Invent 2018: Elastic Load Balancing: Deep Dive and Best Practices](#)
- [AWS re:Invent 2021 - How to choose the right load balancer for your AWS workloads](#)
- [AWS re:Invent 2019: Get the most from Elastic Load Balancing for different workloads](#)

Related examples:

- [Gateway Load Balancer](#)
- [CDK and AWS CloudFormation samples for Log Analysis with Amazon Athena](#)

PERF04-BP05 Choose network protocols to improve performance

Make decisions about protocols for communication between systems and networks based on the impact to the workload's performance.

There is a relationship between latency and bandwidth to achieve throughput. If your file transfer is using Transmission Control Protocol (TCP), higher latencies will most likely reduce overall throughput. There are approaches to fix this with TCP tuning and optimized transfer protocols, but one solution is to use User Datagram Protocol (UDP).

Common anti-patterns:

- You use TCP for all workloads regardless of performance requirements.

Benefits of establishing this best practice: Verifying that an appropriate protocol is used for communication between users and workload components helps improve overall user experience for your applications. For instance, connection-less UDP allows for high speed, but it doesn't offer retransmission or high reliability. TCP is a full featured protocol, but it requires greater overhead for processing the packets.

Level of risk exposed if this best practice is not established: Medium

Implementation guidance

If you have the ability to choose different protocols for your application and you have expertise in this area, optimize your application and end-user experience by using a different protocol. Note that this approach comes with significant difficulty and should only be attempted if you have optimized your application in other ways first.

A primary consideration for improving your workload's performance is to understand the latency and throughput requirements, and then choose network protocols that optimize performance.

When to consider using TCP

TCP provides reliable data delivery, and can be used for communication between workload components where reliability and guaranteed delivery of data is important. Many web-based applications rely on TCP-based protocols, such as HTTP and HTTPS, to open TCP sockets for communication between application components. Email and file data transfer are common applications that also make use of TCP, as it is a simple and reliable transfer mechanism between application components. Using TLS with TCP can add some overhead to the communication, which can result in increased latency and reduced throughput, but it comes with the advantage of security. The overhead comes mainly from the added overhead of the handshake process, which can take several round-trips to complete. Once the handshake is complete, the overhead of encrypting and decrypting data is relatively small.

When to consider using UDP

UDP is a connection-less-oriented protocol and is therefore suitable for applications that need fast, efficient transmission, such as log, monitoring, and VoIP data. Also, consider using UDP if you have workload components that respond to small queries from large numbers of clients to ensure optimal performance of the workload. Datagram Transport Layer Security (DTLS) is the UDP equivalent of Transport Layer Security (TLS). When using DTLS with UDP, the overhead comes from encrypting and decrypting the data, as the handshake process is simplified. DTLS also adds a small amount of overhead to the UDP packets, as it includes additional fields to indicate the security parameters and to detect tampering.

When to consider using SRD

Scalable reliable datagram (SRD) is a network transport protocol optimized for high-throughput workloads due to its ability to load-balance traffic across multiple paths and quickly recover from packet drops or link failures. SRD is therefore best used for high performance computing (HPC) workloads that require high throughput and low latency communication between compute nodes. This might include parallel processing tasks such as simulation, modeling, and data analysis that involve a large amount of data transfer between nodes.

Implementation steps

- Use the [AWS Global Accelerator](#) and [AWS Transfer Family](#) services to improve the throughput of your online file transfer applications. The AWS Global Accelerator service helps you achieve

lower latency between your client devices and your workload on AWS. With AWS Transfer Family, you can use TCP-based protocols such as Secure Shell File Transfer Protocol (SFTP) and File Transfer Protocol over SSL (FTPS) to securely scale and manage your file transfers to AWS storage services.

- Use network latency to determine if TCP is appropriate for communication between workload components. If the network latency between your client application and server is high, then the TCP three-way handshake can take some time, thereby impacting on the responsiveness of your application. Metrics such as time to first byte (TTFB) and round-trip time (RTT) can be used to measure network latency. If your workload serves dynamic content to users, consider using [Amazon CloudFront](#), which establishes a persistent connection to each origin for dynamic content to remove the connection setup time that would otherwise slow down each client request.
- Using TLS with TCP or UDP can result in increased latency and reduced throughput for your workload due to the impact of encryption and decryption. For such workloads, consider SSL/TLS offloading on [Elastic Load Balancing](#) to improve workload performance by allowing the load balancer to handle SSL/TLS encryption and decryption process instead of having backend instances do it. This can help reduce the CPU utilization on the backend instances, which can improve performance and increase capacity.
- Use the [Network Load Balancer \(NLB\)](#) to deploy services that rely on the UDP protocol, such as authentication and authorization, logging, DNS, IoT, and streaming media, to improve the performance and reliability of your workload. The NLB distributes incoming UDP traffic across multiple targets, allowing you to scale your workload horizontally, increase capacity, and reduce the overhead of a single target.
- For your High Performance Computing (HPC) workloads, consider using the [Elastic Network Adapter \(ENA\) Express](#) functionality that uses the SRD protocol to improve network performance by providing a higher single flow bandwidth (25 Gbps) and lower tail latency (99.9 percentile) for network traffic between EC2 instances.
- Use the [Application Load Balancer \(ALB\)](#) to route and load balance your gRPC (Remote Procedure Calls) traffic between workload components or between gRPC clients and services. gRPC uses the TCP-based HTTP/2 protocol for transport and it provides performance benefits such as lighter network footprint, compression, efficient binary serialization, support for numerous languages, and bi-directional streaming.

Resources

Related documents:

- [How to route UDP traffic into Kubernetes](#)
- [Application Load Balancer](#)
- [EC2 Enhanced Networking on Linux](#)
- [EC2 Enhanced Networking on Windows](#)
- [EC2 Placement Groups](#)
- [Enabling Enhanced Networking with the Elastic Network Adapter \(ENA\) on Linux Instances](#)
- [Network Load Balancer](#)
- [Networking Products with AWS](#)
- [Transitioning to Latency-Based Routing in Amazon Route 53](#)
- [VPC Endpoints](#)

Related videos:

- [AWS re:Invent 2022 – Scaling network performance on next-gen Amazon Elastic Compute Cloud instances](#)
- [AWS re:Invent 2022 – Application networking foundations](#)

Related examples:

- [AWS Transit Gateway and Scalable Security Solutions](#)
- [AWS Networking Workshops](#)

PERF04-BP06 Choose your workload's location based on network requirements

Evaluate options for resource placement to reduce network latency and improve throughput, providing an optimal user experience by reducing page load and data transfer times.

Common anti-patterns:

- You consolidate all workload resources into one geographic location.
- You chose the closest Region to your location but not to the workload end user.

Benefits of establishing this best practice: User experience is greatly affected by the latency between the user and your application. By using appropriate AWS Regions and the AWS private global network, you can reduce latency and deliver a better experience to remote users.

Level of risk exposed if this best practice is not established: Medium**Implementation guidance**

Resources, such as Amazon EC2 instances, are placed into Availability Zones within [AWS Regions](#), [AWS Local Zones](#), [AWS Outposts](#), or [AWS Wavelength](#) zones. Selection of this location influences network latency and throughput from a given user location. Edge services like [Amazon CloudFront](#) and [AWS Global Accelerator](#) can also be used to improve network performance by either caching content at edge locations or providing users with an optimal path to the workload through the AWS global network.

Amazon EC2 provides placement groups for networking. A placement group is a logical grouping of instances to decrease latency. Using placement groups with supported instance types and an Elastic Network Adapter (ENA) enables workloads to participate in a low-latency, reduced jitter 25 Gbps network. Placement groups are recommended for workloads that benefit from low network latency, high network throughput, or both.

Latency-sensitive services are delivered at edge locations using AWS global network, such as [Amazon CloudFront](#). These edge locations commonly provide services like content delivery network (CDN) and domain name system (DNS). By having these services at the edge, workloads can respond with low latency to requests for content or DNS resolution. These services also provide geographic services, such as geotargeting of content (providing different content based on the end users' location) or latency-based routing to direct end users to the nearest Region (minimum latency).

Use edge services to reduce latency and to enable content caching. Configure cache control correctly for both DNS and HTTP/HTTPS to gain the most benefit from these approaches.

Implementation steps

- Capture information about the IP traffic going to and from network interfaces.
 - [Logging IP traffic using VPC Flow Logs](#)
 - [How the client IP address is preserved in AWS Global Accelerator](#)
- Analyze network access patterns in your workload to identify how users use your application.
 - Use monitoring tools, such as [Amazon CloudWatch](#) and [AWS CloudTrail](#), to gather data on network activities.
 - Analyze the data to identify the network access pattern.
- Select Regions for your workload deployment based on the following key elements:

- **Where your data is located:** For data-heavy applications (such as big data and machine learning), application code should run as close to the data as possible.
- **Where your users are located:** For user-facing applications, choose a Region (or Regions) close to your workload's users.
- **Other constraints:** Consider constraints such as cost and compliance as explained in [What to Consider when Selecting a Region for your Workloads](#).
- Use [AWS Local Zones](#) to run workloads like video rendering. Local Zones allow you to benefit from having compute and storage resources closer to end users.
- Use [AWS Outposts](#) for workloads that need to remain on-premises and where you want that workload to run seamlessly with the rest of your other workloads in AWS.
- Applications like high-resolution live video streaming, high-fidelity audio, and augmented reality or virtual reality (AR/VR) require ultra-low-latency for 5G devices. For such applications, consider [AWS Wavelength](#). AWS Wavelength embeds AWS compute and storage services within 5G networks, providing mobile edge computing infrastructure for developing, deploying, and scaling ultra-low-latency applications.
- Use local caching or [AWS Caching Solutions](#) for frequently used assets to improve performance, reduce data movement, and lower environmental impact.

Service	When to use
Amazon CloudFront	Use to cache static content such as images, scripts, and videos, as well as dynamic content such as API responses or web applications.
Amazon ElastiCache	Use to cache content for web applications.
DynamoDB Accelerator	Use to add in-memory acceleration to your DynamoDB tables.

- Use services that can help you run code closer to users of your workload like the following:

Service	When to use
Lambda@edge	Use for compute-heavy operations that are initiated when objects are not in the cache.

Service	When to use
Amazon CloudFront Functions	Use for simple use cases like HTTP(s) requests or response manipulations that can be initiated by short-lived functions.
AWS IoT Greengrass	Use to run local compute, messaging, and data caching for connected devices.

- Some applications require fixed entry points or higher performance by reducing first byte latency and jitter, and increasing throughput. These applications can benefit from networking services that provide static anycast IP addresses and TCP termination at edge locations. [AWS Global Accelerator](#) can improve performance for your applications by up to 60% and provide quick failover for multi-region architectures. AWS Global Accelerator provides you with static anycast IP addresses that serve as a fixed entry point for your applications hosted in one or more AWS Regions. These IP addresses permit traffic to ingress onto the AWS global network as close to your users as possible. AWS Global Accelerator reduces the initial connection setup time by establishing a TCP connection between the client and the AWS edge location closest to the client. Review the use of AWS Global Accelerator to improve the performance of your TCP/UDP workloads and provide quick failover for multi-Region architectures.

Resources

Related best practices:

- [COST07-BP02 Implement Regions based on cost](#)
- [COST08-BP03 Implement services to reduce data transfer costs](#)
- [REL10-BP01 Deploy the workload to multiple locations](#)
- [REL10-BP02 Select the appropriate locations for your multi-location deployment](#)
- [SUS01-BP01 Choose Region based on both business requirements and sustainability goals](#)
- [SUS02-BP04 Optimize geographic placement of workloads based on their networking requirements](#)
- [SUS04-BP07 Minimize data movement across networks](#)

Related documents:

- [AWS Global Infrastructure](#)
- [AWS Local Zones and AWS Outposts, choosing the right technology for your edge workload](#)
- [Placement groups](#)
- [AWS Local Zones](#)
- [AWS Outposts](#)
- [AWS Wavelength](#)
- [Amazon CloudFront](#)
- [AWS Global Accelerator](#)
- [AWS Direct Connect](#)
- [AWS Site-to-Site VPN](#)
- [Amazon Route 53](#)

Related videos:

- [AWS Local Zones Explainer Video](#)
- [AWS Outposts: Overview and How it Works](#)
- [AWS re:Invent 2023 - A migration strategy for edge and on-premises workloads](#)
- [AWS re:Invent 2021 - AWS Outposts: Bringing the AWS experience on premises](#)
- [AWS re:Invent 2020: AWS Wavelength: Run apps with ultra-low latency at 5G edge](#)
- [AWS re:Invent 2022 - AWS Local Zones: Building applications for a distributed edge](#)
- [AWS re:Invent 2021 - Building low-latency websites with Amazon CloudFront](#)
- [AWS re:Invent 2022 - Improve performance and availability with AWS Global Accelerator](#)
- [AWS re:Invent 2022 - Build your global wide area network using AWS](#)
- [AWS re:Invent 2020: Global traffic management with Amazon Route 53](#)

Related examples:

- [AWS Global Accelerator Custom Routing Workshop](#)
- [Handling Rewrites and Redirects using Edge Functions](#)

PERF04-BP07 Optimize network configuration based on metrics

Use collected and analyzed data to make informed decisions about optimizing your network configuration.

Common anti-patterns:

- You assume that all performance-related issues are application-related.
- You only test your network performance from a location close to where you have deployed the workload.
- You use default configurations for all network services.
- You overprovision the network resource to provide sufficient capacity.

Benefits of establishing this best practice: Collecting necessary metrics of your AWS network and implementing network monitoring tools allows you to understand network performance and optimize network configurations.

Level of risk exposed if this best practice is not established: Low

Implementation guidance

Monitoring traffic to and from VPCs, subnets, or network interfaces is crucial to understand how to utilize AWS network resources and optimize network configurations. By using the following AWS networking tools, you can further inspect information about the traffic usage, network access and logs.

Implementation steps

- Identify the key performance metrics such as latency or packet loss to collect. AWS provides several tools that can help you to collect these metrics. By using the following tools, you can further inspect information about the traffic usage, network access, and logs:

AWS tool	Where to use
Amazon VPC IP Address Manager.	Use IPAM to plan, track, and monitor IP addresses for your AWS and on-premises workloads. This is a best practice to optimize IP address usage and allocation.

AWS tool	Where to use
VPC Flow logs	Use VPC Flow Logs to capture detailed information about traffic to and from network interfaces in your VPCs. With VPC Flow Logs, you can diagnose overly restrictive or permissive security group rules and determine the direction of the traffic to and from the network interfaces.
AWS Transit Gateway Flow Logs	Use AWS Transit Gateway Flow Logs to capture information about the IP traffic going to and from your transit gateways.
DNS query logging	Log information about public or private DNS queries Route 53 receives. With DNS logs, you can optimize DNS configurations by understanding the domain or subdomain that was requested or Route 53 EDGE locations that responded to DNS queries.
Reachability Analyzer	Reachability Analyzer helps you analyze and debug network reachability. Reachability Analyzer is a configuration analysis tool that allows you to perform connectivity testing between a source resource and a destination resource in your VPCs. This tool helps you verify that your network configuration matches your intended connectivity.

AWS tool	Where to use
Network Access Analyzer	Network Access Analyzer helps you understand network access to your resources. You can use Network Access Analyzer to specify your network access requirements and identify potential network paths that do not meet your specified requirements. By optimizing your corresponding network configuration, you can understand and verify the state of your network and demonstrate if your network on AWS meets your compliance requirements.
Amazon CloudWatch	Use Amazon CloudWatch and turn on the appropriate metrics for network options. Make sure to choose the right network metric for your workload. For example, you can turn on metrics for VPC Network Address Usage, VPC NAT Gateway, AWS Transit Gateway, VPN tunnel, AWS Network Firewall, Elastic Load Balancing, and AWS Direct Connect. Continually monitoring metrics is a good practice to observe and understand your network status and usage, which helps you optimize network configuration based on your observations.

AWS tool	Where to use
AWS Network Manager	Using AWS Network Manager, you can monitor the real-time and historical performance of the AWS Global Network for operational and planning purposes. Network Manager provides aggregate network latency between AWS Regions and Availability Zones and within each Availability Zone, allowing you to better understand how your application performance relates to the performance of the underlying AWS network.
Amazon CloudWatch RUM	Use Amazon CloudWatch RUM to collect the metrics that give you the insights that help you identify, understand, and improve user experience.

- Identify top talkers and application traffic patterns using VPC and AWS Transit Gateway Flow Logs.
- Assess and optimize your current network architecture including VPCs, subnets, and routing. As an example, you can evaluate how different VPC peering or AWS Transit Gateway can help you improve the networking in your architecture.
- Assess the routing paths in your network to verify that the shortest path between destinations is always used. Network Access Analyzer can help you do this.

Resources

Related documents:

- [Public DNS query logging](#)
- [What is IPAM?](#)
- [What is Reachability Analyzer?](#)
- [What is Network Access Analyzer?](#)
- [CloudWatch metrics for your VPCs](#)

- [Optimize performance and reduce costs for network analytics with VPC Flow Logs in Apache Parquet format](#)
- [Monitoring your global and core networks with Amazon CloudWatch metrics](#)
- [Continuously monitor network traffic and resources](#)

Related videos:

- [AWS re:Invent 2023 – A developer's guide to cloud networking](#)
- [AWS re:Invent 2023 – Ready for what's next? Designing networks for growth and flexibility](#)
- [AWS re:Invent 2023 – Advanced VPC designs and new capabilities](#)
- [AWS re:Invent 2022 – Dive deep on AWS networking infrastructure](#)
- [AWS re:Invent 2020 – Networking best practices and tips with the AWS Well-Architected Framework](#)
- [AWS re:Invent 2020 – Monitoring and troubleshooting network traffic](#)

Related examples:

- [AWS Networking Workshops](#)
- [AWS Network Monitoring](#)
- [Observing and diagnosing your network on AWS](#)
- [Finding and addressing network misconfigurations on AWS](#)

Process and culture

Questions

- [PERF 5. How do your organizational practices and culture contribute to performance efficiency in your workload?](#)

PERF 5. How do your organizational practices and culture contribute to performance efficiency in your workload?

When architecting workloads, there are principles and practices that you can adopt to help you better run efficient high-performing cloud workloads. To adopt a culture that fosters performance efficiency of cloud workloads, consider these key principles and practices:

Best practices

- [PERF05-BP01 Establish key performance indicators \(KPIs\) to measure workload health and performance](#)
- [PERF05-BP02 Use monitoring solutions to understand the areas where performance is most critical](#)
- [PERF05-BP03 Define a process to improve workload performance](#)
- [PERF05-BP04 Load test your workload](#)
- [PERF05-BP05 Use automation to proactively remediate performance-related issues](#)
- [PERF05-BP06 Keep your workload and services up-to-date](#)
- [PERF05-BP07 Review metrics at regular intervals](#)

PERF05-BP01 Establish key performance indicators (KPIs) to measure workload health and performance

Identify the KPIs that quantitatively and qualitatively measure workload performance. KPIs help you measure the health and performance of a workload related to a business goal.

Common anti-patterns:

- You only monitor system-level metrics to gain insight into your workload and don't understand business impacts to those metrics.
- You assume that your KPIs are already being published and shared as standard metric data.
- You do not define a quantitative, measurable KPI.
- You do not align KPIs with business goals or strategies.

Benefits of establishing this best practice: Identifying specific KPIs that represent workload health and performance helps align teams on their priorities and define successful business outcomes. Sharing those metrics with all departments provides visibility and alignment on thresholds, expectations, and business impact.

Level of risk exposed if this best practice is not established: High

Implementation guidance

KPIs allow business and engineering teams to align on the measurement of goals and strategies and how these factors combine to produce business outcomes. For example, a website workload

might use page load time as an indication of overall performance. This metric would be one of multiple data points that measures user experience. In addition to identifying the page load time thresholds, you should document the expected outcome or business risk if ideal performance is not met. A long page load time affects your end users directly, decreases their user experience rating, and can lead to a loss of customers. When you define your KPI thresholds, combine both industry benchmarks and your end user expectations. For example, if the current industry benchmark is a webpage loading within a two-second time period, but your end users expect a webpage to load within a one-second time period, then you should take both of these data points into consideration when establishing the KPI.

Your team must evaluate your workload KPIs using real-time granular data and historical data for reference and create dashboards that perform metric math on your KPI data to derive operational and utilization insights. KPIs should be documented and include thresholds that support business goals and strategies, and should be mapped to metrics being monitored. KPIs should be revisited when business goals, strategies, or end user requirements change.

Implementation steps

- **Identify stakeholders:** Identify and document key business stakeholders, including development and operation teams.
- **Define objectives:** Work with these stakeholders to define and document objectives of your workload. Consider the critical performance aspects of your workloads, such as throughput, response time, and cost, as well as business goals, such as user satisfaction.
- **Review industry best practices:** Review industry best practices to identify relevant KPIs aligned with your workload objectives.
- **Identify metrics:** Identify metrics that are aligned with your workload objectives and can help you measure performance and business goals. Establish KPIs based on these metrics. Example metrics are measurements like average response time or number of concurrent users.
- **Define and document KPIs:** Use industry best practices and your workload objectives to set targets for your workload KPI. Use this information to set KPI thresholds for severity or alarm level. Identify and document the risk and impact of a KPI is not met.
- **Implement monitoring:** Use monitoring tools such as [Amazon CloudWatch](#) or [AWS Config](#) to collect metrics and measure KPIs.
- **Visually communicate KPIs:** Use dashboard tools like [Amazon Quick Suite](#) to visualize and communicate KPIs with stakeholders.

- **Analyze and optimize:** Regularly review and analyze KPIs to identify areas of your workload that need to be improved. Work with stakeholders to implement these improvements.
- **Revisit and refine:** Regularly review metrics and KPIs to assess their effectiveness, especially when business goals or workload performance change.

Resources

Related documents:

- [CloudWatch documentation](#)
- [Monitoring, Logging, and Performance AWS Partners](#)
- [AWS observability tools](#)
- [The Importance of Key Performance Indicators \(KPIs\) for Large-Scale Cloud Migrations](#)
- [How to track your cost optimization KPIs with the KPI Dashboard](#)
- [X-Ray Documentation](#)
- [Using Amazon CloudWatch dashboards](#)
- [Quick Suite KPIs](#)

Related videos:

- [AWS re:Invent 2023 - Optimize cost and performance and track progress toward mitigation](#)
- [AWS re:Invent 2023 - Manage resource lifecycle events at scale with AWS Health](#)
- [AWS re:Invent 2023 - Performance & efficiency at Pinterest: Optimizing the latest instances](#)
- [AWS re:Invent 2022 - AWS optimization: Actionable steps for immediate results](#)
- [AWS re:Invent 2023 - Building an effective observability strategy](#)
- [AWS Summit SF 2022 - Full-stack observability and application monitoring with AWS](#)
- [AWS re:Invent 2023 - Scaling on AWS for the first 10 million users](#)
- [AWS re:Invent 2022 - How Amazon uses better metrics for improved website performance](#)
- [Creating an Effective Metrics Strategy for Your Business | AWS Events](#)

Related examples:

- [Creating a dashboard with Quick Suite](#)

PERF05-BP02 Use monitoring solutions to understand the areas where performance is most critical

Understand and identify areas where increasing the performance of your workload will have a positive impact on efficiency or customer experience. For example, a website that has a large amount of customer interaction can benefit from using edge services to move content delivery closer to customers.

Common anti-patterns:

- You assume that standard compute metrics such as CPU utilization or memory pressure are enough to catch performance issues.
- You only use the default metrics recorded by your selected monitoring software.
- You only review metrics when there is an issue.

Benefits of establishing this best practice: Understanding critical areas of performance helps workload owners monitor KPIs and prioritize high-impact improvements.

Level of risk exposed if this best practice is not established: High

Implementation guidance

Set up end-to-end tracing to identify traffic patterns, latency, and critical performance areas. Monitor your data access patterns for slow queries or poorly fragmented and partitioned data. Identify the constrained areas of the workload using load testing or monitoring.

Increase performance efficiency by understanding your architecture, traffic patterns, and data access patterns, and identify your latency and processing times. Identify the potential bottlenecks that might affect the customer experience as the workload grows. After investigating these areas, look at which solution you could deploy to remove those performance concerns.

Implementation steps

- Set up end-to-end monitoring to capture all workload components and metrics. Here are examples of monitoring solutions on AWS.

Service	Where to use
Amazon CloudWatch Real-User Monitoring (RUM)	To capture application performance metrics from real user client-side and frontend sessions.
AWS X-Ray	To trace traffic through the application layers and identify latency between components and dependencies. Use X-Ray service maps to see relationships and latency between workload components.
Amazon Relational Database Service Performance Insights	To view database performance metrics and identify performance improvements.
Amazon RDS Enhanced Monitoring	To view database OS performance metrics.
Amazon DevOps Guru	To detect abnormal operating patterns so you can identify operational issues before they impact your customers.

- Perform tests to generate metrics, identify traffic patterns, bottlenecks, and critical performance areas. Here are some examples of how to perform testing:
 - Set up [CloudWatch Synthetic Canaries](#) to mimic browser-based user activities programmatically using Linux cron jobs or rate expressions to generate consistent metrics over time.
 - Use the [AWS Distributed Load Testing](#) solution to generate peak traffic or test the workload at the expected growth rate.
- Evaluate the metrics and telemetry to identify your critical performance areas. Review these areas with your team to discuss monitoring and solutions to avoid bottlenecks.
- Experiment with performance improvements and measure those changes with data. As an example, you can use [CloudWatch Evidently](#) to test new improvements and performance impacts to your workload.

Resources

Related documents:

- [What's new in AWS Observability at re:Invent 2023](#)
- [Amazon Builders' Library](#)
- [X-Ray Documentation](#)
- [Amazon CloudWatch RUM](#)
- [Amazon DevOps Guru](#)

Related videos:

- [AWS re:Invent 2023 - \[LAUNCH\] Application monitoring for modern workloads](#)
- [AWS re:Invent 2023 - Implementing application observability](#)
- [AWS re:Invent 2023 - Building an effective observability strategy](#)
- [AWS Summit SF 2022 - Full-stack observability and application monitoring with AWS](#)
- [AWS re:Invent 2022 - AWS optimization: Actionable steps for immediate results](#)
- [AWS re:Invent 2022 - The Amazon Builders' Library: 25 years of Amazon operational excellence](#)
- [AWS re:Invent 2022 - How Amazon uses better metrics for improved website performance](#)
- [Visual Monitoring of Applications with Amazon CloudWatch Synthetics](#)

Related examples:

- [Measure page load time with Amazon CloudWatch Synthetics](#)
- [Amazon CloudWatch RUM Web Client](#)
- [X-Ray SDK for Python](#)
- [Distributed Load Testing on AWS](#)

PERF05-BP03 Define a process to improve workload performance

Define a process to evaluate new services, design patterns, resource types, and configurations as they become available. For example, run existing performance tests on new instance offerings to determine their potential to improve your workload.

Common anti-patterns:

- You assume your current architecture is static and won't be updated over time.
- You introduce architecture changes over time with no metric justification.

Benefits of establishing this best practice: By defining your process for making architectural changes, you can use gathered data to influence your workload design over time.

Level of risk exposed if this best practice is not established: Medium

Implementation guidance

Your workload's performance has a few key constraints. Document these so that you know what kinds of innovation might improve the performance of your workload. Use this information when learning about new services or technology as it becomes available to identify ways to alleviate constraints or bottlenecks.

Identify the key performance constraints for your workload. Document your workload's performance constraints so that you know what kinds of innovation might improve the performance of your workload.

Implementation steps

- **Identify KPIs:** Identify your workload performance KPIs as outlined in [PERF05-BP01 Establish key performance indicators \(KPIs\) to measure workload health and performance](#) to baseline your workload.
- **Implement monitoring:** Use [AWS observability tools](#) to collect performance metrics and measure KPIs.
- **Conduct analysis:** Conduct in-depth analysis to identify the areas (like configuration and application code) in your workload that is under-performing as outlined in [PERF05-BP02 Use monitoring solutions to understand the areas where performance is most critical](#). Use your analysis and performance tools to identify the performance improvement strategies.
- **Validate improvements:** Use sandbox or pre-production environments to validate the effectiveness of improvement strategies.
- **Implement changes:** Implement the changes in production and continually monitor the workload's performance. Document the improvements, and communicate the changes to stakeholders.

- **Revisit and refine:** Regularly review your performance improvement process to identify areas for enhancement.

Resources

Related documents:

- [AWS Blog](#)
- [What's New with AWS](#)
- [AWS Skill Builder](#)

Related videos:

- [AWS re:Invent 2022 - Delivering sustainable, high-performing architectures](#)
- [AWS re:Invent 2023 - Optimize cost and performance and track progress toward mitigation](#)
- [AWS re:Invent 2022 - AWS optimization: Actionable steps for immediate results](#)
- [AWS re:Invent 2022 - Optimize your AWS workloads with best-practice guidance](#)

Related examples:

- [AWS Github](#)

PERF05-BP04 Load test your workload

Load test your workload to verify it can handle production load and identify any performance bottleneck.

Common anti-patterns:

- You load test individual parts of your workload but not your entire workload.
- You load test on infrastructure that is not the same as your production environment.
- You only conduct load testing to your expected load and not beyond, to help foresee where you may have future problems.
- You perform load testing without consulting the [Amazon EC2 Testing Policy](#) and submitting a Simulated Event Submissions Form. This results in your test failing to run, as it looks like a denial-of-service event.

Benefits of establishing this best practice: Measuring your performance under a load test will show you where you will be impacted as load increases. This can provide you with the capability of anticipating needed changes before they impact your workload.

Level of risk exposed if this best practice is not established: Low

Implementation guidance

Load testing in the cloud is a process to measure the performance of cloud workload under realistic conditions with expected user load. This process involves provisioning a production-like cloud environment, using load testing tools to generate load, and analyzing metrics to assess the ability of your workload handling a realistic load. Load tests must be run using synthetic or sanitized versions of production data (remove sensitive or identifying information). Automatically carry out load tests as part of your delivery pipeline, and compare the results against pre-defined KPIs and thresholds. This process helps you continue to achieve required performance.

Implementation steps

- **Define your testing objectives:** Identify the performance aspects of your workload that you want to evaluate, such as throughput and response time.
- **Select a testing tool:** Choose and configure the load testing tool that suits your workload.
- **Set up your environment:** Set up the test environment based on your production environment. You can use AWS services to run production-scale environments to test your architecture.
- **Implement monitoring:** Use monitoring tools such as [Amazon CloudWatch](#) to collect metrics across the resources in your architecture. You can also collect and publish custom metrics.
- **Define scenarios:** Define the load testing scenarios and parameters (like test duration and number of users).
- **Conduct load testing:** Perform test scenarios at scale. Take advantage of the AWS Cloud to test your workload to discover where it fails to scale, or if it scales in a non-linear way. For example, use Spot Instances to generate loads at low cost and discover bottlenecks before they are experienced in production.
- **Analyze test results:** Analyze the results to identify performance bottlenecks and areas for improvements.
- **Document and share findings:** Document and report on findings and recommendations. Share this information with stakeholders to help them make informed decision regarding performance optimization strategies.

- **Continually iterate:** Load testing should be performed at regular cadence, especially after a system change of update.

Resources

Related documents:

- [Amazon CloudWatch RUM](#)
- [Amazon CloudWatch Synthetics](#)
- [Distributed Load Testing on AWS](#)

Related videos:

- [AWS Summit ANZ 2023: Accelerate with confidence through AWS Distributed Load Testing](#)
- [AWS re:Invent 2022 - Scaling on AWS for your first 10 million users](#)
- [Solving with AWS Solutions: Distributed Load Testing](#)
- [AWS re:Invent 2021 - Optimize applications through end user insights with Amazon CloudWatch RUM](#)
- [Demo of Amazon CloudWatch Synthetics](#)

Related examples:

- [Distributed Load Testing on AWS](#)

PERF05-BP05 Use automation to proactively remediate performance-related issues

Use key performance indicators (KPIs), combined with monitoring and alerting systems, to proactively address performance-related issues.

Common anti-patterns:

- You only allow operations staff the ability to make operational changes to the workload.
- You let all alarms filter to the operations team with no proactive remediation.

Benefits of establishing this best practice: Proactive remediation of alarm actions allows support staff to concentrate on those items that are not automatically actionable. This helps operations staff handle all alarms without being overwhelmed and instead focus only on critical alarms.

Level of risk exposed if this best practice is not established: Low

Implementation guidance

Use alarms to trigger automated actions to remediate issues where possible. Escalate the alarm to those able to respond if automated response is not possible. For example, you may have a system that can predict expected key performance indicator (KPI) values and alarm when they breach certain thresholds, or a tool that can automatically halt or roll back deployments if KPIs are outside of expected values.

Implement processes that provide visibility into performance as your workload is running. Build monitoring dashboards and establish baseline norms for performance expectations to determine if the workload is performing optimally.

Implementation steps

- **Identify remediation workflow:** Identify and understand the performance issue that can be remediated automatically. Use AWS monitoring solutions such as [Amazon CloudWatch](#) or AWS X-Ray to help you better understand the root cause of the issue.
- **Define the automation process:** Create a step-by-step remediation process that can be used to automatically fix the issue.
- **Configure the initiation event:** Configure the event to automatically initiate the remediation process. For example, you can define a trigger to automatically restart an instance when it reaches a certain threshold of CPU utilization.
- **Automate the remediation:** Use AWS services and technologies to automate the remediation process. For example, [AWS Systems Manager Automation](#) provides a secure and scalable way to automate the remediation process. Make sure to use self-healing logic to revert changes if they do not successfully resolve the issue.
- **Test the workflow** Test the automated remediation process in a pre-production environment.
- **Implement the workflow:** Implement the automated remediation in the production environment.
- **Develop a playbook:** Develop and document a playbook that outlines the steps for the remediation plan, including the initiation events, remediation logic, and actions taken. Make sure to train stakeholders to help them effectively respond to automated remediation events.

- **Review and refine:** Regularly assess the effectiveness of the automated remediation workflow. Adjust initiation events and remediation logic if necessary.

Resources

Related documents:

- [CloudWatch Documentation](#)
- [Monitoring, Logging, and Performance AWS Partner Network Partners](#)
- [X-Ray Documentation](#)
- [Using Alarms and Alarm Actions in CloudWatch](#)
- [Build a Cloud Automation Practice for Operational Excellence: Best Practices from AWS Managed Services](#)
- [Automate your Amazon Redshift performance tuning with automatic table optimization](#)

Related videos:

- [AWS re:Invent 2023 - Strategies for automated scaling, remediation, and smart self-healing](#)
- [AWS re:Invent 2023 - \[LAUNCH\] Application monitoring for modern workloads](#)
- [AWS re:Invent 2023 - Implementing application observability](#)
- [AWS re:Invent 2021 - Intelligently automating cloud operations](#)
- [AWS re:Invent 2022 - Setting up controls at scale in your AWS environment](#)
- [AWS re:Invent 2022 - Automating patch management and compliance using AWS](#)
- [AWS re:Invent 2022 - How Amazon uses better metrics for improved website performance](#)
- [AWS re:Invent 2023 - Take a load off: Diagnose & resolve performance issues with Amazon RDS](#)
- [AWS re:Invent 2021 - {New Launch} Automatically detect and resolve issues with Amazon DevOps Guru](#)
- [AWS re:Invent 2023 - Centralize your operations](#)

Related examples:

- [CloudWatch Logs Customize Alarms](#)

PERF05-BP06 Keep your workload and services up-to-date

Stay up-to-date on new cloud services and features to adopt efficient features, remove issues, and improve the overall performance efficiency of your workload.

Common anti-patterns:

- You assume your current architecture is static and will not be updated over time.
- You do not have any systems or a regular cadence to evaluate if updated software and packages are compatible with your workload.

Benefits of establishing this best practice: By establishing a process to stay up-to-date on new services and offerings, you can adopt new features and capabilities, resolve issues, and improve workload performance.

Level of risk exposed if this best practice is not established: Low

Implementation guidance

Evaluate ways to improve performance as new services, design patterns, and product features become available. Determine which of these could improve performance or increase the efficiency of the workload through evaluation, internal discussion, or external analysis. Define a process to evaluate updates, new features, and services relevant to your workload. For example, build a proof of concept that uses new technologies or consult with an internal group. When trying new ideas or services, run performance tests to measure the impact that they have on the performance of the workload.

Implementation steps

- **Inventory your workload:** Inventory your workload software and architecture and identify components that need to be updated.
- **Identify update sources:** Identify news and update sources related to your workload components. As an example, you can subscribe to the [What's New at AWS blog](#) for the products that match your workload component. You can subscribe to the RSS feed or manage your [email subscriptions](#).
- **Define an update schedule:** Define a schedule to evaluate new services and features for your workload.
 - You can use [AWS Systems Manager Inventory](#) to collect operating system (OS), application, and instance metadata from your Amazon EC2 instances and quickly understand which

instances are running the software and configurations required by your software policy and which instances need to be updated.

- **Assess the new update:** Understand how to update the components of your workload. Take advantage of agility in the cloud to quickly test how new features can improve your workload to gain performance efficiency.
- **Use automation:** Use automation for the update process to reduce the level of effort to deploy new features and limit errors caused by manual processes.
 - You can use [CI/CD](#) to automatically update AMIs, container images, and other artifacts related to your cloud application.
 - You can use tools such as [AWS Systems Manager Patch Manager](#) to automate the process of system updates, and schedule the activity using [AWS Systems Manager Maintenance Windows](#).
- **Document the process:** Document your process for evaluating updates and new services. Provide your owners the time and space needed to research, test, experiment, and validate updates and new services. Refer back to the documented business requirements and KPIs to help prioritize which update will make a positive business impact.

Resources

Related documents:

- [AWS Blog](#)
- [What's New with AWS](#)
- [Implementing up-to-date images with automated EC2 Image Builder pipelines](#)

Related videos:

- [AWS re:Inforce 2022 - Automating patch management and compliance using AWS](#)
- [All Things Patch: AWS Systems Manager | AWS Events](#)

Related examples:

- [Inventory and Patch Management](#)
- [One Observability Workshop](#)

PERF05-BP07 Review metrics at regular intervals

As part of routine maintenance or in response to events or incidents, review which metrics are collected. Use these reviews to identify which metrics were essential in addressing issues and which additional metrics, if they were being tracked, could help identify, address, or prevent issues.

Common anti-patterns:

- You allow metrics to stay in an alarm state for an extended period of time.
- You create alarms that are not actionable by an automation system.

Benefits of establishing this best practice: Continually review metrics that are being collected to verify that they properly identify, address, or prevent issues. Metrics can also become stale if you let them stay in an alarm state for an extended period of time.

Level of risk exposed if this best practice is not established: Medium

Implementation guidance

Constantly improve metric collection and monitoring. As part of responding to incidents or events, evaluate which metrics were helpful in addressing the issue and which metrics could have helped that are not currently being tracked. Use this method to improve the quality of metrics you collect so that you can prevent, or more quickly resolve future incidents.

As part of responding to incidents or events, evaluate which metrics were helpful in addressing the issue and which metrics could have helped that are not currently being tracked. Use this to improve the quality of metrics you collect so that you can prevent or more quickly resolve future incidents.

Implementation steps

- **Define metrics:** Define critical performance metrics to monitor that are aligned to your workload objective, including metrics such as response time and resource utilization.
- **Establish baselines:** Set a baseline and desirable value for each metric. The baseline should provide reference points to identify deviation or anomalies.
- **Set up a cadence:** Set a cadence (like weekly or monthly) to review critical metrics.
- **Identify performance issues:** During each review, assess trends and deviation from the baseline values. Look for any performance bottlenecks or anomalies. For identified issues, conduct in-depth root cause analysis to understand the main reason behind the issue.

- **Identify corrective actions:** Use your analysis to identify corrective actions. This may include parameter tuning, fixing bugs, and scaling resources.
- **Document findings:** Document your findings, including identified issues, root causes, and corrective actions.
- **Iterate and improve:** Continually assess and improve the metrics review process. Use the lesson learned from previous review to enhance the process over time.

Resources

Related documents:

- [CloudWatch Documentation](#)
- [Collect metrics and logs from Amazon EC2 Instances and on-premises servers with the CloudWatch Agent](#)
- [Query your metrics with CloudWatch Metrics Insights](#)
- [Monitoring, Logging, and Performance AWS Partner Network Partners](#)
- [X-Ray Documentation](#)

Related videos:

- [AWS re:Invent 2022 - Setting up controls at scale in your AWS environment](#)
- [AWS re:Invent 2022 - How Amazon uses better metrics for improved website performance](#)
- [AWS re:Invent 2023 - Building an effective observability strategy](#)
- [AWS Summit SF 2022 - Full-stack observability and application monitoring with AWS](#)
- [AWS re:Invent 2023 - Take a load off: Diagnose & resolve performance issues with Amazon RDS](#)

Related examples:

- [Creating a dashboard with Quick Suite](#)
- [CloudWatch Dashboards](#)

Cost optimization

The Cost Optimization pillar includes the ability to run systems to deliver business value at the lowest price point. You can find prescriptive guidance on implementation in the [Cost Optimization Pillar whitepaper](#).

Best practice areas

- [Practice Cloud Financial Management](#)
- [Expenditure and usage awareness](#)
- [Cost-effective resources](#)
- [Manage demand and supply resources](#)
- [Optimize over time](#)

Practice Cloud Financial Management

Question

- [COST 1. How do you implement cloud financial management?](#)

COST 1. How do you implement cloud financial management?

Implementing Cloud Financial Management helps organizations realize business value and financial success as they optimize their cost and usage and scale on AWS.

Best practices

- [COST01-BP01 Establish ownership of cost optimization](#)
- [COST01-BP02 Establish a partnership between finance and technology](#)
- [COST01-BP03 Establish cloud budgets and forecasts](#)
- [COST01-BP04 Implement cost awareness in your organizational processes](#)
- [COST01-BP05 Report and notify on cost optimization](#)
- [COST01-BP06 Monitor cost proactively](#)
- [COST01-BP07 Keep up-to-date with new service releases](#)
- [COST01-BP08 Create a cost-aware culture](#)
- [COST01-BP09 Quantify business value from cost optimization](#)

COST01-BP01 Establish ownership of cost optimization

Create a team (Cloud Business Office, Cloud Center of Excellence, or FinOps team) that is responsible for establishing and maintaining cost awareness across your organization. The owner of cost optimization can be an individual or a team (requires people from finance, technology, and business teams) that understands the entire organization and cloud finance.

Level of risk exposed if this best practice is not established: High

Implementation guidance

This is the introduction of a Cloud Business Office (CBO) or Cloud Center of Excellence (CCOE) function or team that is responsible for establishing and maintaining a culture of cost awareness in cloud computing. This function can be an existing individual, a team within your organization, or a new team of key finance, technology, and organization stakeholders from across the organization.

The function (individual or team) prioritizes and spends the required percentage of their time on cost management and cost optimization activities. For a small organization, the function might spend a smaller percentage of time compared to a full-time function for a larger enterprise.

The function requires a multi-disciplinary approach, with capabilities in project management, data science, financial analysis, and software or infrastructure development. They can improve workload efficiency by running cost optimizations within three different ownerships:

- **Centralized:** Through designated teams such as FinOps team, Cloud Financial Management (CFM) team, Cloud Business Office (CBO), or Cloud Center of Excellence (CCoE), customers can design and implement governance mechanisms and drive best practices company-wide.
- **Decentralized:** Influencing technology teams to run cost optimizations.
- **Hybrid:** Combination of both centralized and decentralized teams can work together to run cost optimizations.

The function may be measured against their ability to run and deliver against cost optimization goals (for example, workload efficiency metrics).

You must secure executive sponsorship for this function, which is a key success factor. The sponsor is regarded as a champion for cost efficient cloud consumption, and provides escalation support for the team to ensure that cost optimization activities are treated with the level of priority defined by the organization. Otherwise, guidance can be ignored and cost saving opportunities will not be

prioritized. Together, the sponsor and team help your organization consume the cloud efficiently and deliver business value.

If you have the Business, Enterprise-On-Ramp or Enterprise [support plan](#) and need help building this team or function, reach out to your Cloud Financial Management (CFM) experts through your account team.

Implementation steps

- **Define key members:** All relevant parts of your organization must contribute and be interested in cost management. Common teams within organizations typically include: finance, application or product owners, management, and technical teams (DevOps). Some are engaged full time (finance or technical), while others are engaged periodically as required. Individuals or teams performing CFM need the following set of skills:
 - **Software development:** in the case where scripts and automation are being built out.
 - **Infrastructure engineering:** to deploy scripts, automate processes, and understand how services or resources are provisioned.
 - **Operations acumen:** CFM is about operating on the cloud efficiently by measuring, monitoring, modifying, planning, and scaling efficient use of the cloud.
- **Define goals and metrics:** The function needs to deliver value to the organization in different ways. These goals are defined and continually evolve as the organization evolves. Common activities include: creating and running education programs on cost optimization across the organization, developing organization-wide standards, such as monitoring and reporting for cost optimization, and setting workload goals on optimization. This function also needs to regularly report to the organization on their cost optimization capability.

You can define value- or cost-based key performance indicators (KPIs). When you define the KPIs, you can calculate expected cost in terms of efficiency and expected business outcome. Value-based KPIs tie cost and usage metrics to business value drivers and help rationalize changes in AWS spend. The first step to deriving value-based KPIs is working together, cross-organizationally, to select and agree upon a standard set of KPIs.

- **Establish regular cadence:** The group (finance, technology and business teams) should come together regularly to review their goals and metrics. A typical cadence involves reviewing the state of the organization, reviewing any programs currently running, and reviewing overall financial and optimization metrics. Afterwards, key workloads are reported on in greater detail.

During these regular reviews, you can review workload efficiency (cost) and business outcome. For example, a 20% cost increase for a workload may align with increased customer usage. In this case, this 20% cost increase can be interpreted as an investment. These regular cadence calls can help teams to identify value KPIs that provide meaning to the entire organization.

Resources

Related documents:

- [AWS CCOE Blog](#)
- [Creating Cloud Business Office](#)
- [CCOE - Cloud Center of Excellence](#)

Related videos:

- [Vanguard CCOE Success Story](#)

Related examples:

- [Using a Cloud Center of Excellence \(CCOE\) to Transform the Entire Enterprise](#)
- [Building a CCOE to transform the entire enterprise](#)
- [7 Pitfalls to Avoid When Building CCOE](#)

COST01-BP02 Establish a partnership between finance and technology

Involve finance and technology teams in cost and usage discussions at all stages of your cloud journey. Teams regularly meet and discuss topics such as organizational goals and targets, current state of cost and usage, and financial and accounting practices.

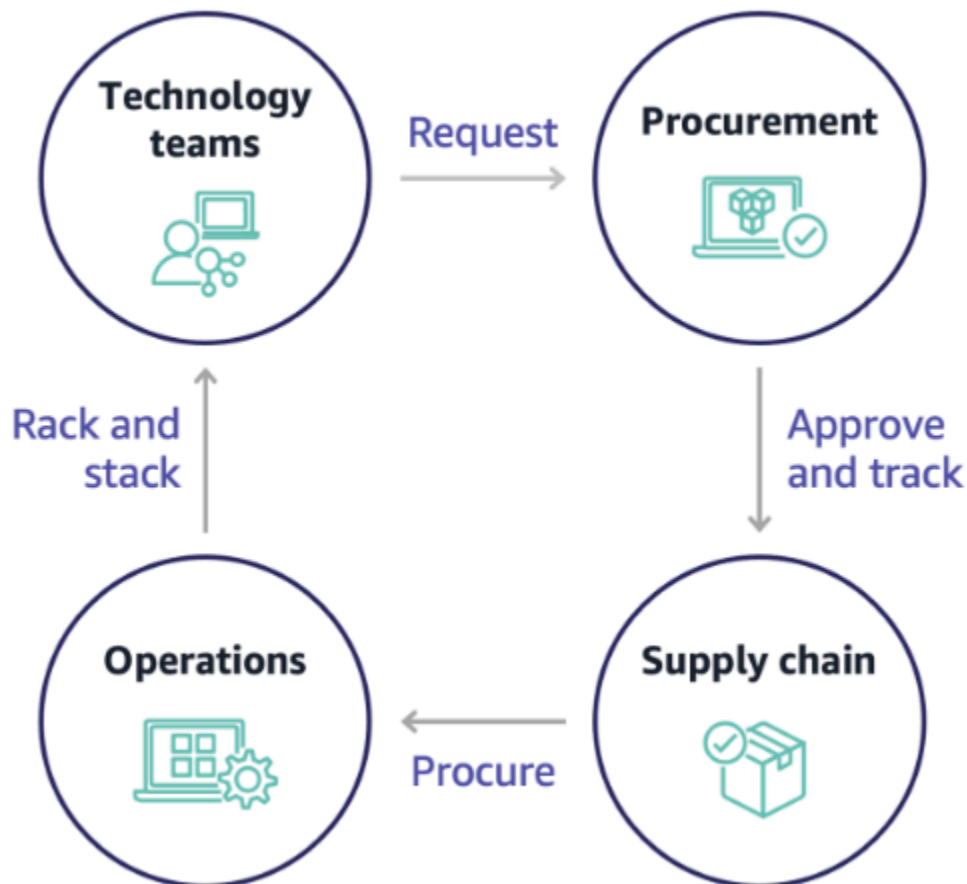
Level of risk exposed if this best practice is not established: High

Implementation guidance

Technology teams innovate faster in the cloud due to shortened approval, procurement, and infrastructure deployment cycles. This can be an adjustment for finance organizations previously used to running time-consuming and resource-intensive processes for procuring and deploying capital in data center and on-premises environments, and cost allocation only at project approval.

From a finance and procurement organization perspective, the process for capital budgeting, capital requests, approvals, procurement, and installing physical infrastructure is one that has been learned and standardized over decades:

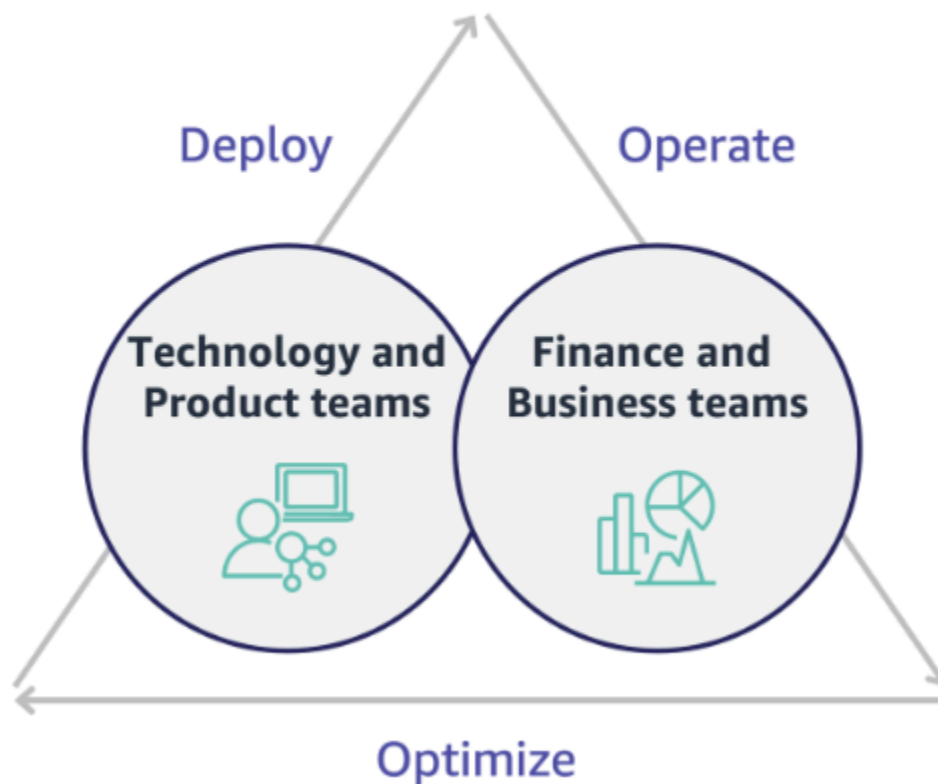
- Engineering or IT teams are typically the requesters
- Various finance teams act as approvers and procurers
- Operations teams rack, stack, and hand off ready-to-use infrastructure



With the adoption of cloud, infrastructure procurement and consumption are no longer beholden to a chain of dependencies. In the cloud model, technology and product teams are no longer just builders, but operators and owners of their products, responsible for most of the activities historically associated with finance and operations teams, including procurement and deployment.

All it really takes to provision cloud resources is an account, and the right set of permissions. This is also what reduces IT and finance risk; which means teams are always a just few clicks or API calls away from terminating idle or unnecessary cloud resources. This is also what allows technology

teams to innovate faster – the agility and ability to spin up and then tear down experiments. While the variable nature of cloud consumption may impact predictability from a capital budgeting and forecasting perspective, cloud provides organizations with the ability to reduce the cost of over-provisioning, as well as reduce the opportunity cost associated with conservative under-provisioning.



Establish a partnership between key finance and technology stakeholders to create a shared understanding of organizational goals and develop mechanisms to succeed financially in the variable spend model of cloud computing. Relevant teams within your organization must be involved in cost and usage discussions at all stages of your cloud journey, including:

- **Financial leads:** CFOs, financial controllers, financial planners, business analysts, procurement, sourcing, and accounts payable must understand the cloud model of consumption, purchasing options, and the monthly invoicing process. Finance needs to partner with technology teams to create and socialize an IT value story, helping business teams understand how technology spend is linked to business outcomes. This way, technology expenditures are viewed not as costs, but rather as investments. Due to the fundamental differences between the cloud (such as the rate of change in usage, pay as you go pricing, tiered pricing, pricing models, and detailed billing and usage information) compared to on-premises operation, it is essential that the finance

organization understands how cloud usage can impact business aspects including procurement processes, incentive tracking, cost allocation and financial statements.

- **Technology leads:** Technology leads (including product and application owners) must be aware of the financial requirements (for example, budget constraints) as well as business requirements (for example, service level agreements). This allows the workload to be implemented to achieve the desired goals of the organization.

The partnership of finance and technology provides the following benefits:

- Finance and technology teams have near real-time visibility into cost and usage.
- Finance and technology teams establish a standard operating procedure to handle cloud spend variance.
- Finance stakeholders act as strategic advisors with respect to how capital is used to purchase commitment discounts (for example, Reserved Instances or AWS Savings Plans), and how the cloud is used to grow the organization.
- Existing accounts payable and procurement processes are used with the cloud.
- Finance and technology teams collaborate on forecasting future AWS cost and usage to align and build organizational budgets.
- Better cross-organizational communication through a shared language, and common understanding of financial concepts.

Additional stakeholders within your organization that should be involved in cost and usage discussions include:

- **Business unit owners:** Business unit owners must understand the cloud business model so that they can provide direction to both the business units and the entire company. This cloud knowledge is critical when there is a need to forecast growth and workload usage, and when assessing longer-term purchasing options, such as Reserved Instances or Savings Plans.
- **Engineering team:** Establishing a partnership between finance and technology teams is essential for building a cost-aware culture that encourages engineers to take action on Cloud Financial Management (CFM). One of the common problems of CFM or finance operations practitioners and finance teams is getting engineers to understand the whole business on cloud, follow best practices, and take recommended actions.
- **Third parties:** If your organization uses third parties (for example, consultants or tools), ensure that they are aligned to your financial goals and can demonstrate both alignment through their

engagement models and a return on investment (ROI). Typically, third parties will contribute to reporting and analysis of any workloads that they manage, and they will provide cost analysis of any workloads that they design.

Implementing CFM and achieving success requires collaboration across finance, technology, and business teams, and a shift in how cloud spend is communicated and evaluated across the organization. Include engineering teams so that they can be part of these cost and usage discussions at all stages, and encourage them to follow best practices and take agreed-upon actions accordingly.

Implementation steps

- **Define key members:** Verify that all relevant members of your finance and technology teams participate in the partnership. Relevant finance members will be those having interaction with the cloud bill. This will typically be CFOs, financial controllers, financial planners, business analysts, procurement, and sourcing. Technology members will typically be product and application owners, technical managers and representatives from all teams that build on the cloud. Other members may include business unit owners, such as marketing, that will influence usage of products, and third parties such as consultants, to achieve alignment to your goals and mechanisms, and to assist with reporting.
- **Define topics for discussion:** Define the topics that are common across the teams, or will need a shared understanding. Follow cost from that time it is created, until the bill is paid. Note any members involved, and organizational processes that are required to be applied. Understand each step or process it goes through and the associated information, such as pricing models available, tiered pricing, discount models, budgeting, and financial requirements.
- **Establish regular cadence:** To create a finance and technology partnership, establish a regular communication cadence to create and maintain alignment. The group needs to come together regularly against their goals and metrics. A typical cadence involves reviewing the state of the organization, reviewing any programs currently running, and reviewing overall financial and optimization metrics. Then key workloads are reported on in greater detail.

Resources

Related documents:

- [AWS News Blog](#)

COST01-BP03 Establish cloud budgets and forecasts

Adjust existing organizational budgeting and forecasting processes to be compatible with the highly variable nature of cloud costs and usage. Processes must be dynamic, using trend-based or business driver-based algorithms or a combination of both.

Level of risk exposed if this best practice is not established: High

Implementation guidance

In traditional on-premises IT setups, customers often face the challenge of planning for fixed costs that change only occasionally, typically with the purchase of new IT hardware and services to meet peak demand. In contrast, AWS Cloud adopts a different approach, where customers pay for the resources they use as dictated by their actual IT and business needs. In the cloud environment, demand can fluctuate on a monthly, daily, or even hourly basis.

Using the cloud brings efficiency, speed, and agility, which results in a highly-variable cost and usage pattern. Costs can decrease or sometimes increase in response to greater workload efficiency or the deployment of new workloads and features. As workloads scale to serve an expanding customer base, cloud usage and costs correspondingly rise due to the increased accessibility of resources. This flexibility in cloud services extends to the costs and forecasts, which creates a degree of elasticity.

It's essential to align closely with these shifting business needs and demand drivers, and aim for the most accurate planning possible. Traditional organizational budget processes need to adapt to accommodate this variability.

Consider cost modeling while you forecast the cost for new workloads. Cost modeling creates a baseline understanding of expected cloud costs, which helps you perform total cost of ownership (TCO), return on investment (ROI), and other financial analysis, set targets and expectations with stakeholders, and identify opportunities for cost optimization.

Your organization should understand the cost definitions and accepted groupings. The level of detail at which you forecast can vary based on your organization's structure and internal workflows. Select a granularity that suits your specific requirements and organizational setup. It is important to understand at what level the forecast is performed:

- **Management account or AWS Organizations level:** The management account is the account that you use to create AWS Organizations. Organizations have one management account by default.

- **Linked or member account:** An account in Organizations is a standard AWS account that contains your AWS resources and the identities that can access those resources.
- **Environment:** An environment is a collection of AWS resources that runs an application version. An environment can be made with multiple linked or member accounts.
- **Project:** A project is a combination of set objectives or tasks to be accomplished within a fixed period. It is important to consider the project lifecycle during your forecast.
- **AWS services:** Groups or categories such as compute or storage services where you can group AWS services for your forecast.
- **Custom grouping:** You can create custom groups based on your organization's needs, such as business units, cost centers, teams, cost allocation tags, cost categories, linked accounts, or combination of these.

Identify the business drivers that can impact your usage cost, and forecast for each of them separately to calculate expected usage in advance. Some of the drivers might be linked to IT and product teams within the organization. Other business drivers, such as marketing events, promotions, geographic expansions, mergers, and acquisitions, are known by your sales, marketing, and business leaders, and it's important to collaborate and account for all those demand drivers as well.

You can use [AWS Cost Explorer](#) for trend-based forecasting in a defined future time range based on your past spend. AWS Cost Explorer's forecasting engine segments your historical data based on charge types (for example, Reserved Instances) and uses a combination of machine learning and rule-based models to predict spend across all charge types individually.

Once you've established your forecast process and built models, you can use [AWS Budgets](#) to set custom budgets at a granular level by specifying the time period, recurrence, or amount (fixed or variable) and add filters such as service, AWS Region, and tags. The budget is usually prepared for a single year and remains fixed, which requires strict adherence from everyone involved. In contrast, forecasts are more flexible, which allows for readjustments throughout the year and provides dynamic projections over a period of one, two, or three years. Both budgets and forecasts play a crucial role when you establish financial expectations among various technology and business stakeholders. Accurate forecasts and implementation also provides accountability to stakeholders who are directly responsible for provisioning cost in the first place, and it can also raise their overall cost awareness.

To stay informed on the performance of your existing budgets, you can create and schedule AWS Budgets reports to email you and your stakeholders on a regular cadence. You can also create AWS

Budgets alerts based on actual costs, which are reactive in nature, or on forecasted costs, which provides time to implement mitigations against potential cost overruns. You can be alerted when your cost or usage actually exceeds a certain level or if they are forecasted to exceed your budgeted amount.

Adjust existing budget and forecast processes to be more dynamic using trend-based algorithms (with historical costs as inputs) and driver-based algorithms (for example, new product launches, Regional expansion, or new environments for workloads), which are ideal for a dynamic and variable spending environment. Once you've determined your trend-based forecast using Cost Explorer or any other tools, use the [AWS Pricing Calculator](#) to estimate your AWS use case and future costs based on the expected usage (traffic, requests-per-second, or required Amazon EC2 instances).

Track the accuracy of that forecast, as budgets should be set based on these forecast calculations and estimations. Monitor the accuracy and effectiveness of the integrated cloud cost forecasts. Regularly review actual spend compared to your forecast, and adjust as needed to improve forecast precision. Track forecast variance, and perform root cause analysis on reported variance to act and adjust forecasts.

As mentioned in the [COST01-BP02 Establish a partnership between finance and technology](#), it is important to foster a partnership and cadence between IT, finance, and other stakeholders to verify that they are all using the same tools or processes for consistency. In cases where budgets may need to change, increase cadence touch points to react to those changes more quickly.

Implementation steps

- **Define the cost language within the organization:** Create a common AWS cost language within the Organization with multiple dimension and grouping. Make sure stakeholders understand forecast granularity, pricing models, and the level of your cost forecasts.
- **Analyze trend-based forecasts:** Use trend-based forecast tools such as AWS Cost Explorer and Amazon Forecast. Analyze your usage cost on multiple dimensions like service, account, tags, and cost categories.
- **Analyze driver-based forecasts:** Identify the impact of business drivers on your cloud usage, and forecast for each of them separately to calculate expected usage cost in advance. Work closely with business unit owners and stakeholders to understand the impact on new drivers, and calculate expected cost changes to define accurate budgets.
- **Update existing forecast and budget processes:** Based on adopted forecast methods such as trend-based, business driver-based, or a combination of both forecasting methods, define

your forecast and budget processes. Budgets should be calculated, realistic, and based on your forecasts.

- **Configure alerts and notifications:** Use AWS Budgets alerts and cost anomaly detection to get alerts and notifications.
- **Perform regular reviews with key stakeholders:** For example, align on changes in business direction and usage with stakeholders in IT, finance, platform teams, and other areas of the business.

Resources

Related documents:

- [AWS Cost Explorer](#)
- [AWS Cost and Usage Report](#)
- [Forecasting with Cost Explorer](#)
- [Quick Suite Forecasting](#)
- [AWS Budgets](#)

Related videos:

- [How can I use AWS Budgets to track my spending and usage](#)
- [AWS Cost Optimization Series: AWS Budgets](#)

Related examples:

- [Understand and build driver-based forecasting](#)
- [How to establish and drive a forecasting culture](#)
- [How to improve your cloud cost forecasting](#)
- [Using the right tools for your cloud cost forecasting](#)

COST01-BP04 Implement cost awareness in your organizational processes

Implement cost awareness, create transparency, and accountability of costs into new or existing processes that impact usage, and leverage existing processes for cost awareness. Implement cost awareness into employee training.

Level of risk exposed if this best practice is not established: High

Implementation guidance

Cost awareness must be implemented in new and existing organizational processes. It is one of the foundational, prerequisite capabilities for other best practices. It is recommended to reuse and modify existing processes where possible — this minimizes the impact to agility and velocity. Report cloud costs to the technology teams and the decision makers in the business and finance teams to raise cost awareness, and establish efficiency key performance indicators (KPIs) for finance and business stakeholders. The following recommendations will help implement cost awareness in your workload:

- Verify that change management includes a cost measurement to quantify the financial impact of your changes. This helps proactively address cost-related concerns and highlight cost savings.
- Verify that cost optimization is a core component of your operating capabilities. For example, you can leverage existing incident management processes to investigate and identify root causes for cost and usage anomalies or cost overruns.
- Accelerate cost savings and business value realization through automation or tooling. When thinking about the cost of implementing, frame the conversation to include an return on investment (ROI) component to justify the investment of time or money.
- Allocate cloud costs by implementing showbacks or chargebacks for cloud spend, including spend on commitment-based purchase options, shared services and marketplace purchases to drive most cost-aware cloud consumption.
- Extend existing training and development programs to include cost-awareness training throughout your organization. It is recommended that this includes continuous training and certification. This will build an organization that is capable of self-managing cost and usage.
- Take advantage of free AWS native tools such as [AWS Cost Anomaly Detection](#), [AWS Budgets](#), and [AWS Budgets Reports](#).

When organizations consistently adopt [Cloud Financial Management](#) (CFM) practices, those behaviours become ingrained in the way of working and decision-making. The result is a culture that is more cost-aware, from developers architecting a new born-in-the-cloud application, to finance managers analyzing the ROI on these new cloud investments.

Implementation steps

- **Identify relevant organizational processes:** Each organizational unit reviews their processes and identifies processes that impact cost and usage. Any processes that result in the creation or termination of a resource need to be included for review. Look for processes that can support cost awareness in your business, such as incident management and training.
- **Establish self-sustaining cost-aware culture:** Make sure all the relevant stakeholders align with cause-of-change and impact as a cost so that they understand cloud cost. This will allow your organization to establish a self-sustaining cost-aware culture of innovation.
- **Update processes with cost awareness:** Each process is modified to be made cost aware. The process may require additional pre-checks, such as assessing the impact of cost, or post-checks validating that the expected changes in cost and usage occurred. Supporting processes such as training and incident management can be extended to include items for cost and usage.

To get help, reach out to CFM experts through your Account team, or explore the resources and related documents below.

Resources

Related documents:

- [AWS Cloud Financial Management](#)

Related examples:

- [Strategy for Efficient Cloud Cost Management](#)
- [Cost Control Blog Series #3: How to Handle Cost Shock](#)
- [A Beginner's Guide to AWS Cost Management](#)

COST01-BP05 Report and notify on cost optimization

Set up cloud budgets and configure mechanisms to detect anomalies in usage. Configure related tools for cost and usage alerts against pre-defined targets and receive notifications when any usage exceeds those targets. Have regular meetings to analyze the cost-effectiveness of your workloads and promote cost awareness.

Level of risk exposed if this best practice is not established: Low

Implementation guidance

You must regularly report on cost and usage optimization within your organization. You can implement dedicated sessions to discuss cost performance, or include cost optimization in your regular operational reporting cycles for your workloads. Use services and tools to monitor your cost performances regularly and implement cost savings opportunities.

View your cost and usage with multiple filters and granularity by using [AWS Cost Explorer](#), which provides dashboards and reports such as costs by service or by account, daily costs, or marketplace costs. Track your progress of cost and usage against configured budgets with [AWS Budgets Reports](#).

Use [AWS Budgets](#) to set custom budgets to track your costs and usage and respond quickly to alerts received from email or Amazon Simple Notification Service (Amazon SNS) notifications if you exceed your threshold. [Set your preferred budget](#) period to daily, monthly, quarterly, or annually, and create specific budget limits to stay informed on how actual or forecasted costs and usage progress toward your budget threshold. You can also set up [alerts](#) and [actions](#) against those alerts to run automatically or through an approval process when a budget target is exceeded.

Implement notifications on cost and usage to ensure that changes in cost and usage can be acted upon quickly if they are unexpected. [AWS Cost Anomaly Detection](#) allows you to reduce cost surprises and enhance control without slowing innovation. AWS Cost Anomaly Detection identifies anomalous spend and root causes, which helps to reduce the risk of billing surprises. With three simple steps, you can create your own contextualized monitor and receive alerts when any anomalous spend is detected.

You can also use [Quick Suite](#) with AWS Cost and Usage Report (CUR) data, to provide highly customized reporting with more granular data. Quick Suite allows you to schedule reports and receive periodic Cost Report emails for historical cost and usage or cost-saving opportunities. Check our [Cost Intelligence Dashboard](#) (CID) solution built on Quick Suite, which gives you advanced visibility.

Use [AWS Trusted Advisor](#), which provides guidance to verify whether provisioned resources are aligned with AWS best practices for cost optimization.

Check your Savings Plans recommendations through visual graphs against your granular cost and usage. Hourly graphs show On-Demand spend alongside the recommended Savings Plans commitment, providing insight into estimated savings, Savings Plans coverage, and Savings Plans utilization. This helps organizations to understand how their Savings Plans apply to each hour of spend without having to invest time and resources into building models to analyze their spend.

Periodically create reports containing a highlight of Savings Plans, Reserved Instances, and Amazon EC2 rightsizing recommendations from AWS Cost Explorer to start reducing the cost associated with steady-state workloads, idle, and underutilized resources. Identify and recoup spend associated with cloud waste for resources that are deployed. Cloud waste occurs when incorrectly-sized resources are created or different usage patterns are observed instead what is expected. Follow AWS best practices to reduce your waste or ask your account team and partner to help you to [optimize and save](#) your cloud costs.

Generate reports regularly for better purchasing options for your resources to drive down unit costs for your workloads. Purchasing options such as Savings Plans, Reserved Instances, or Amazon EC2 Spot Instances offer the deepest cost savings for fault-tolerant workloads and allow stakeholders (business owners, finance, and tech teams) to be part of these commitment discussions.

Share the reports that contain opportunities or new release announcements that may help you to reduce total cost of ownership (TCO) of the cloud. Adopt new services, Regions, features, solutions, or new ways to achieve further cost reductions.

Implementation steps

- **Configure AWS Budgets:** Configure AWS Budgets on all accounts for your workload. Set a budget for the overall account spend, and a budget for the workload by using tags.
 - [Well-Architected Labs: Cost and Governance Usage](#)
- **Report on cost optimization:** Set up a regular cycle to discuss and analyze the efficiency of the workload. Using the metrics established, report on the metrics achieved and the cost of achieving them. Identify and fix any negative trends, as well as positive trends that you can promote across your organization. Reporting should involve representatives from the application teams and owners, finance, and key decision makers with respect to cloud expenditure.

Resources

Related documents:

- [AWS Cost Explorer](#)
- [AWS Trusted Advisor](#)
- [AWS Budgets](#)
- [AWS Cost and Usage Report](#)

- [AWS Budgets Best Practices](#)
- [Amazon S3 Analytics](#)

Related examples:

- [Key ways to start optimizing your AWS cloud costs](#)

COST01-BP06 Monitor cost proactively

Implement tooling and dashboards to monitor cost proactively for the workload. Regularly review the costs with configured tools or out of the box tools, do not just look at costs and categories when you receive notifications. Monitoring and analyzing costs proactively helps to identify positive trends and allows you to promote them throughout your organization.

Level of risk exposed if this best practice is not established: Medium

Implementation guidance

It is recommended to monitor cost and usage proactively within your organization, not just when there are exceptions or anomalies. Highly visible dashboards throughout your office or work environment ensure that key people have access to the information they need, and indicate the organization's focus on cost optimization. Visible dashboards allow you to actively promote successful outcomes and implement them throughout your organization.

Create a daily or frequent routine to use [AWS Cost Explorer](#) or any other dashboard such as [Amazon Quick Suite](#) to see the costs and analyze proactively. Analyze AWS service usage and costs at the AWS account-level, workload-level, or specific AWS service-level with grouping and filtering, and validate whether they are expected or not. Use the hourly- and resource-level granularity and tags to filter and identify incurring costs for the top resources. You can also build your own reports with the [Cost Intelligence Dashboard](#), an [Amazon Quick Suite](#) solution built by AWS Solutions Architects, and compare your budgets with the actual cost and usage.

Implementation steps

- **Report on cost optimization:** Set up a regular cycle to discuss and analyze the efficiency of the workload. Using the metrics established, report on the metrics achieved and the cost of achieving them. Identify and fix any negative trends, and identify positive trends to promote across your organization. Reporting should involve representatives from the application teams and owners, finance, and management.

- **Create and activate daily granularity [AWS Budgets](#) for the cost and usage to take timely actions to prevent any potential cost overruns:** AWS Budgets allow you to configure alert notifications, so you stay informed if any of your budget types fall out of your pre-configured thresholds. The best way to leverage AWS Budgets is to set your expected cost and usage as your limits, so that anything above your budgets can be considered overspend.
- **Create [AWS Cost Anomaly Detection](#) for cost monitor:** [AWS Cost Anomaly Detection](#) uses advanced Machine Learning technology to identify anomalous spend and root causes, so you can quickly take action. It allows you to configure cost monitors that define spend segments you want to evaluate (for example, individual AWS services, member accounts, cost allocation tags, and cost categories), and lets you set when, where, and how you receive your alert notifications. For each monitor, attach multiple alert subscriptions for business owners and technology teams, including a name, a cost impact threshold, and alerting frequency (individual alerts, daily summary, weekly summary) for each subscription.
- **Use [AWS Cost Explorer](#) or integrate your [AWS Cost and Usage Report \(CUR\)](#) data with Amazon Quick Suite dashboards to visualize your organization's costs:** AWS Cost Explorer has an easy-to-use interface that lets you visualize, understand, and manage your AWS costs and usage over time. The [Cost Intelligence Dashboard](#) is a customizable and accessible dashboard to help create the foundation of your own cost management and optimization tool.

Resources

Related documents:

- [AWS Budgets](#)
- [AWS Cost Explorer](#)
- [Daily Cost and Usage Budgets](#)
- [AWS Cost Anomaly Detection](#)

Related examples:

- [AWS Cost Anomaly Detection Alert with Slack](#)

COST01-BP07 Keep up-to-date with new service releases

Consult regularly with experts or AWS Partners to consider which services and features provide lower cost. Review AWS blogs and other information sources.

Level of risk exposed if this best practice is not established: Medium**Implementation guidance**

AWS is constantly adding new capabilities so you can leverage the latest technologies to experiment and innovate more quickly. You may be able to implement new AWS services and features to increase cost efficiency in your workload. Regularly review [AWS Cost Management](#), the [AWS News Blog](#), the [AWS Cost Management blog](#), and [What's New with AWS](#) for information on new service and feature releases. What's New posts provide a brief overview of all AWS service, feature, and Region expansion announcements as they are released.

Implementation steps

- **Subscribe to blogs:** Go to the AWS blogs pages and subscribe to the What's New Blog and other relevant blogs. You can sign up on the [communication preference](#) page with your email address.
- **Subscribe to AWS News:** Regularly review the [AWS News Blog](#) and [What's New with AWS](#) for information on new service and feature releases. Subscribe to the RSS feed, or with your email to follow announcements and releases.
- **Follow AWS Price Reductions:** Regular price cuts on all our services has been a standard way for AWS to pass on the economic efficiencies to our customers gained from our scale. As of September 20, 2023, AWS has reduced prices 134 times since 2006. If you have any pending business decisions due to price concerns, you can review them again after price reductions and new service integrations. You can learn about the previous price reductions efforts, including Amazon Elastic Compute Cloud (Amazon EC2) instances, in the [price-reduction category of the AWS News Blog](#).
- **AWS events and meetups:** Attend your local AWS summit, and any local meetups with other organizations from your local area. If you cannot attend in person, try to attend virtual events to hear more from AWS experts and other customers' business cases.
- **Meet with your account team:** Schedule a regular cadence with your account team, meet with them and discuss industry trends and AWS services. Speak with your account manager, Solutions Architect, and support team.

Resources**Related documents:**

- [AWS Cost Management](#)

- [What's New with AWS](#)
- [AWS News Blog](#)

Related examples:

- [Amazon EC2 – 15 Years of Optimizing and Saving Your IT Costs](#)
- [AWS News Blog - Price Reduction](#)

COST01-BP08 Create a cost-aware culture

Implement changes or programs across your organization to create a cost-aware culture. It is recommended to start small, then as your capabilities increase and your organization's use of the cloud increases, implement large and wide ranging programs.

Level of risk exposed if this best practice is not established: Low

Implementation guidance

A cost-aware culture allows you to scale cost optimization and Cloud Financial Management (financial operations, cloud center of excellence, cloud operations teams, and so on) through best practices that are performed in an organic and decentralized manner across your organization. Cost awareness allows you to create high levels of capability across your organization with minimal effort, compared to a strict top-down, centralized approach.

Creating cost awareness in cloud computing, especially for primary cost drivers in cloud computing, allows teams to understand expected outcomes of any changes in cost perspective. Teams who access the cloud environments should be aware of pricing models and the difference between traditional on-premises datacenters and cloud computing.

The main benefit of a cost-aware culture is that technology teams optimize costs proactively and continually (for example, they are considered a non-functional requirement when architecting new workloads, or making changes to existing workloads) rather than performing reactive cost optimizations as needed.

Small changes in culture can have large impacts on the efficiency of your current and future workloads. Examples of this include:

- Giving visibility and creating awareness in engineering teams to understand what they do, and what they impact in terms of cost.

- Gamifying cost and usage across your organization. This can be done through a publicly visible dashboard, or a report that compares normalized costs and usage across teams (for example, cost-per-workload and cost-per-transaction).
- Recognizing cost efficiency. Reward voluntary or unsolicited cost optimization accomplishments publicly or privately, and learn from mistakes to avoid repeating them in the future.
- Creating top-down organizational requirements for workloads to run at pre-defined budgets.
- Questioning business requirements of changes, and the cost impact of requested changes to the architecture infrastructure or workload configuration to make sure you pay only what you need.
- Making sure the change planner is aware of expected changes that have a cost impact, and that they are confirmed by the stakeholders to deliver business outcomes cost-effectively.

Implementation steps

- **Report cloud costs to technology teams:** To raise cost awareness, and establish efficiency KPIs for finance and business stakeholders.
- **Inform stakeholders or team members about planned changes:** Create an agenda item to discuss planned changes and the cost-benefit impact on the workload during weekly change meetings.
- **Meet with your account team:** Establish a regular meeting cadence with your account team, and discuss industry trends and AWS services. Speak with your account manager, architect, and support team.
- **Share success stories:** Share success stories about cost reduction for any workload, AWS account, or organization to create a positive attitude and encouragement around cost optimization.
- **Training:** Ensure technical teams or team members are trained for awareness of resource costs on AWS Cloud.
- **AWS events and meetups:** Attend local AWS summits, and any local meetups with other organizations from your local area.
- **Subscribe to blogs:** Go to the AWS blogs pages and subscribe to the [What's New Blog](#) and other relevant blogs to follow new releases, implementations, examples, and changes shared by AWS.

Resources

Related documents:

- [AWS Blog](#)

- [AWS Cost Management](#)
- [AWS News Blog](#)

Related examples:

- [AWS Cloud Financial Management](#)

COST01-BP09 Quantify business value from cost optimization

Quantifying business value from cost optimization allows you to understand the entire set of benefits to your organization. Because cost optimization is a necessary investment, quantifying business value allows you to explain the return on investment to stakeholders. Quantifying business value can help you gain more buy-in from stakeholders on future cost optimization investments, and provides a framework to measure the outcomes for your organization's cost optimization activities.

Level of risk exposed if this best practice is not established: Medium

Implementation guidance

Quantifying the business value means measuring the benefits that businesses gain from the actions and decisions they take. Business value can be tangible (like reduced expenses or increased profits) or intangible (like improved brand reputation or increased customer satisfaction).

To quantify business value from cost optimization means determining how much value or benefit you're getting from your efforts to spend more efficiently. For example, if a company spends \$100,000 to deploy a workload on AWS and later optimizes it, the new cost becomes only \$80,000 without sacrificing the quality or output. In this scenario, the quantified business value from cost optimization would be a savings of \$20,000. But beyond just savings, the business might also quantify value in terms of faster delivery times, improved customer satisfaction, or other metrics that result from the cost optimization efforts. Stakeholders need to make decisions about the potential value of cost optimization, the cost of optimizing the workload, and return value.

In addition to reporting savings from cost optimization, it is recommended that you quantify the additional value delivered. Cost optimization benefits are typically quantified in terms of lower costs per business outcome. For example, you can quantify Amazon Elastic Compute Cloud(Amazon EC2) cost savings when you purchase Savings Plans, which reduce cost and maintain workload output levels. You can quantify cost reductions in AWS spending when idle Amazon EC2 instances are removed, or unattached Amazon Elastic Block Store (Amazon EBS) volumes are deleted.

The benefits from cost optimization, however, go above and beyond cost reduction or avoidance. Consider capturing additional data to measure efficiency improvements and business value.

Implementation steps

- **Evaluate business benefits:** This is the process of analyzing and adjusting AWS Cloud cost in ways that maximize the benefit received from each dollar spent. Instead of focusing on cost reduction without business value, consider business benefits and return on investments for cost optimization, which may bring more value out of the money you spend. It's about spending wisely and making investments and expenditures in areas that yield the best return.
- **Analyze forecasting AWS costs:** Forecasting helps finance stakeholders set expectations with other internal and external organization stakeholders, and can improve your organization's financial predictability. [AWS Cost Explorer](#) can be used to perform forecasting for your cost and usage.

Resources

Related documents:

- [AWS Cloud Economics](#)
- [AWS Blog](#)
- [AWS Cost Management](#)
- [AWS News Blog](#)
- [Well-Architected Reliability Pillar whitepaper](#)
- [AWS Cost Explorer](#)

Related videos:

- [Unlock Business Value with Windows on AWS](#)

Related examples:

- [Measuring and Maximizing the Business Value of Customer 360](#)
- [The Business Value of Adopting Amazon Web Services Managed Databases](#)
- [The Business Value of Amazon Web Services for Independent Software Vendors](#)
- [Business Value of Cloud Modernization](#)

- [The Business Value of Migration to Amazon Web Services](#)

Expenditure and usage awareness

Questions

- [COST 2. How do you govern usage?](#)
- [COST 3. How do you monitor your cost and usage?](#)
- [COST 4. How do you decommission resources?](#)

COST 2. How do you govern usage?

Establish policies and mechanisms to verify that appropriate costs are incurred while objectives are achieved. By employing a checks-and-balances approach, you can innovate without overspending.

Best practices

- [COST02-BP01 Develop policies based on your organization requirements](#)
- [COST02-BP02 Implement goals and targets](#)
- [COST02-BP03 Implement an account structure](#)
- [COST02-BP04 Implement groups and roles](#)
- [COST02-BP05 Implement cost controls](#)
- [COST02-BP06 Track project lifecycle](#)

COST02-BP01 Develop policies based on your organization requirements

Develop policies that define how resources are managed by your organization and inspect them periodically. Policies should cover the cost aspects of resources and workloads, including creation, modification, and decommissioning over a resource's lifetime.

Level of risk exposed if this best practice is not established: High

Implementation guidance

Understanding your organization's costs and drivers is critical for managing your cost and usage effectively and identifying cost reduction opportunities. Organizations typically operate multiple workloads run by multiple teams. These teams can be in different organization units, each with

its own revenue stream. The capability to attribute resource costs to the workloads, individual organization, or product owners drives efficient usage behaviour and helps reduce waste. Accurate cost and usage monitoring helps you understand how optimized a workload is, as well as how profitable organization units and products are. This knowledge allows for more informed decision making about where to allocate resources within your organization. Awareness of usage at all levels in the organization is key to driving change, as change in usage drives changes in cost. Consider taking a multi-faceted approach to becoming aware of your usage and expenditures.

The first step in performing governance is to use your organization's requirements to develop policies for your cloud usage. These policies define how your organization uses the cloud and how resources are managed. Policies should cover all aspects of resources and workloads that relate to cost or usage, including creation, modification, and decommissioning over a resource's lifetime. Verify that policies and procedures are followed and implemented for any change in a cloud environment. During your IT change management meetings, raise questions to find out the cost impact of planned changes, whether increasing or decreasing, the business justification, and the expected outcome.

Policies should be simple so that they are easily understood and can be implemented effectively throughout the organization. Policies also need to be easy to follow and interpret (so they are used) and specific (no misinterpretation between teams). Moreover, they need to be inspected periodically (like our mechanisms) and updated as customers business conditions or priorities change, which would make the policy outdated.

Start with broad, high-level policies, such as which geographic Region to use or times of the day that resources should be running. Gradually refine the policies for the various organizational units and workloads. Common policies include which services and features can be used (for example, lower performance storage in test and development environments), which types of resources can be used by different groups (for example, the largest size of resource in a development account is medium) and how long these resources will be in use (whether temporary, short term, or for a specific period of time).

Policy example

The following is a sample policy you can review to create your own cloud governance policies, which focus on cost optimization. Make sure you adjust policy based on your organization's requirements and your stakeholders' requests.

- **Policy name:** Define a clear policy name, such as Resource Optimization and Cost Reduction Policy.

- **Purpose:** Explain why this policy should be used and what is the expected outcome. The objective of this policy is to verify that there is a minimum cost required to deploy and run the desired workload to meet business requirements.
- **Scope:** Clearly define who should use this policy and when it should be used, such as DevOps X Team to use this policy in us-east customers for X environment (production or non-production).

Policy statement

1. Select us-east-1 or multiple us-east regions based on your workload's environment and business requirement (development, user acceptance testing, pre-production, or production).
2. Schedule Amazon EC2 and Amazon RDS instances to run between six in the morning and eight at night (Eastern Standard Time (EST)).
3. Stop all unused Amazon EC2 instances after eight hours and unused Amazon RDS instances after 24 hours of inactivity.
4. Terminate all unused Amazon EC2 instances after 24 hours of inactivity in non-production environments. Remind Amazon EC2 instance owner (based on tags) to review their stopped Amazon EC2 instances in production and inform them that their Amazon EC2 instances will be terminated within 72 hours if they are not in use.
5. Use generic instance family and size such as m5.large and then resize the instance based on CPU and memory utilization using AWS Compute Optimizer.
6. Prioritize using auto scaling to dynamically adjust the number of running instances based on traffic.
7. Use spot instances for non-critical workloads.
8. Review capacity requirements to commit saving plans or reserved instances for predictable workloads and inform Cloud Financial Management Team.
9. Use Amazon S3 lifecycle policies to move infrequently accessed data to cheaper storage tiers. If no retention policy defined, use Amazon S3 Intelligent Tiering to move objects to archived tier automatically.
10. Monitor resource utilization and set alarms to trigger scaling events using Amazon CloudWatch.
11. For each AWS account, use AWS Budgets to set cost and usage budgets for your account based on cost center and business units.
12. Using AWS Budgets to set cost and usage budgets for your account can help you stay on top of your spending and avoid unexpected bills, allowing you to better control your costs.

Procedure: Provide detailed procedures for implementing this policy or refer to other documents that describe how to implement each policy statement. This section should provide step-by-step instructions for carrying out the policy requirements.

To implement this policy, you can use various third-party tools or AWS Config rules to check for compliance with the policy statement and trigger automated remediation actions using AWS Lambda functions. You can also use AWS Organizations to enforce the policy. Additionally, you should regularly review your resource usage and adjust the policy as necessary to verify that it continues to meet your business needs.

Implementation steps

- **Meet with stakeholders:** To develop policies, ask stakeholders (cloud business office, engineers, or functional decision makers for policy enforcement) within your organization to specify their requirements and document them. Take an iterative approach by starting broadly and continually refine down to the smallest units at each step. Team members include those with direct interest in the workload, such as organization units or application owners, as well as supporting groups, such as security and finance teams.
- **Get confirmation:** Make sure teams agree on policies who can access and deploy to the AWS Cloud. Make sure they follow your organization's policies and confirm that their resource creations align with the agreed policies and procedures.
- **Create onboarding training sessions:** Ask new organization members to complete onboarding training courses to create cost awareness and organization requirements. They may assume different policies from their previous experience or not think of them at all.
- **Define locations for your workload:** Define where your workload operates, including the country and the area within the country. This information is used for mapping to AWS Regions and Availability Zones.
- **Define and group services and resources:** Define the services that the workloads require. For each service, specify the types, the size, and the number of resources required. Define groups for the resources by function, such as application servers or database storage. Resources can belong to multiple groups.
- **Define and group the users by function:** Define the users that interact with the workload, focusing on what they do and how they use the workload, not on who they are or their position in the organization. Group similar users or functions together. You can use the AWS managed policies as a guide.
- **Define the actions:** Using the locations, resources, and users identified previously, define the actions that are required by each to achieve the workload outcomes over its life time

(development, operation, and decommission). Identify the actions based on the groups, not the individual elements in the groups, in each location. Start broadly with read or write, then refine down to specific actions to each service.

- **Define the review period:** Workloads and organizational requirements can change over time. Define the workload review schedule to ensure it remains aligned with organizational priorities.
- **Document the policies:** Verify the policies that have been defined are accessible as required by your organization. These policies are used to implement, maintain, and audit access of your environments.

Resources

Related documents:

- [Change Management in the Cloud](#)
- [AWS Managed Policies for Job Functions](#)
- [AWS multiple account billing strategy](#)
- [Actions, Resources, and Condition Keys for AWS Services](#)
- [AWS Management and Governance](#)
- [Control access to AWS Regions using IAM policies](#)
- [Global Infrastructures Regions and AZs](#)

Related videos:

- [AWS Management and Governance at Scale](#)

COST02-BP02 Implement goals and targets

Implement both cost and usage goals and targets for your workload. Goals provide direction to your organization on expected outcomes, and targets provide specific measurable outcomes to be achieved for your workloads.

Level of risk exposed if this best practice is not established: High

Implementation guidance

Develop cost and usage goals and targets for your organization. As a growing organization on AWS, it is important to set and track goals for cost optimization. These goals or [key performance](#)

[indicators \(KPIs\)](#) can include things like percent of spend on-demand or adoption of certain optimized services such as AWS Graviton instances or gp3 EBS volume types. Set measurable and achievable goals to help you measure efficiency improvements, which is important for your business operations. Goals provide guidance and direction to your organization on expected outcomes.

Targets provide specific, measurable outcomes to be achieved. In short, a goal is the direction you want to go, and a target is how far in that direction and when that goal should be achieved (use guidance of specific, measurable, assignable, realistic and timely, or SMART). An example of a goal is that platform usage should increase significantly, with only a minor (non-linear) increase in cost. An example target is a 20% increase in platform usage, with less than a five percent increase in costs. Another common goal is that workloads need to be more efficient every six months. The accompanying target would be that the cost per business metrics needs to decrease by five percent every six months. Use the right metrics, and set calculated KPIs for your organization. You can start with basic KPIs and evolve later based on business needs.

A goal for cost optimization is to increase workload efficiency, which corresponds to a decrease in the cost per business outcome of the workload over time. Implement this goal for all workloads, and set a target like a five percent increase in efficiency every six months to a year. In the cloud, you can achieve this through the establishment of capability in cost optimization, as well as new service and feature releases.

Targets are the quantifiable benchmarks you want to reach to meet your goals and benchmarks compare your actual results against a target. Establish benchmarks with KPIs for the cost per unit of compute services (such as Spot adoption, Graviton adoption, latest instance types, and On-Demands coverage), storage services (such as EBS GP3 adoption, obsolete EBS snapshots, and Amazon S3 standard storage), or database service usage (such as RDS open-source engines, Graviton adoption, and On-demand coverage). These benchmarks and KPIs can help you verify that you use AWS services in the most cost-effective manner.

The following table provides a list of standard AWS metrics for reference. Each organization can have different target values for these KPIs.

Category	KPI (%)	Description
Compute	EC2 usage Coverage	EC2 instances (in cost or hours) using SP+RI+Spot

Category	KPI (%)	Description
		compared to total (in cost or hours) of EC2 instances
Compute	Compute SP/RI utilization	Utilized SP or RI hours compared to total available SP or RI hours
Compute	EC2/Hour cost	EC2 cost divided by the number of EC2 instances running in that hour
Compute	vCPU cost	Cost per vCPU for all instances
Compute	Latest Instance Generation	Percentage of instances on Graviton (or other modern generation instance types)
Database	RDS coverage	RDS instances (in cost or hours) using RI compared to total (in cost or hours) of RDS instances
Database	RDS utilization	Utilized RI hours compared to total available RI hours
Database	RDS uptime	RDS cost divided by the number of RDS instances running in that hour
Database	Latest Instance Generation	Percentage of instances on Graviton (or other modern instance types)

Category	KPI (%)	Description
Storage	Storage utilization	Optimized storage cost (for example Glacier, deep archive, or Infrequent Access) divided by total storage cost
Tagging	Untagged resources	Cost Explorer: 1. Filter out credits, discounts , taxes, refunds, marketplace, and copy the latest monthly cost 2. Select Show only untagged resources in Cost Explorer 3. Divide the amount in untagged resources with your monthly cost.

Using this table, include target or benchmark values, which should be calculated based on your organizational goals. You need to measure certain metrics for your business and understand business outcome for that workload to define accurate and realistic KPIs. When you evaluate performance metrics within an organization, distinguish between different types of metrics that serve distinct purposes. These metrics primarily measure the performance and efficiency of the technical infrastructure rather than directly the overall business impact. For instance, they might track server response times, network latency, or system uptime. These metrics are crucial to assess how well the infrastructure supports the organization's technical operations. However, they don't provide direct insight into broader business objectives like customer satisfaction, revenue growth, or market share. To gain a comprehensive understanding of business performance, complement these efficiency metrics with strategic business metrics that directly correlate with business outcomes.

Establish near real-time visibility over your KPIs and related savings opportunities and track your progress over time. To get started with the definition and tracking of KPI goals, we recommend the KPI dashboard from [Cloud Intelligence Dashboards](#) (CID). Based on the data from Cost and Usage

Report (CUR), the KPI dashboard provides a series of recommended cost optimization KPIs, with the ability to set custom goals and track progress over time.

If you have other solutions to set and track KPI goals, make sure these methods are adopted by all cloud financial management stakeholders in your organization.

Implementation steps

- **Define expected usage levels:** To begin, focus on usage levels. Engage with the application owners, marketing, and greater business teams to understand what the expected usage levels are for the workload. How might customer demand change over time, and what can change due to seasonal increases or marketing campaigns?
- **Define workload resourcing and costs:** With usage levels defined, quantify the changes in workload resources required to meet those usage levels. You may need to increase the size or number of resources for a workload component, increase data transfer, or change workload components to a different service at a specific level. Specify the costs at each of these major points, and predict the change in cost when there is a change in usage.
- **Define business goals:** Take the output from the expected changes in usage and cost, combine this with expected changes in technology, or any programs that you are running, and develop goals for the workload. Goals must address usage and cost, as well as the relationship between the two. Goals must be simple, high-level, and help people understand what the business expects in terms of outcomes (such as making sure unused resources are kept below certain cost level). You don't need to define goals for each unused resource type or define costs that can cause losses in goals and targets. Verify that there are organizational programs (for example, capability building like training and education) if there are expected changes in cost without changes in usage.
- **Define targets:** For each of the defined goals, specify a measurable target. If the goal is to increase efficiency in the workload, the target should quantify the amount of improvement (typically in business outputs for each dollar spent) and when it should be delivered. For example, you could set a goal to minimize waste due to over-provisioning. With this goal, your target can be that waste due to compute over-provisioning in the first tier of production workloads should not exceed ten percent of tier compute cost. Additionally, a second target could be that waste due to compute over-provisioning in the second tier of production workloads should not exceed five percent of tier compute cost.

Resources

Related documents:

- [AWS managed policies for job functions](#)
- [AWS multiple account billing strategy](#)
- [Control access to AWS Regions using IAM policies](#)
- [S.M.A.R.T. Goals](#)
- [How to track your cost optimization KPIs with the CID KPI Dashboard](#)

Related videos:

- [Well-Architected Labs: Goals and Targets \(Level 100\)](#)

Related examples:

- [What is a unit metric?](#)
- [Selecting a unit metric to support your business](#)
- [Unit metrics in practice – lessons learned](#)
- [How unit metrics help create alignment between business functions](#)

COST02-BP03 Implement an account structure

Implement a structure of accounts that maps to your organization. This assists in allocating and managing costs throughout your organization.

Level of risk exposed if this best practice is not established: High

Implementation guidance

AWS Organizations allows you to create multiple AWS accounts which can help you centrally govern your environment as you scale your workloads on AWS. You can model your organizational hierarchy by grouping AWS accounts in organizational unit (OU) structure and creating multiple AWS accounts under each OU. To create an account structure, you need to decide which of your AWS accounts will be the management account first. After that, you can create new AWS accounts or select existing accounts as member accounts based on your designed account structure by following [management account best practices](#) and [member account best practices](#).

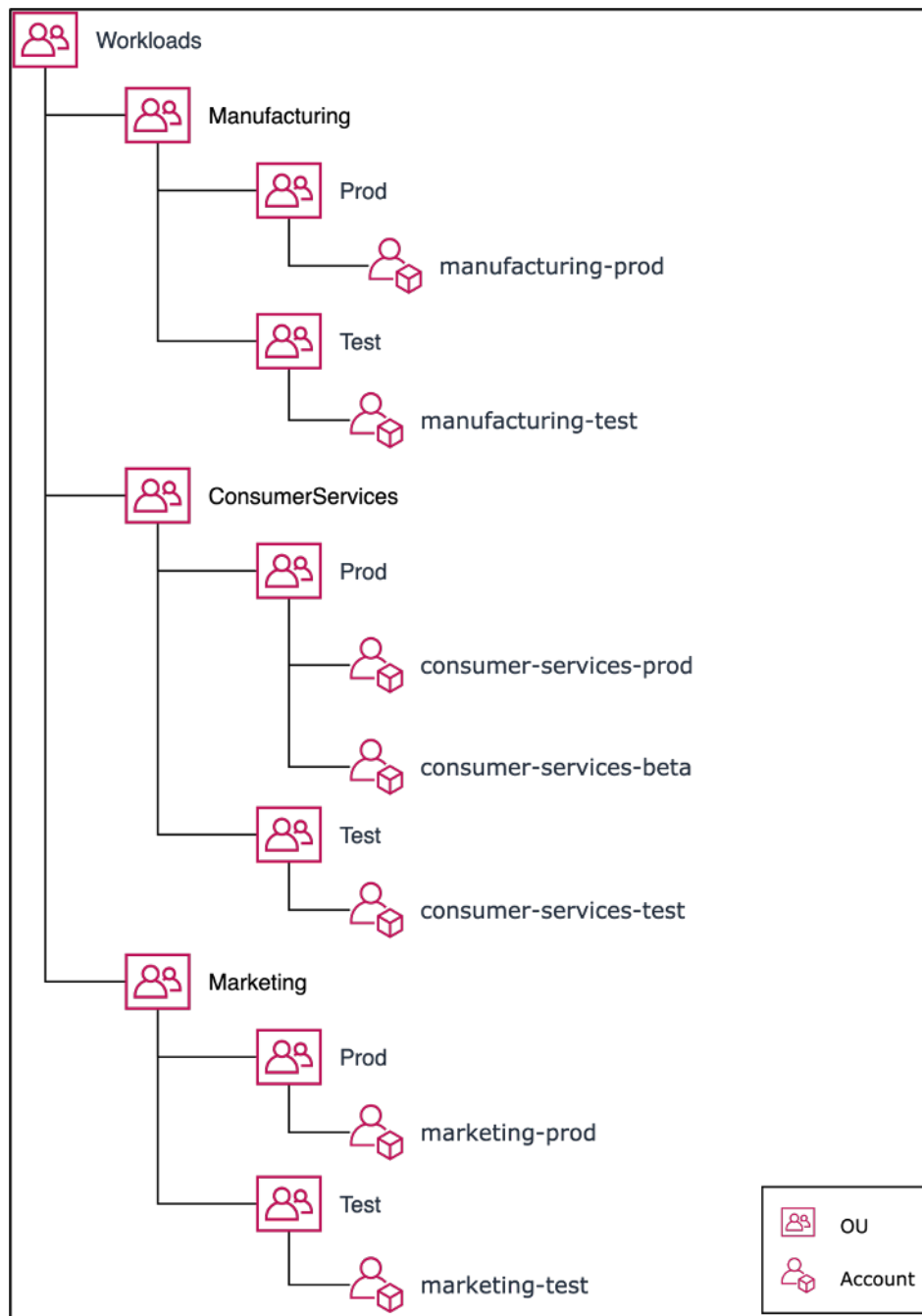
It is advised to always have at least one management account with one member account linked to it, regardless of your organization size or usage. All workload resources should reside only within member accounts and no resource should be created in management account. There is no one size fits all answer for how many AWS accounts you should have. Assess your current and future operational and cost models to ensure that the structure of your AWS accounts reflects your organization's goals. Some companies create multiple AWS accounts for business reasons, for example:

- Administrative or fiscal and billing isolation is required between organization units, cost centers, or specific workloads.
- AWS service limits are set to be specific to particular workloads.
- There is a requirement for isolation and separation between workloads and resources.

Within [AWS Organizations](#), [consolidated billing](#) creates the construct between one or more member accounts and the management account. Member accounts allow you to isolate and distinguish your cost and usage by groups. A common practice is to have separate member accounts for each organization unit (such as finance, marketing, and sales), or for each environment lifecycle (such as development, testing and production), or for each workload (workload a, b, and c), and then aggregate these linked accounts using consolidated billing.

Consolidated billing allows you to consolidate payment for multiple member AWS accounts under a single management account, while still providing visibility for each linked account's activity. As costs and usage are aggregated in the management account, this allows you to maximize your service volume discounts, and maximize the use of your commitment discounts (Savings Plans and Reserved Instances) to achieve the highest discounts.

The following diagram shows how you can use AWS Organizations with organizational units (OU) to group multiple accounts, and place multiple AWS accounts under each OU. It is recommended to use OUs for various use cases and workloads which provides patterns for organizing accounts.



Example of grouping multiple AWS accounts under organizational units.

[AWS Control Tower](#) can quickly set up and configure multiple AWS accounts, ensuring that governance is aligned with your organization's requirements.

Implementation steps

- **Define separation requirements:** Requirements for separation are a combination of multiple factors, including security, reliability, and financial constructs. Work through each factor in order

and specify whether the workload or workload environment should be separate from other workloads. Security promotes adherence to access and data requirements. Reliability manages limits so that environments and workloads do not impact others. Review the security and reliability pillars of the Well-Architected Framework periodically and follow the provided best practices. Financial constructs create strict financial separation (different cost center, workload ownerships and accountability). Common examples of separation are production and test workloads being run in separate accounts, or using a separate account so that the invoice and billing data can be provided to the individual business units or departments in the organization or stakeholder who owns the account.

- **Define grouping requirements:** Requirements for grouping do not override the separation requirements, but are used to assist management. Group together similar environments or workloads that do not require separation. An example of this is grouping multiple test or development environments from one or more workloads together.
- **Define account structure:** Using these separations and groupings, specify an account for each group and maintain separation requirements. These accounts are your member or linked accounts. By grouping these member accounts under a single management or payer account, you combine usage, which allows for greater volume discounts across all accounts, which provides a single bill for all accounts. It's possible to separate billing data and provide each member account with an individual view of their billing data. If a member account must not have its usage or billing data visible to any other account, or if a separate bill from AWS is required, define multiple management or payer accounts. In this case, each member account has its own management or payer account. Resources should always be placed in member or linked accounts. The management or payer accounts should only be used for management.

Resources

Related documents:

- [Using Cost Allocation Tags](#)
- [AWS managed policies for job functions](#)
- [AWS multiple account billing strategy](#)
- [Control access to AWS Regions using IAM policies](#)
- [AWS Control Tower](#)
- [AWS Organizations](#)
- Best practices for [management accounts](#) and [member accounts](#)

- [Organizing Your AWS Environment Using Multiple Accounts](#)
- [Turning on shared reserved instances and Savings Plans discounts](#)
- [Consolidated billing](#)
- [Consolidated billing](#)

Related examples:

- [Splitting the CUR and Sharing Access](#)

Related videos:

- [Introducing AWS Organizations](#)
- [Set Up a Multi-Account AWS Environment that Uses Best Practices for AWS Organizations](#)

Related examples:

- [Defining an AWS Multi-Account Strategy for telecommunications companies](#)
- [Best Practices for Optimizing AWS accounts](#)
- [Best Practices for Organizational Units with AWS Organizations](#)

COST02-BP04 Implement groups and roles

Implement groups and roles that align to your policies and control who can create, modify, or decommission instances and resources in each group. For example, implement development, test, and production groups. This applies to AWS services and third-party solutions.

Level of risk exposed if this best practice is not established: Low

Implementation guidance

User roles and groups are fundamental building blocks in the design and implementation of secure and efficient systems. Roles and groups help organizations balance the need for control with the requirement for flexibility and productivity, ultimately supporting organizational objectives and user needs. As recommended in [Identity and access management](#) section of AWS Well-Architected Framework Security Pillar, you need robust identity management and permissions in place to provide access to the right resources for the right people under the right conditions. Users

receive only the access necessary to complete their tasks. This minimizes the risk associated with unauthorized access or misuse.

After you develop policies, you can create logical groups and user roles within your organization. This allows you to assign permissions, control usage, and help implement robust access control mechanisms, preventing unauthorized access to sensitive information. Begin with high-level groupings of people. Typically, this aligns with organizational units and job roles (for example, a systems administrator in the IT Department, financial controller, or business analysts). The groups categorize people that do similar tasks and need similar access. Roles define what a group must do. It is easier to manage permissions for groups and roles than for individual users. Roles and groups assign permissions consistently and systematically across all users, preventing errors and inconsistencies.

When a user's role changes, administrators can adjust access at the role or group level, rather than reconfiguring individual user accounts. For example, a systems administrator in IT requires access to create all resources, but an analytics team member only needs to create analytics resources.

Implementation steps

- **Implement groups:** Using the groups of users defined in your organizational policies, implement the corresponding groups, if necessary. For best practices on users, groups and authentication, see the [Security Pillar](#) of the AWS Well-Architected Framework.
- **Implement roles and policies:** Using the actions defined in your organizational policies, create the required roles and access policies. For best practices on roles and policies, see the [Security Pillar](#) of the AWS Well-Architected Framework.

Resources

Related documents:

- [AWS managed policies for job functions](#)
- [AWS multiple account billing strategy](#)
- [AWS Well-Architected Framework Security Pillar](#)
- [AWS Identity and Access Management \(IAM\)](#)
- [AWS Identity and Access Management policies](#)

Related videos:

- [Why use Identity and Access Management](#)

Related examples:

- [Control access to AWS Regions using IAM policies](#)
- [Starting your Cloud Financial Management journey: Cloud cost operations](#)

COST02-BP05 Implement cost controls

Implement controls based on organization policies and defined groups and roles. These certify that costs are only incurred as defined by organization requirements such as control access to regions or resource types.

Level of risk exposed if this best practice is not established: Medium

Implementation guidance

A common first step in implementing cost controls is to set up notifications when cost or usage events occur outside of policies. You can act quickly and verify if corrective action is required without restricting or negatively impacting workloads or new activity. After you know the workload and environment limits, you can enforce governance. [AWS Budgets](#) allows you to set notifications and define monthly budgets for your AWS costs, usage, and commitment discounts (Savings Plans and Reserved Instances). You can create budgets at an aggregate cost level (for example, all costs), or at a more granular level where you include only specific dimensions such as linked accounts, services, tags, or Availability Zones.

Once you set up your budget limits with AWS Budgets, use [AWS Cost Anomaly Detection](#) to reduce your unexpected cost. AWS Cost Anomaly Detection is a cost management service that uses machine learning to continually monitor your cost and usage to detect unusual spends. It helps you identify anomalous spend and root causes, so you can quickly take action. First, create a cost monitor in AWS Cost Anomaly Detection, then choose your alerting preference by setting up a dollar threshold (such as an alert on anomalies with impact greater than \$1,000). Once you receive alerts, you can analyze the root cause behind the anomaly and impact on your costs. You can also monitor and perform your own anomaly analysis in AWS Cost Explorer.

Enforce governance policies in AWS through [AWS Identity and Access Management](#) and [AWS Organizations Service Control Policies \(SCP\)](#). IAM allows you to securely manage access to AWS services and resources. Using IAM, you can control who can create or manage AWS resources,

the type of resources that can be created, and where they can be created. This minimizes the possibility of resources being created outside of the defined policy. Use the roles and groups created previously and assign [IAM policies](#) to enforce the correct usage. SCP offers central control over the maximum available permissions for all accounts in your organization, keeping your accounts stay within your access control guidelines. SCPs are available only in an organization that has all features turned on, and you can configure the SCPs to either deny or allow actions for member accounts by default. For more details on implementing access management, see the [Well-Architected Security Pillar whitepaper](#).

Governance can also be implemented through management of [AWS service quotas](#). By ensuring service quotas are set with minimum overhead and accurately maintained, you can minimize resource creation outside of your organization's requirements. To achieve this, you must understand how quickly your requirements can change, understand projects in progress (both creation and decommissioning of resources), and factor in how fast quota changes can be implemented. [Service quotas](#) can be used to increase your quotas when required.

Implementation steps

- **Implement notifications on spend:** Using your defined organization policies, create [AWS Budgets](#) to notify you when spending is outside of your policies. Configure multiple cost budgets, one for each account, which notify you about overall account spending. Configure additional cost budgets within each account for smaller units within the account. These units vary depending on your account structure. Some common examples are AWS Regions, workloads (using tags), or AWS services. Configure an email distribution list as the recipient for notifications, and not an individual's email account. You can configure an actual budget for when an amount is exceeded, or use a forecasted budget for notifying on forecasted usage. You can also preconfigure AWS Budget Actions that can enforce specific IAM or SCP policies, or stop target Amazon EC2 or Amazon RDS instances. Budget Actions can be started automatically or require workflow approval.
- **Implement notifications on anomalous spend:** Use [AWS Cost Anomaly Detection](#) to reduce your surprise costs in your organization and analyze root cause of potential anomalous spend. Once you create cost monitor to identify unusual spend at your specified granularity and configure notifications in AWS Cost Anomaly Detection, it sends you alert when unusual spend is detected. This will allow you to analyze root cause behind the anomaly and understand the impact on your cost. Use AWS Cost Categories while configuring AWS Cost Anomaly Detection to identify which project team or business unit team can analyze the root cause of the unexpected cost and take timely necessary actions.

- **Implement controls on usage:** Using your defined organization policies, implement IAM policies and roles to specify which actions users can perform and which actions they cannot. Multiple organizational policies may be included in an AWS policy. In the same way that you defined policies, start broadly and then apply more granular controls at each step. Service limits are also an effective control on usage. Implement the correct service limits on all your accounts.

Resources

Related documents:

- [AWS managed policies for job functions](#)
- [AWS multiple account billing strategy](#)
- [Control access to AWS Regions using IAM policies](#)
- [AWS Budgets](#)
- [AWS Cost Anomaly Detection](#)
- [Control Your AWS Costs](#)

Related videos:

- [How can I use AWS Budgets to track my spending and usage](#)

Related examples:

- [Example IAM access management policies](#)
- [Example service control policies](#)
- [AWS Budgets Actions](#)
- [Create IAM Policy to control access to Amazon EC2 resources using Tags](#)
- [Restrict the access of IAM Identity to specific Amazon EC2 resources](#)
- [Slack integrations for Cost Anomaly Detection using Amazon Q Developer in chat applications](#)

COST02-BP06 Track project lifecycle

Track, measure, and audit the lifecycle of projects, teams, and environments to avoid using and paying for unnecessary resources.

Level of risk exposed if this best practice is not established: Low

Implementation guidance

By effectively tracking the project lifecycle, organizations can achieve better cost control through enhanced planning, management, and resource optimization. The insights gained through tracking are invaluable for making informed decisions that contribute to the cost-effectiveness and overall success of the project.

Tracking the entire lifecycle of the workload helps you understand when workloads or workload components are no longer required. The existing workloads and components may appear to be in use, but when AWS releases new services or features, they can be decommissioned or adopted. Check the previous stages of workloads. After a workload is in production, previous environments can be decommissioned or greatly reduced in capacity until they are required again.

You can tag resources with a timeframe or reminder to pin the time that the workload was reviewed. For example, if the development environment was last reviewed months ago, it could be a good time to review it again to explore if new services can be adopted or if the environment is in use. You can group and tag your applications with [myApplications](#) on AWS to manage and track metadata such as criticality, environment, last reviewed, and cost center. You can both track your workload's lifecycle and monitor and manage the cost, health, security posture, and performance of your applications.

AWS provides various management and governance services you can use for entity lifecycle tracking. You can use [AWS Config](#) or [AWS Systems Manager](#) to provide a detailed inventory of your AWS resources and configuration. It is recommended that you integrate with your existing project or asset management systems to keep track of active projects and products within your organization. Combining your current system with the rich set of events and metrics provided by AWS allows you to build a view of significant lifecycle events and proactively manage resources to reduce unnecessary costs.

Similar to [Application Lifecycle Management \(ALM\)](#), tracking project lifecycle should involve multiple processes, tools, and teams working together, such as design and development, testing, production, support, and workload redundancy.

By carefully monitoring each phase of a project's lifecycle, organizations gain crucial insights and enhanced control, facilitating successful project planning, implementation, and completion. This careful oversight verifies that projects not only meet quality standards, but are delivered on time and within budget, promoting overall cost efficiency.

For more details on implementing entity lifecycle tracking, see [AWS Well-Architected Operational Excellence Pillar whitepaper](#).

Implementation steps

- **Establish project lifecycle monitoring process:** [The Cloud Center of Excellence team](#) must establish project lifecycle monitoring process. Establish a structured and systematic approach to monitoring workloads in order to improve control, visibility, and performance of the projects. Make the monitoring process transparent, collaborative, and focused on continuous improvement to maximize its effectiveness and value.
- **Perform workload reviews:** As defined by your organizational policies, set up a regular cadence to audit your existing projects and perform workload reviews. The amount of effort spent in the audit should be proportional to the approximate risk, value, or cost to the organization. Key areas to include in the audit would be risk to the organization of an incident or outage, value, or contribution to the organization (measured in revenue or brand reputation), cost of the workload (measured as total cost of resources and operational costs), and usage of the workload (measured in number of organization outcomes per unit of time). If these areas change over the lifecycle, adjustments to the workload are required, such as full or partial decommissioning.

Resources

Related documents:

- [Guidance for Tagging on AWS](#)
- [What Is ALM \(Application Lifecycle Management\)?](#)
- [AWS managed policies for job functions](#)

Related examples:

- [Control access to AWS Regions using IAM policies](#)

Related Tools

- [AWS Config](#)
- [AWS Systems Manager](#)
- [AWS Budgets](#)
- [AWS Organizations](#)
- [AWS CloudFormation](#)

COST 3. How do you monitor your cost and usage?

Establish policies and procedures to monitor and appropriately allocate your costs. This permits you to measure and improve the cost efficiency of this workload.

Best practices

- [COST03-BP01 Configure detailed information sources](#)
- [COST03-BP02 Add organization information to cost and usage](#)
- [COST03-BP03 Identify cost attribution categories](#)
- [COST03-BP04 Establish organization metrics](#)
- [COST03-BP05 Configure billing and cost management tools](#)
- [COST03-BP06 Allocate costs based on workload metrics](#)

COST03-BP01 Configure detailed information sources

Set up cost management and reporting tools for enhanced analysis and transparency of cost and usage data. Configure your workload to create log entries that facilitate the tracking and segregation of costs and usage.

Level of risk exposed if this best practice is not established: High

Implementation guidance

Detailed billing information such as hourly granularity in cost management tools allow organizations to track their consumptions with further details and help them to identify some of the cost increase reasons. These data sources provide the most accurate view of cost and usage across your entire organization.

You can use AWS Data Exports to create exports of the AWS Cost and Usage Report (CUR) 2.0. This is the new and recommended way to receive your detailed cost and usage data from AWS. It provides daily or hourly usage granularity, rates, costs, and usage attributes for all chargeable AWS services (the same information as CUR), along with some improvements. All possible dimensions are in the CUR such as tagging, location, resource attributes, and account IDs.

There are three export types based on the type of export you want to create: a standard data export, an export to a cost and usage dashboard with Quick Suite integration, or a legacy data export.

- **Standard data export:** A customized export of a table that delivers to Amazon S3 on a recurring basis.
- **Cost and usage dashboard:** An export and integration to Quick Suite to deploy a pre-built cost and usage dashboard.
- **Legacy data export:** An export of the legacy AWS Cost and Usage Report (CUR).

You can create data exports with the following customizations:

- Include resource IDs
- Split cost allocation data
- Hourly granularity
- Versioning
- Compression type and file format

For your workloads that run containers on Amazon ECS or Amazon EKS, enable split cost allocation data so that you can allocate your container costs to individual business units and teams, based on how your container workloads consume shared compute and memory resources. Split cost allocation data introduces cost and usage data for new container-level resources to AWS Cost and Usage Report. Split cost allocation data is calculated by computing the cost of individual ECS services and tasks running on the cluster.

A cost and usage dashboard exports the cost and usage dashboard table to an S3 bucket on a recurring basis and deploys a prebuilt cost and usage dashboard to Quick Suite. Use this option if you want to quickly deploy a dashboard of your cost and usage data without the ability for customization.

If desired, you can still export CUR in legacy mode, where you can integrate other processing services such as [AWS Glue](#) to prepare the data for analysis and perform data analysis with [Amazon Athena](#) using SQL to query the data.

Implementation steps

- **Create data exports:** Create customized exports with the data you want and control the schema of your exports. Create billing and cost management data exports using basic SQL, and visualize your billing and cost management data by integrating with Quick Suite. You can also export your data in standard mode to analyze your data with other processing tools like Amazon Athena.

- **Configure the cost and usage report:** Using the billing console, configure at least one cost and usage report. Configure a report with hourly granularity that includes all identifiers and resource IDs. You can also create other reports with different granularities to provide higher-level summary information.
- **Configure hourly granularity in Cost Explorer:** To access cost and usage data with hourly granularity for the past 14 days, consider enabling hourly and resource level data in the billing console.
- **Configure application logging:** Verify that your application logs each business outcome that it delivers so it can be tracked and measured. Ensure that the granularity of this data is at least hourly so it matches with the cost and usage data. For more details on logging and monitoring, see [Well-Architected Operational Excellence Pillar](#).

Resources

Related documents:

- [AWS Data Exports](#)
- [AWS Glue](#)
- [Quick Suite](#)
- [AWS Cost Management Pricing](#)
- [Tagging AWS resources](#)
- [Analyzing your costs with Cost Explorer](#)
- [Managing AWS Cost and Usage Reports](#)

Related examples:

- [AWS Account Setup](#)
- [Data Exports for AWS Billing and Cost Management](#)
- [AWS Cost Explorer Common Use Cases](#)

COST03-BP02 Add organization information to cost and usage

Define a tagging schema based on your organization, workload attributes, and cost allocation categories so that you can filter and search for resources or monitor cost and usage in cost

management tools. Implement consistent tagging across all resources where possible by purpose, team, environment, or other criteria relevant to your business.

Level of risk exposed if this best practice is not established: Medium

Implementation guidance

Implement [tagging in AWS](#) to add organization information to your resources, which will then be added to your cost and usage information. A tag is a key-value pair — the key is defined and must be unique across your organization, and the value is unique to a group of resources. An example of a key-value pair is the key is `Environment`, with a value of `Production`. All resources in the production environment will have this key-value pair. Tagging allows you categorize and track your costs with meaningful, relevant organization information. You can apply tags that represent organization categories (such as cost centers, application names, projects, or owners), and identify workloads and characteristics of workloads (such as test or production) to attribute your costs and usage throughout your organization.

When you apply tags to your AWS resources (such as Amazon Elastic Compute Cloud instances or Amazon Simple Storage Service buckets) and activate the tags, AWS adds this information to your Cost and Usage Reports. You can run reports and perform analysis on tagged and untagged resources to allow greater compliance with internal cost management policies and ensure accurate attribution.

Creating and implementing an AWS tagging standard across your organization's accounts helps you manage and govern your AWS environments in a consistent and uniform manner. Use [Tag Policies](#) in AWS Organizations to define rules for how tags can be used on AWS resources in your accounts in AWS Organizations. Tag Policies allow you to easily adopt a standardized approach for tagging AWS resources

[AWS Tag Editor](#) allows you to add, delete, and manage tags of multiple resources. With Tag Editor, you search for the resources that you want to tag, and then manage tags for the resources in your search results.

[AWS Cost Categories](#) allows you to assign organization meaning to your costs, without requiring tags on resources. You can map your cost and usage information to unique internal organization structures. You define category rules to map and categorize costs using billing dimensions, such as accounts and tags. This provides another level of management capability in addition to tagging. You can also map specific accounts and tags to multiple projects.

Implementation steps

- **Define a tagging schema:** Gather all stakeholders from across your business to define a schema. This typically includes people in technical, financial, and management roles. Define a list of tags that all resources must have, as well as a list of tags that resources should have. Verify that the tag names and values are consistent across your organization.
- **Tag resources:** Using your defined cost attribution categories, [place tags](#) on all resources in your workloads according to the categories. Use tools such as the CLI, Tag Editor, or AWS Systems Manager to increase efficiency.
- **Implement AWS Cost Categories:** You can create [Cost Categories](#) without implementing tagging. Cost categories use the existing cost and usage dimensions. Create category rules from your schema and implement them into cost categories.
- **Automate tagging:** To verify that you maintain high levels of tagging across all resources, automate tagging so that resources are automatically tagged when they are created. Use services such as [AWS CloudFormation](#) to verify that resources are tagged when created. You can also create a custom solution to tag automatically using Lambda functions or use a microservice that scans the workload periodically and removes any resources that are not tagged, which is ideal for test and development environments.
- **Monitor and report on tagging:** To verify that you maintain high levels of tagging across your organization, report and monitor the tags across your workloads. You can use [AWS Cost Explorer](#) to view the cost of tagged and untagged resources, or use services such as [Tag Editor](#). Regularly review the number of untagged resources and take action to add tags until you reach the desired level of tagging.

Resources

Related documents:

- [Tagging Best Practices](#)
- [AWS CloudFormation Resource Tag](#)
- [AWS Cost Categories](#)
- [Tagging AWS resources](#)
- [Analyzing your costs with AWS Budgets](#)
- [Analyzing your costs with Cost Explorer](#)
- [Managing AWS Cost and Usage Reports](#)

Related videos:

- [How can I tag my AWS resources to divide up my bill by cost center or project](#)
- [Tagging AWS Resources](#)

COST03-BP03 Identify cost attribution categories

Identify organization categories such as business units, departments or projects that could be used to allocate cost within your organization to the internal consuming entities. Use those categories to enforce spend accountability, create cost awareness and drive effective consumption behaviors.

Level of risk exposed if this best practice is not established: High

Implementation guidance

The process of categorizing costs is crucial in budgeting, accounting, financial reporting, decision making, benchmarking, and project management. By classifying and categorizing expenses, teams can gain a better understanding of the types of costs they incur throughout their cloud journey helping teams make informed decisions and manage budgets effectively.

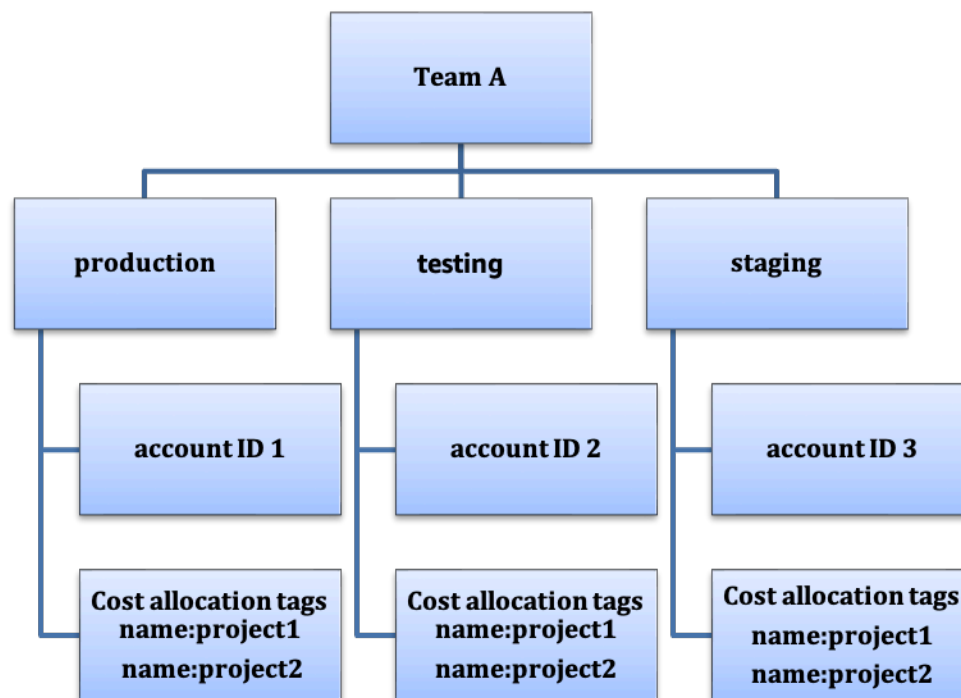
Cloud spend accountability establishes a strong incentive for disciplined demand and cost management. The result is significantly greater cloud cost savings for organizations that allocate most of their cloud spend to consuming business units or teams. Moreover, allocating cloud spend helps organizations adopt more best practices of centralized cloud governance.

Work with your finance team and other relevant stakeholders to understand the requirements of how costs must be allocated within your organization during your regular cadence calls. Workload costs must be allocated throughout the entire lifecycle, including development, testing, production, and decommissioning. Understand how the costs incurred for learning, staff development, and idea creation are attributed in the organization. This can be helpful to correctly allocate accounts used for this purpose to training and development budgets instead of generic IT cost budgets.

After defining your cost attribution categories with stakeholders in your organization, use [AWS Cost Categories](#) to group your cost and usage information into meaningful categories in the AWS Cloud, such as cost for a specific project, or AWS accounts for departments or business units. You can create custom categories and map your cost and usage information into these categories based on rules you define using various dimensions such as account, tag, service, or charge type. Once cost categories are set up, you can view your cost and usage information by these categories, which allows your organization to make better strategic and purchasing decisions. These categories are visible in AWS Cost Explorer, AWS Budgets, and AWS Cost and Usage Report as well.

For example, create cost categories for your business units (DevOps team), and under each category create multiple rules (rules for each sub category) with multiple dimensions (AWS accounts, cost allocation tags, services or charge type) based on your defined groupings. With cost categories, you can organize your costs using a rule-based engine. The rules that you configure organize your costs into categories. Within these rules, you can filter by using multiple dimensions for each category such as specific AWS accounts, AWS services, or charge types. You can then use these categories across multiple products in the [AWS Billing and Cost Management and Cost Management console](#). This includes AWS Cost Explorer, AWS Budgets, AWS Cost and Usage Report, and AWS Cost Anomaly Detection.

As an example, the following diagram displays how to group your costs and usage information in your organization by having multiple teams (cost category), multiple environments (rules), and each environment having multiple resources or assets (dimensions).



Cost and usage organization chart

You can create groupings of costs using cost categories as well. After you create the cost categories (allowing up to 24 hours after creating a cost category for your usage records to be updated with values), they appear in [AWS Cost Explorer](#), [AWS Budgets](#), [AWS Cost and Usage Report](#), and [AWS Cost Anomaly Detection](#). In AWS Cost Explorer and AWS Budgets, a cost category appears as an additional billing dimension. You can use this to filter for the specific cost category value, or group by the cost category.

Implementation steps

- **Define your organization categories:** Meet with internal stakeholders and business units to define categories that reflect your organization's structure and requirements. These categories should directly map to the structure of existing financial categories, such as business unit, budget, cost center, or department. Look at the outcomes the cloud delivers for your business such as training or education, as these are also organization categories.
- **Define your functional categories:** Meet with internal stakeholders and business units to define categories that reflect the functions that you have within your business. This may be the workload or application names, and the type of environment, such as production, testing, or development.
- **Define AWS Cost Categories:** Create cost categories to organize your cost and usage information by using [AWS Cost Categories](#) and map your AWS cost and usage into [meaningful categories](#). Multiple categories can be assigned to a resource, and a resource can be in multiple different categories, so define as many categories as needed so that you can [manage your costs](#) within the categorized structure using AWS Cost Categories.

Resources

Related documents:

- [Tagging AWS resources](#)
- [Using Cost Allocation Tags](#)
- [Analyzing your costs with AWS Budgets](#)
- [Analyzing your costs with Cost Explorer](#)
- [Managing AWS Cost and Usage Reports](#)
- [AWS Cost Categories](#)
- [Managing your costs with AWS Cost Categories](#)
- [Creating cost categories](#)
- [Tagging cost categories](#)
- [Splitting charges within cost categories](#)
- [AWS Cost Categories Features](#)

Related examples:

- [Organize your cost and usage data with AWS Cost Categories](#)
- [Managing your costs with AWS Cost Categories](#)

COST03-BP04 Establish organization metrics

Establish the organization metrics that are required for this workload. Example metrics of a workload are customer reports produced, or web pages served to customers.

Level of risk exposed if this best practice is not established: High

Implementation guidance

Understand how your workload's output is measured against business success. Each workload typically has a small set of major outputs that indicate performance. If you have a complex workload with many components, then you can prioritize the list, or define and track metrics for each component. Work with your teams to understand which metrics to use. This unit will be used to understand the efficiency of the workload, or the cost for each business output.

Implementation steps

- **Define workload outcomes:** Meet with the stakeholders in the business and define the outcomes for the workload. These are a primary measure of customer usage and must be business metrics and not technical metrics. There should be a small number of high-level metrics (less than five) per workload. If the workload produces multiple outcomes for different use cases, then group them into a single metric.
- **Define workload component outcomes:** Optionally, if you have a large and complex workload, or can easily break your workload into components (such as microservices) with well-defined inputs and outputs, define metrics for each component. The effort should reflect the value and cost of the component. Start with the largest components and work towards the smaller components.

Resources

Related documents:

- [Tagging AWS resources](#)
- [Analyzing your costs with AWS Budgets](#)
- [Analyzing your costs with Cost Explorer](#)

- [Managing AWS Cost and Usage Reports](#)

COST03-BP05 Configure billing and cost management tools

Configure cost management tools that meet your organization's policies to manage and optimize cloud spending. This includes services, tools, and resources to organize and track cost and usage data, enhance control through consolidated billing and access permission, improve planning through budgeting and forecasts, receive notifications or alerts, and lower cost with resources and pricing optimizations.

Level of risk exposed if this best practice is not established: High

Implementation guidance

To establish strong accountability, consider your account strategy first as part of your cost allocation strategy. Get this right, and you may not need to go any further. Otherwise, there can be unawareness and further pain points.

To encourage accountability of cloud spend, grant users access to tools that provide visibility into their costs and usage. AWS recommends that you configure all workloads and teams for the following purposes:

- **Organize:** Establish your cost allocation and governance baseline with your own tagging strategy and taxonomy. Create multiple AWS Accounts with tools such as AWS Control Tower or AWS Organization. Tag the supported AWS resources and categorize them meaningfully based on your organization structure (business units, departments, or projects). Tag account names for specific cost centers and map them with AWS Cost Categories to group accounts for business units to their cost centers so that business unit owner can see multiple accounts' consumption in one place.
- **Access:** Track organization-wide billing information in consolidated billing. Verify the right stakeholders and business owners have access.
- **Control:** Build effective governance mechanisms with the right guardrails to prevent unexpected scenarios when using Service Control Policies (SCP), tag policies, IAM policies and budget alerts. For example, you can allow teams to create specific resources in preferred regions only by using effective control mechanisms and prevent resource creations without specific tag (such as cost-center).
- **Current state:** Configure a dashboard that shows current levels of cost and usage. The dashboard should be available in a highly visible place within the work environment like an operations

dashboard. You can export data and use the Cost and Usage Dashboard from the AWS Cost Optimization Hub or any supported product to create this visibility. You may need to create different dashboards for different personas. For example, manager dashboard may differ from an engineering dashboard.

- **Notifications:** Provide notifications when cost or usage exceeds defined limits and anomalies occur with AWS Budgets or AWS Cost Anomaly Detection.
- **Reports:** Summarize all cost and usage information. Raise awareness and accountability of your cloud spend with detailed, attributable cost data. Create reports that are relevant to the team consuming them and contain recommendations.
- **Tracking:** Show the current cost and usage against configured goals or targets.
- **Analysis:** Allow team members to perform custom and deep analysis down to the hourly, daily or monthly granularity with different filters (resource, account, tag, etc.).
- **Inspect:** Stay up to date with your resource deployment and cost optimization opportunities. Get notifications using Amazon CloudWatch, Amazon SNS, or Amazon SES for resource deployments at the organization level. Review cost optimization recommendations with AWS Trusted Advisor or AWS Compute Optimizer.
- **Trend reports:** Display the variability in cost and usage over the required period with the required granularity.
- **Forecasts:** Show estimated future costs, estimate your resource usage, and spend with forecast dashboards you create.

You can use [AWS Cost Optimization Hub](#) to understand potential cost-saving opportunities consolidated from a centralized location and create data exports for integration with Amazon Athena. You can also use the AWS Cost Optimization Hub to deploy the Cost and Usage Dashboard, which utilizes Quick Suite for interactive cost analysis and secure cost insight sharing.

If you don't have essential skills or bandwidth in your organization, you can work with [AWS ProServ](#), [AWS Managed Services \(AMS\)](#), or [AWS Partners](#). You can also use third-party tools but ensure you validate the value proposition.

Implementation steps

- **Allow team-based access to tools:** Configure your accounts and create groups that have access to the required cost and usage reports for their consumptions and use [AWS Identity and Access Management](#) to [control access](#) to the tools such as AWS Cost Explorer. These groups must

include representatives from all teams that own or manage an application. This certifies that every team has access to their cost and usage information to track their consumption.

- **Organize Costs Tags and Categories:** organize your costs across teams, business units, applications, environments, and projects. Use resource tags to organize costs, by cost allocation tags. Create Cost Categories based on the dimensions by using tags, accounts, services, etc. to map your costs.
- **Configure AWS Budgets:** [Configure AWS Budgets](#) on all accounts for your workloads. Set budgets for the overall account spend, and budgets for the workloads by using tags and cost categories. Configure notifications in AWS Budgets to receive alerts for when you exceed your budgeted amounts, or when your estimated costs exceed your budgets.
- **Configure AWS Cost Anomaly Detection:** Use [AWS Cost Anomaly Detection](#) for your accounts, core services or cost categories you created to monitor your cost and usage and detect unusual spends. You can receive alerts individually in aggregated reports and receive alerts in an email or an Amazon SNS topic which allows you to analyze and determine the root cause of the anomaly and identify the factor that is driving the cost increase.
- **Use cost analysis tools:** Configure [AWS Cost Explorer](#) for your workload and accounts to visualize your cost data for further analysis. Create a dashboard for the workload that tracks overall spend, key usage metrics for the workload, and forecast of future costs based on your historical cost data.
- **Use cost-saving analysis tools:** Use AWS Cost Optimization Hub to identify savings opportunities with tailored recommendations including deleting unused resources, rightsizing, savings Plans, reservations and compute optimizer recommendations.
- **Configure advanced tools:** You can optionally create visuals to facilitate interactive analysis and sharing of cost insights. With Data Exports on AWS Cost Optimization Hub, you can create cost and usage dashboard powered by Quick Suite for your organization that provides additional detail and granularity. You can also implement advanced analysis capability by using data exports in [Amazon Athena](#) for advanced queries, and create dashboards on [Quick Suite](#). Work with [AWS Partners](#) to adopt cloud management solutions for consolidated cloud bill monitoring and optimization.

Resources

Related documents:

- [What is AWS Billing and Cost Management and Cost Management?](#)
- [Establishing your best practice AWS environment](#)

- [Best Practices for Tagging AWS Resources](#)
- [Tagging your AWS resources](#)
- [AWS Cost Categories](#)
- [Analyzing your costs with AWS Budgets](#)
- [Analyzing your costs with AWS Cost Explorer](#)
- [What is AWS Data Exports?](#)

Related videos:

- [Deploying Cloud Intelligence Dashboards](#)
- [Get Alerts on any FinOps or Cost Optimization Metric or KPI](#)

Related examples:

- [Cost and Usage Dashboard powered by Quick Suite](#)
- [AWS Cost and Usage Governance Workshop](#)

COST03-BP06 Allocate costs based on workload metrics

Allocate the workload's costs based on usage metrics or business outcomes to measure workload cost efficiency. Implement a process to analyze the cost and usage data with analytics services, which can provide insight and charge back capability.

Level of risk exposed if this best practice is not established: Low

Implementation guidance

Cost optimization means delivering business outcomes at the lowest price point, which can only be achieved by allocating workload costs based on workload metrics (measured by workload efficiency). Monitor the defined workload metrics through log files or other application monitoring. Combine this data with the workload's costs, which can be obtained by looking at costs with a specific tag value or account ID. Perform this analysis at the hourly level. Your efficiency typically changes if you have static cost components (for example, a backend database running permanently) with a varying request rate (for example, usage peaks at nine in the morning to five in the evening, with few requests at night). Understanding the relationship between the static and variable costs helps you focus your optimization activities.

Creating workload metrics for shared resources may be challenging compared to resources like containerized applications on Amazon Elastic Container Service (Amazon ECS) and Amazon API Gateway. However, there are certain ways you can categorize usage and track cost. If you need to track Amazon ECS and AWS Batch shared resources, you can enable split cost allocation data in AWS Cost Explorer. With split cost allocation data, you can understand and optimize the cost and usage of your containerized applications and allocate application costs back to individual business entities based on how shared compute and memory resources are consumed.

Implementation steps

- **Allocate costs to workload metrics:** Using the defined metrics and configured tags, create a metric that combines the workload output and workload cost. Use analytics services such as Amazon Athena and Amazon Quick Suite to create an efficiency dashboard for the overall workload and any components.

Resources

Related documents:

- [Tagging AWS resources](#)
- [Analyzing your costs with AWS Budgets](#)
- [Analyzing your costs with Cost Explorer](#)
- [Managing AWS Cost and Usage Reports](#)

Related examples:

- [Improve cost visibility of Amazon ECS and AWS Batch with AWS Split Cost Allocation Data](#)

COST 4. How do you decommission resources?

Implement change control and resource management from project inception to end-of-life. This ensures you shut down or terminates unused resources to reduce waste.

Best practices

- [COST04-BP01 Track resources over their lifetime](#)
- [COST04-BP02 Implement a decommissioning process](#)
- [COST04-BP03 Decommission resources](#)

- [COST04-BP04 Decommission resources automatically](#)
- [COST04-BP05 Enforce data retention policies](#)

COST04-BP01 Track resources over their lifetime

Define and implement a method to track resources and their associations with systems over their lifetime. You can use tagging to identify the workload or function of the resource.

Level of risk exposed if this best practice is not established: High

Implementation guidance

Decommission workload resources that are no longer required. A common example is resources used for testing: after testing has been completed, the resources can be removed. Tracking resources with tags (and running reports on those tags) can help you identify assets for decommission, as they will not be in use or the license on them will expire. Using tags is an effective way to track resources, by labeling the resource with its function, or a known date when it can be decommissioned. Reporting can then be run on these tags. Example values for feature tagging are feature-X testing to identify the purpose of the resource in terms of the workload lifecycle. Another example is using LifeSpan or TTL for the resources, such as to-be-deleted tag key name and value to define the time period or specific time for decommissioning.

Implementation steps

- **Implement a tagging scheme:** Implement a tagging scheme that identifies the workload the resource belongs to, verifying that all resources within the workload are tagged accordingly. Tagging helps you categorize resources by purpose, team, environment, or other criteria relevant to your business. For more detail on tagging uses cases, strategies, and techniques, see [AWS Tagging Best Practices](#).
- **Implement workload throughput or output monitoring:** Implement workload throughput monitoring or alarming, initiating on either input requests or output completions. Configure it to provide notifications when workload requests or outputs drop to zero, indicating the workload resources are no longer used. Incorporate a time factor if the workload periodically drops to zero under normal conditions. For more detail on unused or underutilized resources, see [AWS Trusted Advisor Cost Optimization checks](#).
- **Group AWS resources:** Create groups for AWS resources. You can use [AWS Resource Groups](#) to organize and manage your AWS resources that are in the same AWS Region. You can add tags to

most of your resources to help identify and sort your resources within your organization. Use [Tag Editor](#) add tags to supported resources in bulk. Consider using [AWS Service Catalog](#) to create, manage, and distribute portfolios of approved products to end users and manage the product lifecycle.

Resources

Related documents:

- [AWS Auto Scaling](#)
- [AWS Trusted Advisor](#)
- [AWS Trusted Advisor Cost Optimization Checks](#)
- [Tagging AWS resources](#)
- [Publishing Custom Metrics](#)

Related videos:

- [How to optimize costs using AWS Trusted Advisor](#)

Related examples:

- [Organize AWS resources](#)
- [Optimize cost using AWS Trusted Advisor](#)

COST04-BP02 Implement a decommissioning process

Implement a process to identify and decommission unused resources.

Level of risk exposed if this best practice is not established: High

Implementation guidance

Implement a standardized process across your organization to identify and remove unused resources. The process should define the frequency searches are performed and the processes to remove the resource to verify that all organization requirements are met.

Implementation steps

- **Create and implement a decommissioning process:** Work with the workload developers and owners to build a decommissioning process for the workload and its resources. The process should cover the method to verify if the workload is in use, and also if each of the workload resources are in use. Detail the steps necessary to decommission the resource, removing them from service while ensuring compliance with any regulatory requirements. Any associated resources should be included, such as licenses or attached storage. Notify the workload owners that the decommissioning process has been started.

Use the following decommission steps to guide you on what should be checked as part of your process:

- **Identify resources to be decommissioned:** Identify resources that are eligible for decommissioning in your AWS Cloud. Record all necessary information and schedule the decommission. In your timeline, be sure to account for if (and when) unexpected issues arise during the process.
- **Coordinate and communicate:** Work with workload owners to confirm the resource to be decommissioned
- **Record metadata and create backups:** Record metadata (such as public IPs, Region, AZ, VPC, Subnet, and Security Groups) and create backups (such as Amazon Elastic Block Store snapshots or taking AMI, keys export, and Certificate export) if it is required for the resources in the production environment or if they are critical resources.
- **Validate infrastructure-as-code:** Determine whether resources were deployed with AWS CloudFormation, Terraform, AWS Cloud Development Kit (AWS CDK), or any other infrastructure-as-code deployment tool so they can be re-deployed if necessary.
- **Prevent access:** Apply restrictive controls for a period of time, to prevent the use of resources while you determine if the resource is required. Verify that the resource environment can be reverted to its original state if required.
- **Follow your internal decommissioning process:** Follow the administrative tasks and decommissioning process of your organization, like removing the resource from your organization domain, removing the DNS record, and removing the resource from your configuration management tool, monitoring tool, automation tool and security tools.

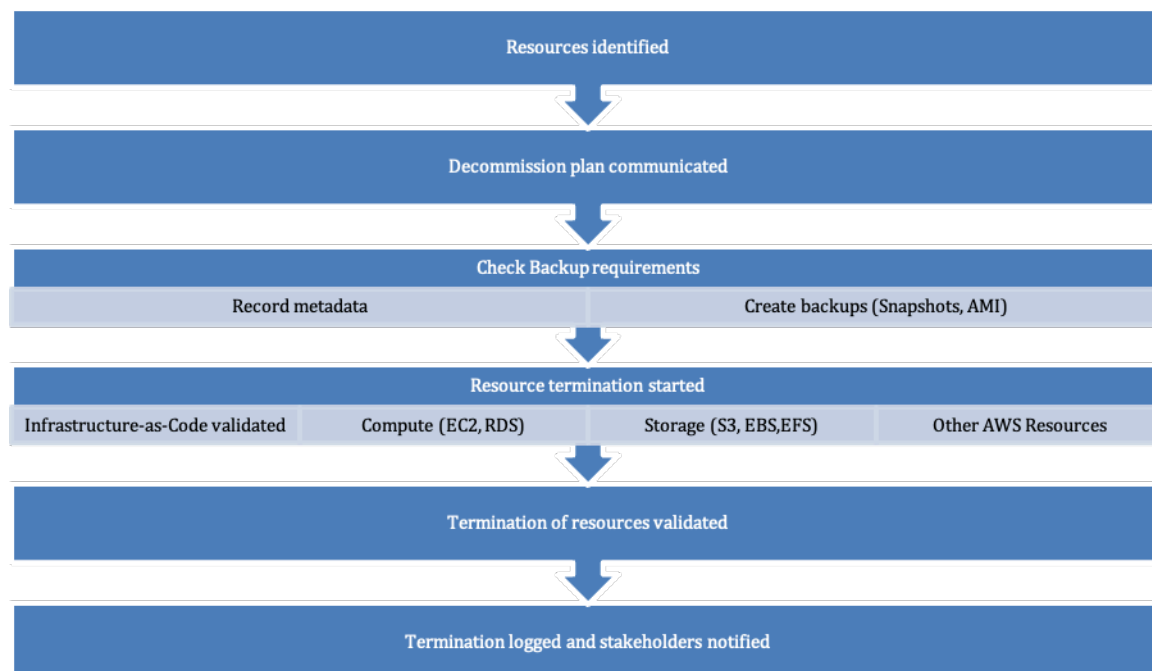
If the resource is an Amazon EC2 instance, consult the following list. [For more detail, see How do I delete or terminate my Amazon EC2 resources?](#)

- Stop or terminate all your Amazon EC2 instances and load balancers. Amazon EC2 instances are visible in the console for a short time after they're terminated. You aren't billed for any instances that aren't in the running state

- Delete your Auto Scaling infrastructure.
- Release all Dedicated Hosts.
- Delete all Amazon EBS volumes and Amazon EBS snapshots.
- Release all Elastic IP addresses.
- Deregister all Amazon Machine Images (AMIs).
- Terminate all AWS Elastic Beanstalk environments.

If the resource is an object in Amazon Glacier storage and if you delete an archive before meeting the minimum storage duration, you will be charged a prorated early deletion fee. Amazon Glacier minimum storage duration depends on the storage class used. For a summary of minimum storage duration for each storage class, see [Performance across the Amazon S3 storage classes](#). For detail on how early deletion fees are calculated, see [Amazon S3 pricing](#).

The following simple decommissioning process flowchart outlines the decommissioning steps. Before decommissioning resources, verify that resources you have identified for decommissioning are not being used by the organization.



Resource decommissioning flow.

Resources

Related documents:

- [AWS Auto Scaling](#)
- [AWS Trusted Advisor](#)
- [AWS CloudTrail](#)

Related videos:

- [Delete CloudFormation stack but retain some resources](#)
- [Find out which user launched Amazon EC2 instance](#)

Related examples:

- [Delete or terminate Amazon EC2 resources](#)
- [Find out which user launched an Amazon EC2 instance](#)

COST04-BP03 Decommission resources

Decommission resources initiated by events such as periodic audits, or changes in usage. Decommissioning is typically performed periodically and can be manual or automated.

Level of risk exposed if this best practice is not established: Medium

Implementation guidance

The frequency and effort to search for unused resources should reflect the potential savings, so an account with a small cost should be analyzed less frequently than an account with larger costs. Searches and decommission events can be initiated by state changes in the workload, such as a product going end of life or being replaced. Searches and decommission events may also be initiated by external events, such as changes in market conditions or product termination.

Implementation steps

- **Decommission resources:** This is the depreciation stage of AWS resources that are no longer needed or ending of a licensing agreement. Complete all final checks completed before moving to the disposal stage and decommissioning resources to prevent any unwanted disruptions like taking snapshots or backups. Using the decommissioning process, decommission each of the resources that have been identified as unused.

Resources

Related documents:

- [AWS Auto Scaling](#)
- [AWS Trusted Advisor](#)

COST04-BP04 Decommission resources automatically

Design your workload to gracefully handle resource termination as you identify and decommission non-critical resources, resources that are not required, or resources with low utilization.

Level of risk exposed if this best practice is not established: Low

Implementation guidance

Use automation to reduce or remove the associated costs of the decommissioning process. Designing your workload to perform automated decommissioning will reduce the overall workload costs during its lifetime. You can use [Amazon EC2 Auto Scaling](#) or [Application Auto Scaling](#) to perform the decommissioning process. You can also implement custom code using the [API or SDK](#) to decommission workload resources automatically.

[Modern applications](#) are built serverless-first, a strategy that prioritizes the adoption of serverless services. AWS developed [serverless services](#) for all three layers of your stack: compute, integration, and data stores. Using serverless architecture will allow you to save costs during low-traffic periods with scaling up and down automatically.

Implementation steps

- **Implement Amazon EC2 Auto Scaling or Application Auto Scaling:** For resources that are supported, configure them with Amazon EC2 Auto Scaling or Application Auto Scaling. These services can help you optimize your utilization and cost efficiencies when consuming AWS services. When demand drops, these services will automatically remove any excess resource capacity so you avoid overspending.
- **Configure CloudWatch to terminate instances:** Instances can be configured to terminate using [CloudWatch alarms](#). Using the metrics from the decommissioning process, implement an alarm with an Amazon Elastic Compute Cloud action. Verify the operation in a non-production environment before rolling out.

- **Implement code within the workload:** You can use the AWS SDK or AWS CLI to decommission workload resources. Implement code within the application that integrates with AWS and terminates or removes resources that are no longer used.
- **Use serverless services:** Prioritize building [serverless architectures](#) and [event-driven architecture](#) on AWS to build and run your applications. AWS offers multiple serverless technology services that inherently provide automatically optimized resource utilization and automated decommissioning (scale in and scale out). With serverless applications, resource utilization is automatically optimized and you never pay for over-provisioning.

Resources

Related documents:

- [Amazon EC2 Auto Scaling](#)
- [Getting Started with Amazon EC2 Auto Scaling](#)
- [Application Auto Scaling](#)
- [AWS Trusted Advisor](#)
- [Serverless on AWS](#)
- [Create Alarms to Stop, Terminate, Reboot, or Recover an Instance](#)
- [Adding terminate actions to Amazon CloudWatch alarms](#)

Related examples:

- [Scheduling automatic deletion of AWS CloudFormation stacks](#)

COST04-BP05 Enforce data retention policies

Define data retention policies on supported resources to handle object deletion per your organizations' requirements. Identify and delete unnecessary or orphaned resources and objects that are no longer required.

Level of risk exposed if this best practice is not established: Medium

Use data retention policies and lifecycle policies to reduce the associated costs of the decommissioning process and storage costs for the identified resources. Defining your data retention policies and lifecycle policies to perform automated storage class migration and deletion

will reduce the overall storage costs during its lifetime. You can use Amazon Data Lifecycle Manager to automate the creation and deletion of Amazon Elastic Block Store snapshots and Amazon EBS-backed Amazon Machine Images (AMIs), and use Amazon S3 Intelligent-Tiering or an Amazon S3 lifecycle configuration to manage the lifecycle of your Amazon S3 objects. You can also implement custom code using the [API or SDK](#) to create lifecycle policies and policy rules for objects to be deleted automatically.

Implementation steps

- **Use Amazon Data Lifecycle Manager:** Use lifecycle policies on Amazon Data Lifecycle Manager to automate deletion of Amazon EBS snapshots and Amazon EBS-backed AMIs.
- **Set up lifecycle configuration on a bucket:** Use Amazon S3 lifecycle configuration on a bucket to define actions for Amazon S3 to take during an object's lifecycle, as well as deletion at the end of the object's lifecycle, based on your business requirements.

Resources

Related documents:

- [AWS Trusted Advisor](#)
- [Amazon Data Lifecycle Manager](#)
- [How to set lifecycle configuration on Amazon S3 bucket](#)

Related videos:

- [Automate Amazon EBS Snapshots with Amazon Data Lifecycle Manager](#)
- [Empty an Amazon S3 bucket using a lifecycle configuration rule](#)

Related examples:

- [Empty an Amazon S3 bucket using a lifecycle configuration rule](#)

Cost-effective resources

Questions

- [COST 5. How do you evaluate cost when you select services?](#)

- [COST 6. How do you meet cost targets when you select resource type, size and number?](#)
- [COST 7. How do you use pricing models to reduce cost?](#)
- [COST 8. How do you plan for data transfer charges?](#)

COST 5. How do you evaluate cost when you select services?

Amazon EC2, Amazon EBS, and Amazon S3 are building-block AWS services. Managed services, such as Amazon RDS and Amazon DynamoDB, are higher level, or application level, AWS services. By selecting the appropriate building blocks and managed services, you can optimize this workload for cost. For example, using managed services, you can reduce or remove much of your administrative and operational overhead, freeing you to work on applications and business-related activities.

Best practices

- [COST05-BP01 Identify organization requirements for cost](#)
- [COST05-BP02 Analyze all components of the workload](#)
- [COST05-BP03 Perform a thorough analysis of each component](#)
- [COST05-BP04 Select software with cost-effective licensing](#)
- [COST05-BP05 Select components of this workload to optimize cost in line with organization priorities](#)
- [COST05-BP06 Perform cost analysis for different usage over time](#)

COST05-BP01 Identify organization requirements for cost

Work with team members to define the balance between cost optimization and other pillars, such as performance and reliability, for this workload.

Level of risk exposed if this best practice is not established: High

Implementation guidance

In most organizations, the information technology (IT) department is comprised of multiple small teams, each with its own agenda and focus area, that reflects the specialises and skills of its team members. You need to understand your organization's overall objectives, priorities, goals and how each department or project contributes to these objectives. Categorizing all essential resources, including personnel, equipment, technology, materials, and external services, is crucial

for achieving organizational objectives and comprehensive budget planning. Adopting this systematic approach to cost identification and understanding is fundamental for establishing a realistic and robust cost plan for the organization.

When selecting services for your workload, it is key that you understand your organization priorities. Create a balance between cost optimization and other AWS Well-Architected Framework pillars, such as performance and reliability. This process should be conducted systematically and regularly to reflect changes in the organization's objectives, market conditions, and operational dynamics. A fully cost-optimized workload is the solution that is most aligned to your organization's requirements, not necessarily the lowest cost. Meet with all teams in your organization, such as product, business, technical, and finance to collect information. Evaluate the impact of tradeoffs between competing interests or alternative approaches to help make informed decisions when determining where to focus efforts or choosing a course of action.

For example, accelerating speed to market for new features may be emphasized over cost optimization, or you may choose a relational database for non-relational data to simplify the effort to migrate a system, rather than migrating to a database optimized for your data type and updating your application.

Implementation steps

- **Identify organization requirements for cost:** Meet with team members from your organization, including those in product management, application owners, development and operational teams, management, and financial roles. Prioritize the Well-Architected pillars for this workload and its components. The output should be a list of the pillars in order. You can also add a weight to each pillar to indicate how much additional focus it has, or how similar the focus is between two pillars.
- **Address the technical debt and document it:** During the workload review, address the technical debt. Document a backlog item to revisit the workload in the future, with the goal of refactoring or re-architecting to optimize it further. It's essential to clearly communicate the trade-offs that were made to other stakeholders.

Resources

Related best practices:

- [REL11-BP07 Architect your product to meet availability targets and uptime service level agreements \(SLAs\)](#)

- [OPS01-BP06 Evaluate tradeoffs](#)

Related documents:

- [AWS Total Cost of Ownership \(TCO\) Calculator](#)
- [Amazon S3 storage classes](#)
- [Cloud products](#)

COST05-BP02 Analyze all components of the workload

Verify every workload component is analyzed, regardless of current size or current costs. The review effort should reflect the potential benefit, such as current and projected costs.

Level of risk exposed if this best practice is not established: High

Implementation guidance

Workload components, which are designed to deliver business value to the organization, may encompass various services. For each component, one might choose specific AWS Cloud services to address business needs. This selection could be influenced by factors such as familiarity with or prior experience using these services.

After identifying your organization's requirements as mentioned in [COST05-BP01 Identify organization requirements for cost](#), perform a thorough analysis on all components in your workload. Analyze each component considering current and projected costs and sizes. Consider the cost of analysis against any potential workload savings over its lifecycle. The effort expended on the analysis of all components of this workload should correspond to the potential savings or improvements anticipated from optimization of that specific component. For example, if the cost of the proposed resource is \$10 per month, and under forecasted loads would not exceed \$15 per month, spending a day of effort to reduce costs by 50% (five dollars per month) could exceed the potential benefit over the life of the system. Use a faster and more efficient data-based estimation to create the best overall outcome for this component.

Workloads can change over time, and the right set of services may not be optimal if the workload architecture or usage changes. Analysis for selection of services must incorporate current and future workload states and usage levels. Implementing a service for future workload state or usage may reduce overall costs by reducing or removing the effort required to make future changes. For example, using EMR Serverless might be the appropriate choice initially. However, as consumption

for that service increases, transitioning to EMR on EC2 could reduce costs for that component of the workload.

[AWS Cost Explorer](#) and the AWS Cost and Usage Reports ([CUR](#)) can analyze the cost of a proof of concept (PoC) or running environment. You can also use [AWS Pricing Calculator](#) to estimate workload costs.

Write a workflow to be followed by technical teams to review their workloads. Keep this workflow simple, but also cover all the necessary steps to make sure the teams understand each component of the workload and its pricing. Your organization can then follow and customize this workflow based on the specific needs of each team.

1. **List each service in use for your workload:** This is a good starting point. Identify all of the services currently in use and where costs are originate from.
2. **Understand how pricing works for those services:** Understand the [pricing model](#) of each service. Different AWS services have different pricing models based on factors like usage volume, data transfer, and feature-specific pricing.
3. **Focus on the services that have unexpected workload costs and that do not align with your expected usage and business outcome:** Identify outliers or services where the cost is not proportional to the value or usage by using AWS Cost Explorer or AWS Cost and Usage Reports. It's important to correlate costs with business outcomes to prioritize optimization efforts.
4. **AWS Cost Explorer, CloudWatch Logs, VPC Flow Logs, and Amazon S3 Storage Lens to understand the root cause of those high costs:** These tools are instrumental in the diagnosis of high costs. Each service offers a different lens to view and analyze usage and costs. For instance, Cost Explorer helps determine overall cost trends, CloudWatch Logs provides operational insights, VPC Flow Logs displays IP traffic, and Amazon S3 Storage Lens is useful for storage analytics.
5. **Use AWS Budgets to set budgets for certain amounts for services or accounts:** Setting budgets is a proactive way to manage costs. Use AWS Budgets to set custom budget thresholds and receive alerts when costs exceed those thresholds.
6. **Configure Amazon CloudWatch alarms to send billing and usage alerts:** Set up monitoring and alerts for cost and usage metrics. CloudWatch alarms can notify you when certain thresholds are breached, which improves intervention response time.

Facilitate notable enhancement and financial savings over time through strategic review of all workload components and irrespective of their present attributes. The effort invested in this review

process should be deliberate, with careful consideration of the potential advantages that might be realized.

Implementation steps

- **List the workload components:** Build a list of your workload's components. Use this list to verify that each component was analyzed. The effort spent should reflect the criticality to the workload as defined by your organization's priorities. Group together resources functionally to improve efficiency (for example, production database storage, if there are multiple databases).
- **Prioritize the component list:** Take the component list and prioritize it in order of effort. This is typically in order of the cost of the component, from most expensive to least expensive or the criticality as defined by your organization's priorities.
- **Perform the analysis:** For each component on the list, review the options and services available, and choose the option that aligns best with your organizational priorities.

Resources

Related documents:

- [AWS Pricing Calculator](#)
- [AWS Cost Explorer](#)
- [Amazon S3 storage classes](#)
- [AWS Cloud products](#)

Related videos:

- [AWS Cost Optimization Series: CloudWatch](#)

COST05-BP03 Perform a thorough analysis of each component

Look at overall cost to the organization of each component. Calculate the total cost of ownership by factoring in cost of operations and management, especially when using managed services by cloud provider. The review effort should reflect potential benefit (for example, time spent analyzing is proportional to component cost).

Level of risk exposed if this best practice is not established: High

Implementation guidance

Consider the time savings that will allow your team to focus on retiring technical debt, innovation, value-adding features and building what differentiates the business. For example, you might need to lift and shift (also known as rehost) your databases from your on-premises environment to the cloud as rapidly as possible and optimize later. It is worth exploring the possible savings attained by using managed services on AWS that may remove or reduce license costs. Managed services on AWS remove the operational and administrative burden of maintaining a service, such as patching or upgrading the OS, and allow you to focus on innovation and business.

Since managed services operate at cloud scale, they can offer a lower cost per transaction or service. You can make potential optimizations in order to achieve some tangible benefit, without changing the core architecture of the application. For example, you may be looking to reduce the amount of time you spend managing database instances by migrating to a database-as-a-service platform like [Amazon Relational Database Service \(Amazon RDS\)](#) or migrating your application to a fully managed platform like [AWS Elastic Beanstalk](#).

Usually, managed services have attributes that you can set to ensure sufficient capacity. You must set and monitor these attributes so that your excess capacity is kept to a minimum and performance is maximized. You can modify the attributes of AWS Managed Services using the AWS Management Console or AWS APIs and SDKs to align resource needs with changing demand. For example, you can increase or decrease the number of nodes on an Amazon EMR cluster (or an Amazon Redshift cluster) to scale out or in.

You can also pack multiple instances on an AWS resource to activate higher density usage. For example, you can provision multiple small databases on a single Amazon Relational Database Service (Amazon RDS) database instance. As usage grows, you can migrate one of the databases to a dedicated Amazon RDS database instance using a snapshot and restore process.

When provisioning workloads on managed services, you must understand the requirements of adjusting the service capacity. These requirements are typically time, effort, and any impact to normal workload operation. The provisioned resource must allow time for any changes to occur, provision the required overhead to allow this. The ongoing effort required to modify services can be reduced to virtually zero by using APIs and SDKs that are integrated with system and monitoring tools, such as Amazon CloudWatch.

[Amazon RDS](#), [Amazon Redshift](#), and [Amazon ElastiCache](#) provide a managed database service. [Amazon Athena](#), [Amazon EMR](#), and [Amazon OpenSearch Service](#) provide a managed analytics service.

[AMS](#) is a service that operates AWS infrastructure on behalf of enterprise customers and partners. It provides a secure and compliant environment that you can deploy your workloads onto. AMS uses enterprise cloud operating models with automation to allow you to meet your organization requirements, move into the cloud faster, and reduce your on-going management costs.

Implementation steps

- **Perform a thorough analysis:** Using the component list, work through each component from the highest priority to the lowest priority. For the higher priority and more costly components, perform additional analysis and assess all available options and their long term impact. For lower priority components, assess if changes in usage would change the priority of the component, and then perform an analysis of appropriate effort.
- **Compare managed and unmanaged resources:** Consider the operational cost for the resources you manage and compare them with AWS managed resources. For example, review your databases running on Amazon EC2 instances and compare with Amazon RDS options (an AWS managed service) or Amazon EMR compared to running Apache Spark on Amazon EC2. When moving from a self-managed workload to a AWS fully managed workload, research your options carefully. The three most important factors to consider are the [type of managed service](#) you want to use, the process you will use to [migrate your data](#) and understand the [AWS shared responsibility model](#).

Resources

Related documents:

- [AWS Total Cost of Ownership \(TCO\) Calculator](#)
- [Amazon S3 storage classes](#)
- [AWS Cloud products](#)
- [AWS Shared Responsibility Model](#)

Related videos:

- [Why move to a managed database?](#)
- [What is Amazon EMR and how can I use it for processing data?](#)

Related examples:

- [Why to move to a managed database](#)
- [Consolidate data from identical SQL Server databases into a single Amazon RDS for SQL Server database using AWS DMS](#)
- [Deliver data at scale to Amazon Managed Streaming for Apache Kafka \(Amazon MSK\)](#)
- [Migrate an ASP.NET web application to AWS Elastic Beanstalk](#)

COST05-BP04 Select software with cost-effective licensing

Open-source software eliminates software licensing costs, which can contribute significant costs to workloads. Where licensed software is required, avoid licenses bound to arbitrary attributes such as CPUs, look for licenses that are bound to output or outcomes. The cost of these licenses scales more closely to the benefit they provide.

Level of risk exposed if this best practice is not established: Low

Implementation guidance

Open source originated in the context of software development to indicate that the software complies with certain free distribution criteria. Open source software is composed of source code that anyone can inspect, modify, and enhance. Based on business requirements, skill of engineers, forecasted usage, or other technology dependencies, organizations can consider using open source software on AWS to minimize their license costs. In other words, the cost of software licenses can be reduced through the use of [open source software](#). This can have significant impact on workload costs as the size of the workload scales.

Measure the benefits of licensed software against the total cost to optimize your workload. Model any changes in licensing and how they would impact your workload costs. If a vendor changes the cost of your database license, investigate how that impacts the overall efficiency of your workload. Consider historical pricing announcements from your vendors for trends of licensing changes across their products. Licensing costs may also scale independently of throughput or usage, such as licenses that scale by hardware (CPU bound licenses). These licenses should be avoided because costs can rapidly increase without corresponding outcomes.

For instance, operating an Amazon EC2 instance in us-east-1 with a Linux operating system allows you to cut costs by approximately 45%, compared to running another Amazon EC2 instance that runs on Windows.

The [AWS Pricing Calculator](#) offers a comprehensive way to compare the costs of various resources with different license options, such as Amazon RDS instances and different database engines.

Additionally, the AWS Cost Explorer provides an invaluable perspective for the costs of existing workloads, especially those that come with different licenses. For license management, [AWS License Manager](#) offers a streamlined method to oversee and handle software licenses. Customers can deploy and operationalize their preferred open source software in the AWS Cloud.

Implementation steps

- **Analyze license options:** Review the licensing terms of available software. Look for open source versions that have the required functionality, and whether the benefits of licensed software outweigh the cost. Favorable terms align the cost of the software to the benefits it provides.
- **Analyze the software provider:** Review any historical pricing or licensing changes from the vendor. Look for any changes that do not align to outcomes, such as punitive terms for running on specific vendors hardware or platforms. Additionally, look for how they perform audits, and penalties that could be imposed.

Resources

Related documents:

- [Open Source at AWS](#)
- [AWS Total Cost of Ownership \(TCO\) Calculator](#)
- [Amazon S3 storage classes](#)
- [Cloud products](#)

Related examples:

- [Open Source Blogs](#)
- [AWS Open Source Blogs](#)
- [Optimization and Licensing Assessment](#)

COST05-BP05 Select components of this workload to optimize cost in line with organization priorities

Factor in cost when selecting all components for your workload. This includes using application-level and managed services or serverless, containers, or event-driven architecture to reduce overall cost. Minimize license costs by using open-source software, software that does not have license fees, or alternatives to reduce spending.

Level of risk exposed if this best practice is not established: Medium

Implementation guidance

Consider the cost of services and options when selecting all components. This includes using application level and managed services, such as [Amazon Relational Database Service](#) (Amazon RDS), [Amazon DynamoDB](#), [Amazon Simple Notification Service](#) (Amazon SNS), and [Amazon Simple Email Service](#) (Amazon SES) to reduce overall organization cost.

Use serverless and containers for compute, such as [AWS Lambda](#) and [Amazon Simple Storage Service](#) (Amazon S3) for static websites. Containerize your application if possible and use AWS Managed Container Services such as [Amazon Elastic Container Service](#) (Amazon ECS) or [Amazon Elastic Kubernetes Service](#) (Amazon EKS).

Minimize license costs by using open-source software, or software that does not have license fees (for example, Amazon Linux for compute workloads or migrate databases to Amazon Aurora).

You can use serverless or application-level services such as [Lambda](#), [Amazon Simple Queue Service](#) (Amazon SQS), [Amazon SNS](#), and [Amazon SES](#). These services remove the need for you to manage a resource and provide the function of code execution, queuing services, and message delivery. The other benefit is that they scale in performance and cost in line with usage, allowing efficient cost allocation and attribution.

Using [event-driven architecture](#) is also possible with serverless services. Event-driven architectures are push-based, so everything happens on demand as the event presents itself in the router. This way, you're not paying for continuous polling to check for an event. This means less network bandwidth consumption, less CPU utilization, less idle fleet capacity, and fewer SSL/TLS handshakes.

For more information on serverless, see [Well-Architected Serverless Application lens whitepaper](#).

Implementation steps

- **Select each service to optimize cost:** Using your prioritized list and analysis, select each option that provides the best match with your organizational priorities. Instead of increasing the capacity to meet the demand, consider other options which may give you better performance with lower cost. For example, if you need to review expected traffic for your databases on AWS, consider either increasing the instance size or using Amazon ElastiCache services (Redis or Memcached) to provide cached mechanisms for your databases.

- **Evaluate event-driven architecture:** Using serverless architecture also allows you to build event-driven architecture for distributed microservice-based applications, which helps you build scalable, resilient, agile and cost-effective solutions.

Resources

Related documents:

- [AWS Total Cost of Ownership \(TCO\) Calculator](#)
- [AWS Serverless](#)
- [What is Event-Driven Architecture](#)
- [Amazon S3 storage classes](#)
- [Cloud products](#)
- [Amazon ElastiCache \(Redis OSS\)](#)

Related examples:

- [Getting started with event-driven architecture](#)
- [Event-driven architecture](#)
- [How Statsig runs 100x more cost-effectively using Amazon ElastiCache \(Redis OSS\)](#)
- [Best practices for working with AWS Lambda functions](#)

COST05-BP06 Perform cost analysis for different usage over time

Workloads can change over time. Some services or features are more cost effective at different usage levels. By performing the analysis on each component over time and at projected usage, the workload remains cost-effective over its lifetime.

Level of risk exposed if this best practice is not established: Medium

Implementation guidance

As AWS releases new services and features, the optimal services for your workload may change. Effort required should reflect potential benefits. Workload review frequency depends on your organization requirements. If it is a workload of significant cost, implementing new services sooner will maximize cost savings, so more frequent review can be advantageous. Another initiation for

review is change in usage patterns. Significant changes in usage can indicate that alternate services would be more optimal.

If you need to move data into AWS Cloud, you can select any wide variety of services AWS offers and partner tools to help you migrate your data sets, whether they are files, databases, machine images, block volumes, or even tape backups. For example, to move a large amount of data to and from AWS or process data at the edge, you can use one of the AWS purpose-built devices to cost effectively move petabytes of data offline. Another example is for higher data transfer rates, a direct connect service may be cheaper than a VPN which provides the required consistent connectivity for your business.

Based on the cost analysis for different usage over time, review your scaling activity. Analyze the result to see if the scaling policy can be tuned to add instances with multiple instance types and purchase options. Review your settings to see if the minimum can be reduced to serve user requests but with a smaller fleet size, and add more resources to meet the expected high demand.

Perform cost analysis for different usage over time by discussing with stakeholders in your organization and use [AWS Cost Explorer's](#) forecast feature to predict the potential impact of service changes. Monitor usage level launches using AWS Budgets, CloudWatch billing alarms and AWS Cost Anomaly Detection to identify and implement the most cost-effective services sooner.

Implementation steps

- **Define predicted usage patterns:** Working with your organization, such as marketing and product owners, document what the expected and predicted usage patterns will be for the workload. Discuss with business stakeholders about both historical and forecasted cost and usage increases and make sure increases align with business requirements. Identify calendar days, weeks, or months where you expect more users to use your AWS resources, which indicate that you should increase the capacity of the existing resources or adopt additional services to reduce the cost and increase performance.
- **Perform cost analysis at predicted usage:** Using the usage patterns defined, perform analysis at each of these points. The analysis effort should reflect the potential outcome. For example, if the change in usage is large, a thorough analysis should be performed to verify any costs and changes. In other words, when cost increases, usage should increase for business as well.

Resources

Related documents:

- [AWS Total Cost of Ownership \(TCO\) Calculator](#)
- [Amazon S3 storage classes](#)
- [Cloud products](#)
- [Amazon EC2 Auto Scaling](#)
- [Cloud Data Migration](#)
- [AWS Snow Family](#)

Related videos:

- [AWS OpsHub for Snow Family](#)

COST 6. How do you meet cost targets when you select resource type, size and number?

Verify that you choose the appropriate resource size and number of resources for the task at hand. You minimize waste by selecting the most cost effective type, size, and number.

Best practices

- [COST06-BP01 Perform cost modeling](#)
- [COST06-BP02 Select resource type, size, and number based on data](#)
- [COST06-BP03 Select resource type, size, and number automatically based on metrics](#)
- [COST06-BP04 Consider using shared resources](#)

COST06-BP01 Perform cost modeling

Identify organization requirements (such as business needs and existing commitments) and perform cost modeling (overall costs) of the workload and each of its components. Perform benchmark activities for the workload under different predicted loads and compare the costs. The modeling effort should reflect the potential benefit. For example, time spent is proportional to component cost.

Level of risk exposed if this best practice is not established: High

Implementation guidance

Perform cost modeling for your workload and each of its components to understand the balance between resources, and find the correct size for each resource in the workload, given a specific level of performance. Understanding cost considerations can inform your organizational business case and decision-making process when evaluating the value realization outcomes for planned workload deployment.

Perform benchmark activities for the workload under different predicted loads and compare the costs. The modeling effort should reflect potential benefit; for example, time spent is proportional to component cost or predicted saving. For best practices, refer to the [Review section of the Performance Efficiency Pillar of the AWS Well-Architected Framework](#).

As an example, to create cost modeling for a workload consisting of compute resources, [AWS Compute Optimizer](#) can assist with cost modeling for running workloads. It provides right-sizing recommendations for compute resources based on historical usage. Make sure CloudWatch Agents are deployed to the Amazon EC2 instances to collect memory metrics which help you with more accurate recommendations within AWS Compute Optimizer. This is the ideal data source for compute resources because it is a free service that uses machine learning to make multiple recommendations depending on levels of risk.

There are [multiple services](#) you can use with custom logs as data sources for rightsizing operations for other services and workload components, such as [AWS Trusted Advisor](#), [Amazon CloudWatch](#) and [Amazon CloudWatch Logs](#). AWS Trusted Advisor checks resources and flags resources with low utilization which can help you rightsize your resources and create cost modeling.

The following are recommendations for cost modeling data and metrics:

- The monitoring must accurately reflect the user experience. Select the correct granularity for the time period and thoughtfully choose the maximum or 99th percentile instead of the average.
- Select the correct granularity for the time period of analysis that is required to cover any workload cycles. For example, if a two-week analysis is performed, you might be overlooking a monthly cycle of high utilization, which could lead to under-provisioning.
- Choose the right AWS services for your planned workload by considering your existing commitments, selected pricing models for other workloads, and ability to innovate faster and focus on your core business value.

Implementation steps

- **Perform cost modeling for resources:** Deploy the workload or a proof of concept into a separate account with the specific resource types and sizes to test. Run the workload with the test data and record the output results, along with the cost data for the time the test was run. Afterwards, redeploy the workload or change the resource types and sizes and run the test again. Include license fees of any products you may use with these resources and estimated operations (labor or engineer) costs for deploying and managing these resources while creating cost modeling. Consider cost modeling for a period (hourly, daily, monthly, yearly or three years).

Resources

Related documents:

- [AWS Auto Scaling](#)
- [Identifying Opportunities to Right Size](#)
- [Amazon CloudWatch features](#)
- [Cost Optimization: Amazon EC2 Right Sizing](#)
- [AWS Compute Optimizer](#)
- [AWS Pricing Calculator](#)

Related examples:

- [Perform a Data-Driven Cost modeling](#)
- [Estimate the cost of planned AWS resource configurations](#)
- [Choose the right AWS tools](#)

COST06-BP02 Select resource type, size, and number based on data

Select resource size or type based on data about the workload and resource characteristics. For example, compute, memory, throughput, or write intensive. This selection is typically made using a previous (on-premises) version of the workload, using documentation, or using other sources of information about the workload.

Level of risk exposed if this best practice is not established: Medium

Implementation guidance

Amazon EC2 provides a wide selection of instance types with different levels of CPU, memory, storage, and networking capacity to fit different use cases. These instance types feature different blends of CPU, memory, storage, and networking capabilities, giving you versatility when selecting the right resource combination for your projects. Every instance type comes in multiple sizes, so that you can adjust your resources based on your workload's demands. To determine which instance type you need, gather details about the system requirements of the application or software that you plan to run on your instance. These details should include the following:

- Operating system
- Number of CPU cores
- GPU cores
- Amount of system memory (RAM)
- Storage type and space
- Network bandwidth requirement

Identify the purpose of compute requirements and which instance is needed, and then explore the various Amazon EC2 instance families. Amazon offers the following instance type families:

- General Purpose
- Compute Optimized
- Memory Optimized
- Storage Optimized
- Accelerated Computing
- HPC Optimized

For a deeper understanding of the specific purposes and use cases that a particular Amazon EC2 instance family can fulfill, see [AWS Instance types](#).

System requirements gathering is critical for you to select the specific instance family and instance type that best serves your needs. Instance type names are comprised of the family name and the instance size. For example, the t2.micro instance is from the T2 family and is micro-sized.

Select resource size or type based on workload and resource characteristics (for example, compute, memory, throughput, or write intensive). This selection is typically made using cost modeling, a

previous version of the workload (such as an on-premises version), using documentation, or using other sources of information about the workload (whitepapers or published solutions). Using AWS pricing calculators or cost management tools can assist in making informed decisions about instance types, sizes, and configurations.

Implementation steps

- **Select resources based on data:** Use your cost modeling data to select the anticipated workload usage level, and choose the specified resource type and size. Relying on the cost modeling data, determine the number of virtual CPUs, total memory (GiB), the local instance store volume (GB), Amazon EBS volumes, and the network performance level, taking into account the data transfer rate required for the instance. Always make selections based on detailed analysis and accurate data to optimize performance while managing costs effectively.

Resources

Related documents:

- [AWS Instance types](#)
- [AWS Auto Scaling](#)
- [Amazon CloudWatch features](#)
- [Cost Optimization: EC2 Right Sizing](#)

Related videos:

- [Selecting the right Amazon EC2 instance for your workloads](#)
- [Right size your service](#)

Related examples:

- [It just got easier to discover and compare Amazon EC2 instance types](#)

COST06-BP03 Select resource type, size, and number automatically based on metrics

Use metrics from the currently running workload to select the right size and type to optimize for cost. Appropriately provision throughput, sizing, and storage for compute, storage, data, and

networking services. This can be done with a feedback loop such as automatic scaling or by custom code in the workload.

Level of risk exposed if this best practice is not established: Low

Implementation guidance

Create a feedback loop within the workload that uses active metrics from the running workload to make changes to that workload. You can use a managed service, such as [AWS Auto Scaling](#), which you configure to perform the right sizing operations for you. AWS also provides [APIs, SDKs](#), and features that allow resources to be modified with minimal effort. You can program a workload to stop-and-start an Amazon EC2 instance to allow a change of instance size or instance type. This provides the benefits of right-sizing while removing almost all the operational cost required to make the change.

Some AWS services have built in automatic type or size selection, such as [Amazon Simple Storage Service Intelligent-Tiering](#). Amazon S3 Intelligent-Tiering automatically moves your data between two access tiers, frequent access and infrequent access, based on your usage patterns.

Implementation steps

- **Increase your observability by configuring workload metrics:** Capture key metrics for the workload. These metrics provide an indication of the customer experience, such as workload output, and align to the differences between resource types and sizes, such as CPU and memory usage. For compute resource, analyze performance data to right size your Amazon EC2 instances. Identify idle instances and ones that are underutilized. Key metrics to look for are CPU usage and memory utilization (for example, 40% CPU utilization at 90% of the time as explained in [Rightsizing with AWS Compute Optimizer and Memory Utilization Enabled](#)). Identify instances with a maximum CPU usage and memory utilization of less than 40% over a four-week period. These are the instances to right size to reduce costs. For storage resources such as Amazon S3, you can use [Amazon S3 Storage Lens](#), which allows you to see 28 metrics across various categories at the bucket level, and 14 days of historical data in the dashboard by default. You can filter your Amazon S3 Storage Lens dashboard by summary and cost optimization or events to analyze specific metrics.
- **View rightsizing recommendations:** Use the rightsizing recommendations in AWS Compute Optimizer and the Amazon EC2 rightsizing tool in the Cost Management console, or review AWS Trusted Advisor right-sizing your resources to make adjustments on your workload. It is important to use the [right tools](#) when right-sizing different resources and follow [right-sizing](#)

[guidelines](#) whether it is an Amazon EC2 instance, AWS storage classes, or Amazon RDS instance types. For storage resources, you can use Amazon S3 Storage Lens, which gives you visibility into object storage usage, activity trends, and makes actionable recommendations to optimize costs and apply data protection best practices. Using the contextual recommendations that [Amazon S3 Storage Lens](#) derives from analysis of metrics across your organization, you can take immediate steps to optimize your storage.

- **Select resource type and size automatically based on metrics:** Using the workload metrics, manually or automatically select your workload resources. For compute resources, configuring AWS Auto Scaling or implementing code within your application can reduce the effort required if frequent changes are needed, and it can potentially implement changes sooner than a manual process. You can launch and automatically scale a fleet of On-Demand Instances and Spot Instances within a single Auto Scaling group. In addition to receiving discounts for using Spot Instances, you can use Reserved Instances or a Savings Plan to receive discounted rates of the regular On-Demand Instance pricing. All of these factors combined help you optimize your cost savings for Amazon EC2 instances and determine the desired scale and performance for your application. You can also use an [attribute-based instance type selection \(ABS\)](#) strategy in [Auto Scaling Groups \(ASG\)](#), which lets you express your instance requirements as a set of attributes, such as vCPU, memory, and storage. You can automatically use newer generation instance types when they are released and access a broader range of capacity with Amazon EC2 Spot Instances. Amazon EC2 Fleet and Amazon EC2 Auto Scaling select and launch instances that fit the specified attributes, removing the need to manually pick instance types. For storage resources, you can use the [Amazon S3 Intelligent Tiering](#) and [Amazon EFS Infrequent Access](#) features, which allow you to select storage classes automatically that deliver automatic storage cost savings when data access patterns change, without performance impact or operational overhead.

Resources

Related documents:

- [AWS Auto Scaling](#)
- [AWS Right-Sizing](#)
- [AWS Compute Optimizer](#)
- [Amazon CloudWatch features](#)
- [CloudWatch Getting Set Up](#)
- [CloudWatch Publishing Custom Metrics](#)

- [Getting Started with Amazon EC2 Auto Scaling](#)
- [Amazon S3 Storage Lens](#)
- [Amazon S3 Intelligent-Tiering](#)
- [Amazon EFS Infrequent Access](#)
- [Launch an Amazon EC2 Instance Using the SDK](#)

Related videos:

- [Right Size Your Services](#)

Related examples:

- [Attribute based Instance Type Selection for Auto Scaling for Amazon EC2 Fleet](#)
- [Optimizing Amazon Elastic Container Service for cost using scheduled scaling](#)
- [Predictive scaling with Amazon EC2 Auto Scaling](#)
- [Optimize Costs and Gain Visibility into Usage with Amazon S3 Storage Lens](#)

COST06-BP04 Consider using shared resources

For already-deployed services at the organization level for multiple business units, consider using shared resources to increase utilization and reduce total cost of ownership (TCO). Using shared resources can be a cost-effective option to centralize the management and costs by using existing solutions, sharing components, or both. Manage common functions like monitoring, backups, and connectivity either within an account boundary or in a dedicated account. You can also reduce cost by implementing standardization, reducing duplication, and reducing complexity.

Level of risk exposed if this best practice is not established: Medium

Implementation guidance

Where multiple workloads cause the same function, use existing solutions and shared components to improve management and optimize costs. Consider using existing resources (especially shared ones), such as non-production database servers or directory services, to mitigate cloud costs by following security best practices and organizational regulations. For optimal value realization and efficiency, it is crucial to allocate costs back (using showback and chargeback) to the pertinent areas of the business driving consumption.

Showback refers to reports that break down cloud costs into attributable categories, such as consumers, business units, general ledger accounts, or other responsible entities. The goal of showback is to show teams, business units, or individuals the cost of their consumed cloud resources.

Chargeback means to allocate central service spend to cost units based on a strategy suitable for a specific financial management process. For customers, chargeback charges the cost incurred from one shared services account to different financial cost categories suitable for a customer reporting process. By establishing chargeback mechanisms, you can report costs incurred by different business units, products, and teams.

Workloads can be categorized as critical and non-critical. Based on this classification, use shared resources with general configurations for less critical workloads. To further optimize costs, reserve dedicated servers solely for critical workloads. Share resources or provision them across several accounts to manage them efficiently. Even with distinct development, testing, and production environments, secure sharing is feasible and does not compromise organizational structure.

To improve your understanding and optimize cost and usage for containerized applications, use split cost allocation data which helps you allocate costs to individual business entities based on how the application consumes shared compute and memory resources. Split cost allocation data helps you achieve task-level showback and chargeback in container workloads running on Amazon Elastic Container Service (Amazon ECS) or Amazon Elastic Kubernetes Service (Amazon EKS).

For distributed architectures, build a shared services VPC, which provides centralized access to shared services required by workloads in each of the VPCs. These shared services can include resources such as directory services or VPC endpoints. To reduce administrative overhead and cost, share resources from a central location instead of building them in each VPC.

When you use shared resources, you can save on operational costs, maximize resource utilization, and improve consistency. In a multi-account design, you can host some AWS services centrally and access them using several applications and accounts in a hub to save cost. You can use [AWS Resource Access Manager \(AWS RAM\)](#) to share other common resources, such as [VPC subnets and AWS Transit Gateway attachments](#), [AWS Network Firewall](#), or [Amazon SageMaker AI pipelines](#). In a multi-account environment, use AWS RAM to create a resource once and share it with other accounts.

Organizations should tag shared costs effectively and verify that they do not have a significant portion of their costs untagged or unallocated. If you do not allocate shared costs effectively and no one takes accountability for shared costs management, shared cloud costs can spiral. You

should know where you have incurred costs at the resource, workload, team, or organization level, as this knowledge enhances your understanding of the value delivered at the applicable level when compared to the business outcomes achieved. Ultimately, organizations benefit from cost savings as a result of sharing cloud infrastructure. Encourage cost allocation on shared cloud resources to optimize cloud spend.

Implementation steps

- **Evaluate existing resources:** Review existing workloads that use similar services for your workload. Depending on the workload's components, consider existing platforms if business logic or technical requirement allow.
- **Use resource sharing in AWS RAM and restrict accordingly:** Use AWS RAM to share resources with other AWS accounts within your organization. When you share resources, you don't need to duplicate resources in multiple accounts, which minimizes the operational burden of resource maintenance. This process also helps you securely share the resources that you have created with roles and users in your account, as well as with other AWS accounts.
- **Tag resources:** Tag resources that are candidates for cost reporting and categorize them within cost categories. Activate these cost related resource tags for cost allocation to provide visibility of AWS resources usage. Focus on creating an appropriate level of granularity with respect to cost and usage visibility, and influence cloud consumption behaviors through cost allocation reporting and KPI tracking.

Resources

Related best practices:

- [SEC03-BP08 Share resources securely within your organization](#)

Related documents:

- [What is AWS Resource Access Manager?](#)
- [AWS services that you can use with AWS Organizations](#)
- [Shareable AWS resources](#)
- [AWS Cost and Usage \(CUR\) Queries](#)

Related videos:

- [AWS Resource Access Manager - granular access control with managed permissions](#)
- [How to design your AWS cost allocation strategy](#)
- [AWS Cost Categories](#)

Related examples:

- [How-to chargeback shared services: An AWS Transit Gateway example](#)
- [How to build a chargeback/showback model for Savings Plans using the CUR](#)
- [Using VPC Sharing for a Cost-Effective Multi-Account Microservice Architecture](#)
- [Improve cost visibility of Amazon EKS with AWS Split Cost Allocation Data](#)
- [Improve cost visibility of Amazon ECS and AWS Batch with AWS Split Cost Allocation Data](#)

COST 7. How do you use pricing models to reduce cost?

Use the pricing model that is most appropriate for your resources to minimize expense.

Best practices

- [COST07-BP01 Perform pricing model analysis](#)
- [COST07-BP02 Choose Regions based on cost](#)
- [COST07-BP03 Select third-party agreements with cost-efficient terms](#)
- [COST07-BP04 Implement pricing models for all components of this workload](#)
- [COST07-BP05 Perform pricing model analysis at the management account level](#)

COST07-BP01 Perform pricing model analysis

Analyze each component of the workload. Determine if the component and resources will be running for extended periods (for commitment discounts) or dynamic and short-running (for spot or on-demand). Perform an analysis on the workload using the recommendations in cost management tools and apply business rules to those recommendations to achieve high returns.

Level of risk exposed if this best practice is not established: High

Implementation guidance

AWS has multiple [pricing models](#) that allow you to pay for your resources in the most cost-effective way that suits your organization's needs and depending on product. Work with your teams to

determine the most appropriate pricing model. Often your pricing model consists of a combination of multiple options, as determined by your availability

On-Demand Instances allow you to pay for compute or database capacity by the hour or by the second (60 seconds minimum) depending on which instances you run, without long-term commitments or upfront payments.

Savings Plans are a flexible pricing model that offers low prices on Amazon EC2, Lambda, and AWS Fargate usage, in exchange for a commitment to a consistent amount of usage (measured in dollars per hour) over one year or three years terms.

Spot Instances are an Amazon EC2 pricing mechanism that allows you request spare compute capacity at discounted hourly rate (up to 90% off the on-demand price) without upfront commitment.

Reserved Instances allow you up to 75 percent discount by prepaying for capacity. For more details, see [Optimizing costs with reservations](#).

You might choose to include a Savings Plans for the resources associated with the production, quality, and development environments. Alternatively, because sandbox resources are only powered on when needed, you might choose an on-demand model for the resources in that environment. Use Amazon [Spot Instances](#) to reduce Amazon EC2 costs or use [Compute Savings Plans](#) to reduce Amazon EC2, Fargate, and Lambda cost. The [AWS Cost Explorer](#) recommendations tool provides opportunities for commitment discounts with Saving plans.

If you have been purchasing [Reserved Instances](#) for Amazon EC2 in the past or have established cost allocation practices inside your organization, you can continue using Amazon EC2 Reserved Instances for the time being. However, we recommend working on a strategy to use Savings Plans in the future as a more flexible cost savings mechanism. You can refresh Savings Plans (SP) Recommendations in AWS Cost Management to generate new Savings Plans Recommendations at any time. Use Reserved Instances (RI) to reduce Amazon RDS, Amazon Redshift, Amazon ElastiCache, and Amazon OpenSearch Service costs. Saving Plans and Reserved Instances are available in three options: all upfront, partial upfront and no upfront payments. Use the recommendations provided in AWS Cost Explorer RI and SP purchase recommendations.

To find opportunities for Spot workloads, use an hourly view of your overall usage, and look for regular periods of changing usage or elasticity. You can use Spot Instances for various fault-tolerant and flexible applications. Examples include stateless web servers, API endpoints, big data and analytics applications, containerized workloads, CI/CD, and other flexible workloads.

Analyze your Amazon EC2 and Amazon RDS instances whether they can be turned off when you don't use (after hours and weekends). This approach will allow you to reduce costs by 70% or more compared to using them 24/7. If you have Amazon Redshift clusters that only need to be available at specific times, you can pause the cluster and later resume it. When the Amazon Redshift cluster or Amazon EC2 and Amazon RDS Instance is stopped, the compute billing halts and only the storage charge applies.

Note that [On-Demand Capacity reservations](#) (ODCR) are not a pricing discount. Capacity Reservations are charged at the equivalent On-Demand rate, whether you run instances in reserved capacity or not. They should be considered when you need to provide enough capacity for the resources you plan to run. ODCRs don't have to be tied to long-term commitments, as they can be cancelled when you no longer need them, but they can also benefit from the discounts that Savings Plans or Reserved Instances provide.

Implementation steps

- **Analyze workload elasticity:** Using the hourly granularity in Cost Explorer or a custom dashboard, analyze your workload's elasticity. Look for regular changes in the number of instances that are running. Short duration instances are candidates for Spot Instances or Spot Fleet.
 - [Well-Architected Lab: Cost Explorer](#)
 - [Well-Architected Lab: Cost Visualization](#)
- **Review existing pricing contracts:** Review current contracts or commitments for long term needs. Analyze what you currently have and how much those commitments are in use. Leverage pre-existing contractual discounts or enterprise agreements. [Enterprise Agreements](#) give customers the option to tailor agreements that best suit their needs. For long term commitments, consider reserved pricing discounts, Reserved Instances or Savings Plans for the specific instance type, instance family, AWS Region, and Availability Zones.
- **Perform a commitment discount analysis:** Using Cost Explorer in your account, review the Savings Plans and Reserved Instance recommendations. To verify that you implement the correct recommendations with the required discounts and risk, follow the [Well-Architected labs](#).

Resources

Related documents:

- [Accessing Reserved Instance recommendations](#)

- [Instance purchasing options](#)
- [AWS Enterprise](#)

Related videos:

- [Save up to 90% and run production workloads on Spot](#)

Related examples:

- [Well-Architected Lab: Cost Explorer](#)
- [Well-Architected Lab: Cost Visualization](#)
- [Well-Architected Lab: Pricing Models](#)

COST07-BP02 Choose Regions based on cost

Resource pricing may be different in each Region. Identify Regional cost differences and only deploy in Regions with higher costs to meet latency, data residency and data sovereignty requirements. Factoring in Region cost helps you pay the lowest overall price for this workload.

Level of risk exposed if this best practice is not established: Medium

Implementation guidance

The [AWS Cloud Infrastructure](#) is global, hosted in [multiple locations world-wide](#), and built around AWS Regions, Availability Zones, Local Zones, AWS Outposts, and Wavelength Zones. A Region is a physical location in the world and each Region is a separate geographic area where AWS has multiple Availability Zones. Availability Zones which are multiple isolated locations within each Region consist of one or more discrete data centers, each with redundant power, networking, and connectivity.

Each AWS Region operates within local market conditions, and resource pricing is different in each Region due to differences in the cost of land, fiber, electricity, and taxes, for example. Choose a specific Region to operate a component of or your entire solution so that you can run at the lowest possible price globally. Use [AWS Calculator](#) to estimate the costs of your workload in various Regions by searching services by location type (Region, wave length zone and local zone) and Region.

When you architect your solutions, a best practice is to seek to place computing resources closer to users to provide lower latency and strong data sovereignty. Select the geographic location based on your business, data privacy, performance, and security requirements. For applications with global end users, use multiple locations.

Use Regions that provide lower prices for AWS services to deploy your workloads if you have no obligations in data privacy, security and business requirements. For example, if your default Region is Asia Pacific (Sydney) (ap-southwest-2), and if there are no restrictions (data privacy, security, for example) to use other Regions, deploying non-critical (development and test) Amazon EC2 instances in US East (N. Virginia) (us-east-1) will cost you less.

	<i>Compliance</i>	<i>Latency</i>	<i>Cost</i>	<i>Services / Features</i>
Region 1	✓	15 ms	\$\$	✓
Region 2	✓	20 ms	\$\$\$	X
Region 3	✓	80 ms	\$	✓
Region 4	✓	15 ms	\$\$	✓
Region 5	✓	20 ms	\$\$\$	X
Region 6	✓	15 ms	\$	✓
Region 7	✓	80 ms	\$	✓
Region 8	✓	15 ms	\$	X

Region feature matrix table

The preceding matrix table shows us that Region 6 is the best option for this given scenario because latency is low compared to other Regions, service is available, and it is the least expensive Region.

Implementation steps

- **Review AWS Region pricing:** Analyze the workload costs in the current Region. Starting with the highest costs by service and usage type, calculate the costs in other Regions that are available. If the forecasted saving outweighs the cost of moving the component or workload, migrate to the new Region.
- **Review requirements for multi-Region deployments:** Analyze your business requirements and obligations (data privacy, security, or performance) to find out if there are any restrictions for you

to not to use multiple Regions. If there are no obligations to restrict you to use single Region, then use multiple Regions.

- **Analyze required data transfer:** Consider data transfer costs when selecting Regions. Keep your data close to your customer and close to the resources. Select less costly AWS Regions where data flows and where there is minimal data transfer. Depending on your business requirements for data transfer, you can use [Amazon CloudFront](#), [AWS PrivateLink](#), [AWS Direct Connect](#), and [AWS Virtual Private Network](#) to reduce your networking costs, improve performance, and enhance security.

Resources

Related documents:

- [Accessing Reserved Instance recommendations](#)
- [Amazon EC2 pricing](#)
- [Instance purchasing options](#)
- [Region Table](#)

Related videos:

- [Save up to 90% and run production workloads on Spot](#)

Related examples:

- [Overview of Data Transfer Costs for Common Architectures](#)
- [Cost Considerations for Global Deployments](#)
- [What to Consider when Selecting a Region for your Workloads](#)

COST07-BP03 Select third-party agreements with cost-efficient terms

Cost efficient agreements and terms ensure the cost of these services scales with the benefits they provide. Select agreements and pricing that scale when they provide additional benefits to your organization.

Level of risk exposed if this best practice is not established: Medium

Implementation guidance

There are multiple products on the market that can help you manage costs in your cloud environments. They may have some differences in terms of features that depend on customer requirements, such as some focusing on cost governance or cost visibility and others on cost optimization. One key factor for effective cost optimization and governance is using the right tool with necessary features and the right pricing model. These products have different pricing models. Some charge you a certain percentage of your monthly bill, while others charge a percentage of your realized savings. Ideally, you should pay only for what you need.

When you use third-party solutions or services in the cloud, it's important that the pricing structures are aligned to your desired outcomes. Pricing should scale with the outcomes and value it provides. For example, in software that takes a percentage of savings it provides, the more you save (outcome), the more it charges. License agreements where you pay more as your expenses increase might not always be in your best interest for optimizing costs. However, if the vendor offers clear benefits for all parts of your bill, this scaling fee might be justified.

For example, a solution that provides recommendations for Amazon EC2 and charges a percentage of your entire bill can become more expensive if you use other services that provide no benefit. Another example is a managed service that is charged at a percentage of the cost of managed resources. A larger instance size may not necessarily require more management effort, but can be charged more. Verify that these service pricing arrangements include a cost optimization program or features in their service to drive efficiency.

Customers may find these products on the market more advanced or easier to use. You need to consider the cost of these products and think about potential cost optimization outcomes in the long term.

Implementation steps

- **Analyze third-party agreements and terms:** Review the pricing in third party agreements. Perform modeling for different levels of your usage, and factor in new costs such as new service usage, or increases in current services due to workload growth. Decide if the additional costs provide the required benefits to your business.

Resources

Related documents:

- [Accessing Reserved Instance recommendations](#)

- [Instance purchasing options](#)

Related videos:

- [Save up to 90% and run production workloads on Spot](#)

COST07-BP04 Implement pricing models for all components of this workload

Permanently running resources should utilize reserved capacity such as Savings Plans or Reserved Instances. Short-term capacity is configured to use Spot Instances, or Spot Fleet. On-Demand Instances are only used for short-term workloads that cannot be interrupted and do not run long enough for reserved capacity, between 25% to 75% of the period, depending on the resource type.

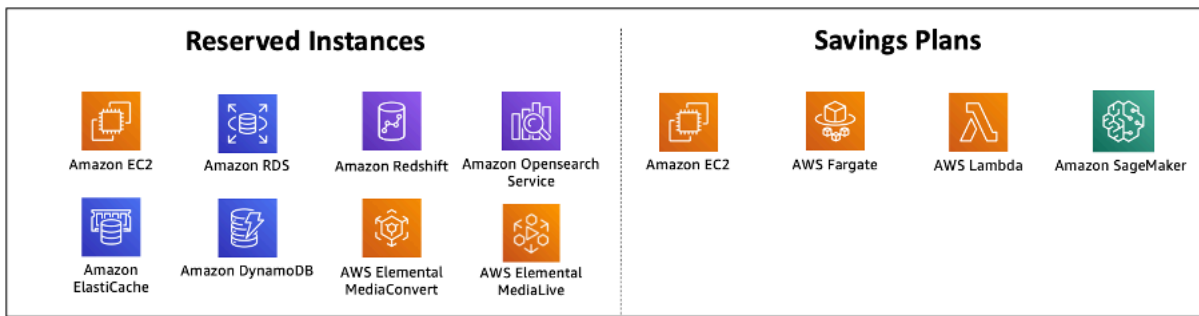
Level of risk exposed if this best practice is not established: Low

Implementation guidance

To improve cost efficiency, AWS provides multiple commitment recommendations based on your past usage. You can use these recommendations to understand what you can save, and how the commitment will be used. You can use these services as On-Demand, Spot, or make a commitment for a certain period of time and reduce your on-demand costs with Reserved Instances (RIs) and Savings Plans (SPs). You need to understand not only each workload components and multiple AWS services, but also commitment discounts, purchase options, and Spot Instances for these services to optimize your workload.

Consider the requirements of your workload's components, and understand the different pricing models for these services. Define the availability requirement of these components. Determine if there are multiple independent resources that perform the function in the workload, and what the workload requirements are over time. Compare the cost of the resources using the default On-Demand pricing model and other applicable models. Factor in any potential changes in resources or workload components.

For example, let's look at this Web Application Architecture on AWS. This sample workload consists of multiple AWS services, such as Amazon Route 53, AWS WAF, Amazon CloudFront, Amazon EC2 instances, Amazon RDS instances, Load Balancers, Amazon S3 storage, and Amazon Elastic File System (Amazon EFS). You need to review each of these services, and identify potential cost saving opportunities with different pricing models. Some of them may be eligible for RIs or SPs, while some of them may be available only on-demand. As the following image shows, some of the AWS services can be committed using RIs or SPs.



AWS services committed using Reserved Instances and Savings Plans

Implementation steps

- **Implement pricing models:** Using your analysis results, purchase Savings Plans, Reserved Instances, or implement Spot Instances. If it is your first commitment purchase, choose the top five or ten recommendations in the list, then monitor and analyze the results over the next month or two. AWS Cost Management Console guides you through the process. Review the RI or SP recommendations from the console, customize the recommendations (type, payment, and term), and review hourly commitment (for example \$20 per hour), and then add to cart. Discounts apply automatically to eligible usage. Purchase a small amount of commitment discounts in regular cycles (for example every 2 weeks or monthly). Implement Spot Instances for workloads that can be interrupted or are stateless. Finally, select on-demand Amazon EC2 instances and allocate resources for the remaining requirements.
- **Workload review cycle:** Implement a review cycle for the workload that specifically analyzes pricing model coverage. Once the workload has the required coverage, purchase additional commitment discounts partially (every few months), or as your organization usage changes.

Resources

Related documents:

- [Understanding your Savings Plans recommendations](#)
- [Accessing Reserved Instance recommendations](#)
- [How to Purchase Reserved Instances](#)
- [Instance purchasing options](#)
- [Spot Instances](#)
- [Reservation models for other AWS services](#)

- [Savings Plans Supported Services](#)

Related videos:

- [Save up to 90% and run production workloads on Spot](#)

Related examples:

- [What should you consider before purchasing Savings Plans?](#)
- [How can I use Cost Explorer to analyze my spending and usage?](#)

COST07-BP05 Perform pricing model analysis at the management account level

Check billing and cost management tools and see recommended discounts with commitments and reservations to perform regular analysis at the management account level.

Level of risk exposed if this best practice is not established: Low

Implementation guidance

Performing regular cost modeling helps you implement opportunities to optimize across multiple workloads. For example, if multiple workloads use On-Demand Instances at an aggregate level, the risk of change is lower, and implementing a commitment-based discount can achieve a lower overall cost. It is recommended to perform analysis in regular cycles of two weeks to one month. This allows you to make small adjustment purchases, so the coverage of your pricing models continues to evolve with your changing workloads and their components.

Use the [AWS Cost Explorer](#) recommendations tool to find opportunities for commitment discounts in your management account. Recommendations at the management account level are calculated considering usage across all of the accounts in your AWS organization that have Reserve Instances (RI) or Savings Plans (SP). They're also calculated when discount sharing is activated to recommend a commitment that maximizes savings across accounts.

While purchasing at the management account level optimizes for max savings in many cases, there may be situations where you might consider purchasing SPs at the linked account level, like when you want the discounts to apply first to usage in that particular linked account. Member account recommendations are calculated at the individual account level, to maximize savings for each isolated account. If your account owns both RI and SP commitments, they will be applied in this order:

1. Zonal RI
2. Standard RI
3. Convertible RI
4. Instance Savings Plan
5. Compute Savings Plan

If you purchase an SP at the management account level, the savings will be applied based on highest to lowest discount percentage. SPs at the management account level look across all linked accounts and apply the savings wherever the discount will be the highest. If you wish to restrict where the savings are applied, you can purchase a Savings Plan at the linked account level and any time that account is running eligible compute services, the discount will be applied there first. When the account is not running eligible compute services, the discount will be shared across the other linked accounts under the same management account. Discount sharing is turned on by default, but can be turned off if needed.

In a Consolidated Billing Family, Savings Plans are applied first to the owner account's usage, and then to other accounts' usage. This occurs only if you have sharing enabled. Your Savings Plans are applied to your highest savings percentage first. If there are multiple usages with equal savings percentages, Savings Plans are applied to the first usage with the lowest Savings Plans rate. Savings Plans continue to apply until there are no more remaining uses or your commitment is exhausted. Any remaining usage is charged at the On-Demand rates. You can refresh Savings Plans Recommendations in AWS Cost Management to generate new Savings Plans Recommendations at any time.

After analyzing flexibility of instances, you can commit by following recommendations. Create cost modeling by analyzing the workload's short-term costs with potential different resource options, analyzing AWS pricing models, and aligning them with your business requirements to find out total cost of ownership and [cost optimization](#) opportunities.

Implementation steps

Perform a commitment discount analysis: Use Cost Explorer in your account review the Savings Plans and Reserved Instance recommendations. Make sure you understand Saving Plan recommendations, and estimate your monthly spend and monthly savings. Review recommendations at the management account level, which are calculated considering usage across all of the member accounts in your AWS organization that have RI or Savings Plans discount sharing enabled for maximum savings across accounts. You can verify that you implemented the

correct recommendations with the required discounts and risk by following the Well-Architected labs.

Resources

Related documents:

- [How does AWS pricing work?](#)
- [Instance purchasing options](#)
- [Saving Plan Overview](#)
- [Saving Plan recommendations](#)
- [Accessing Reserved Instance recommendations](#)
- [Understanding your Saving Plans recommendation](#)
- [How Savings Plans apply to your AWS usage](#)
- [Saving Plans with Consolidated Billing](#)
- [Turning on shared reserved instances and Savings Plans discounts](#)

Related videos:

- [Save up to 90% and run production workloads on Spot](#)

Related examples:

- [What should I consider before purchasing a Savings Plan?](#)
- [How can I use rolling Savings Plans to reduce commitment risk?](#)
- [When to Use Spot Instances](#)

COST 8. How do you plan for data transfer charges?

Verify that you plan and monitor data transfer charges so that you can make architectural decisions to minimize costs. A small yet effective architectural change can drastically reduce your operational costs over time.

Best practices

- [COST08-BP01 Perform data transfer modeling](#)

- [COST08-BP02 Select components to optimize data transfer cost](#)
- [COST08-BP03 Implement services to reduce data transfer costs](#)

COST08-BP01 Perform data transfer modeling

Gather organization requirements and perform data transfer modeling of the workload and each of its components. This identifies the lowest cost point for its current data transfer requirements.

Level of risk exposed if this best practice is not established: High

Implementation guidance

When designing a solution in the cloud, data transfer fees are usually neglected due to habits of designing architecture using on-premises data centers or lack of knowledge. Data transfer charges in AWS are determined by the source, destination, and volume of traffic. Factoring in these fees during the design phase can lead to cost savings. Understanding where the data transfer occurs in your workload, the cost of the transfer, and its associated benefit is very important to accurately estimate total cost of ownership (TCO). This allows you to make an informed decision to modify or accept the architectural decision. For example, you may have a Multi-Availability Zone configuration where you replicate data between the Availability Zones.

You model the components of services which transfer the data in your workload, and decide that this is an acceptable cost (similar to paying for compute and storage in both Availability Zones) to achieve the required reliability and resilience. Model the costs over different usage levels. Workload usage can change over time, and different services may be more cost effective at different levels.

While modeling your data transfer, think about how much data is ingested and where that data comes from. Additionally, consider how much data is processed and how much storage or compute capacity is needed. During modeling, follow networking best practices for your workload architecture to optimize your potential data transfer costs.

The AWS Pricing Calculator can help you see estimated costs for specific AWS services and expected data transfer. If you have a workload already running (for test purposes or in a pre-production environment), use [AWS Cost Explorer](#) or [AWS Cost and Usage Report](#) (CUR) to understand and model your data transfer costs. Configure a proof of concept (PoC) or test your workload, and run a test with a realistic simulated load. You can model your costs at different workload demands.

Implementation steps

- **Identify requirements:** What is the primary goal and business requirements for the planned data transfer between source and destination? What is the expected business outcome at the end? Gather business requirements and define expected outcome.
- **Identify source and destination:** What is the data source and destination for the data transfer, such as within AWS Regions, to AWS services, or out to the internet?
 - [Data transfer within an AWS Region](#)
 - [Data transfer between AWS Regions](#)
 - [Data transfer out to the internet](#)
- **Identify data classifications:** What is the data classification for this data transfer? What kind of data is it? How big is the data? How frequently must data be transferred? Is data sensitive?
- **Identify AWS services or tools to use:** Which AWS services are used for this data transfer? Is it possible to use an already-provisioned service for another workload?
- **Calculate data transfer costs:** Use [AWS Pricing](#) the data transfer modeling you created previously to calculate the data transfer costs for the workload. Calculate the data transfer costs at different usage levels, for both increases and reductions in workload usage. Where there are multiple options for the workload architecture, calculate the cost for each option for comparison.
- **Link costs to outcomes:** For each data transfer cost incurred, specify the outcome that it achieves for the workload. If it is transfer between components, it may be for decoupling, if it is between Availability Zones it may be for redundancy.
- **Create data transfer modeling:** After gathering all information, create a conceptual base data transfer modeling for multiple use cases and different workloads.

Resources

Related documents:

- [AWS caching solutions](#)
- [AWS Pricing](#)
- [Amazon EC2 Pricing](#)
- [Amazon VPC pricing](#)
- [Understanding data transfer charges](#)

Related videos:

- [Monitoring and Optimizing Your Data Transfer Costs](#)
- [S3 Transfer Acceleration](#)

Related examples:

- [Overview of Data Transfer Costs for Common Architectures](#)
- [AWS Prescriptive Guidance for Networking](#)

COST08-BP02 Select components to optimize data transfer cost

All components are selected, and architecture is designed to reduce data transfer costs. This includes using components such as wide-area-network (WAN) optimization and Multi-Availability Zone (AZ) configurations

Level of risk exposed if this best practice is not established: Medium

Implementation guidance

Architecting for data transfer minimizes data transfer costs. This may involve using content delivery networks to locate data closer to users, or using dedicated network links from your premises to AWS. You can also use WAN optimization and application optimization to reduce the amount of data that is transferred between components.

When transferring data to or within the AWS Cloud, it is essential to know the destination based on varied use cases, the nature of the data, and the available network resources in order to select the right AWS services to optimize data transfer. AWS offers a range of data transfer services tailored for diverse data migration requirements. Select the right [data storage](#) and [data transfer](#) options based on the business needs within your organization.

When planning or reviewing your workload architecture, consider the following:

- **Use VPC endpoints within AWS:** VPC endpoints allow for private connections between your VPC and supported AWS services. This allows you to avoid using the public internet, which can lead to data transfer costs.
- **Use a NAT gateway:** Use a [NAT gateway](#) so that instances in a private subnet can connect to the internet or to the services outside your VPC. Check whether the resources behind the NAT gateway that send the most traffic are in the same Availability Zone as the NAT gateway. If they

are not, create new NAT gateways in the same Availability Zone as the resource to reduce cross-AZ data transfer charges.

- **Use AWS Direct Connect** AWS Direct Connect bypasses the public internet and establishes a direct, private connection between your on-premises network and AWS. This can be more cost-effective and consistent than transferring large volumes of data over the internet.
- **Avoid transferring data across Regional boundaries:** Data transfers between AWS Regions (from one Region to another) typically incur charges. It should be a very thoughtful decision to pursue a multi-Region path. For more detail, see [Multi-Region scenarios](#).
- **Monitor data transfer:** Use Amazon CloudWatch and [VPC flow logs](#) to capture details about your data transfer and network usage. Analyze captured network traffic information in your VPCs, such as IP address or range going to and from network interfaces.
- **Analyze your network usage:** Use metering and reporting tools such as AWS Cost Explorer, CUDOS Dashboards, or CloudWatch to understand data transfer cost of your workload.

Implementation steps

- **Select components for data transfer:** Using the data transfer modeling explained in [COST08-BP01 Perform data transfer modeling](#), focus on where the largest data transfer costs are or where they would be if the workload usage changes. Look for alternative architectures or additional components that remove or reduce the need for data transfer (or lower its cost).

Resources

Related best practices:

- [COST08-BP01 Perform data transfer modeling](#)
- [COST08-BP03 Implement services to reduce data transfer costs](#)

Related documents:

- [Cloud Data Migration](#)
- [AWS caching solutions](#)
- [Deliver content faster with Amazon CloudFront](#)

Related examples:

- [Overview of Data Transfer Costs for Common Architectures](#)
- [AWS Network Optimization Tips](#)
- [Optimize performance and reduce costs for network analytics with VPC Flow Logs in Apache Parquet format](#)

COST08-BP03 Implement services to reduce data transfer costs

Implement services to reduce data transfer. For example, use edge locations or content delivery networks (CDN) to deliver content to end users, build caching layers in front of your application servers or databases, and use dedicated network connections instead of VPN for connectivity to the cloud.

Level of risk exposed if this best practice is not established: Medium

Implementation guidance

There are various AWS services that can help you to optimize your network data transfer usage. Depending on your workload components, type, and cloud architecture, these services can assist you in compression, caching, and sharing and distribution of your traffic on the cloud.

- [Amazon CloudFront](#) is a global content delivery network that delivers data with low latency and high transfer speeds. It caches data at edge locations across the world, which reduces the load on your resources. By using CloudFront, you can reduce the administrative effort in delivering content to large numbers of users globally with minimum latency. The [security savings bundle](#) can help you to save up to 30% on your CloudFront usage if you plan to grow your usage over time.
- [AWS Direct Connect](#) allows you to establish a dedicated network connection to AWS. This can reduce network costs, increase bandwidth, and provide a more consistent network experience than internet-based connections.
- [AWS VPN](#) allows you to establish a secure and private connection between your private network and the AWS global network. It is ideal for small offices or business partners because it provides simplified connectivity, and it is a fully managed and elastic service.
- [VPC Endpoints](#) allow connectivity between AWS services over private networking and can be used to reduce public data transfer and [NAT gateway](#) costs. [Gateway VPC endpoints](#) have no hourly charges, and support Amazon S3 and Amazon DynamoDB. [Interface VPC endpoints](#) are provided by [AWS PrivateLink](#) and have an hourly fee and per-GB usage cost.

- [NAT gateways](#) provide built-in scaling and management for reducing costs as opposed to a standalone NAT instance. Place NAT gateways in the same Availability Zones as high traffic instances and consider using VPC endpoints for the instances that need to access Amazon DynamoDB or Amazon S3 to reduce the data transfer and processing costs.
- Use [AWS Snow Family](#) devices which have computing resources to collect and process data at the edge. AWS Snow Family devices ([Snowball Edge](#), [Snowball Edge](#) and [Snowmobile](#)) allow you to move petabytes of data to the AWS Cloud cost effectively and offline.

Implementation steps

- **Implement services:** Select applicable AWS network services based on your service workload type using the data transfer modeling and reviewing VPC Flow Logs. Look at where the largest costs and highest volume flows are. Review the AWS services and assess whether there is a service that reduces or removes the transfer, specifically networking and content delivery. Also look for caching services where there is repeated access to data or large amounts of data.

Resources

Related documents:

- [AWS Direct Connect](#)
- [AWS Explore Our Products](#)
- [AWS caching solutions](#)
- [Amazon CloudFront](#)
- [AWS Snow Family](#)
- [Amazon CloudFront Security Savings Bundle](#)

Related videos:

- [Monitoring and Optimizing Your Data Transfer Costs](#)
- [AWS Cost Optimization Series: CloudFront](#)
- [How can I reduce data transfer charges for my NAT gateway?](#)

Related examples:

- [How-to chargeback shared services: An AWS Transit Gateway example](#)
- [Understand AWS data transfer details in depth from cost and usage report using Athena query and QuickSight](#)
- [Overview of Data Transfer Costs for Common Architectures](#)
- [Using AWS Cost Explorer to analyze data transfer costs](#)
- [Cost-Optimizing your AWS architectures by utilizing Amazon CloudFront features](#)
- [How can I reduce data transfer charges for my NAT gateway?](#)

Manage demand and supply resources

Question

- [COST 9. How do you manage demand, and supply resources?](#)

COST 9. How do you manage demand, and supply resources?

For a workload that has balanced spend and performance, verify that everything you pay for is used and avoid significantly underutilizing instances. A skewed utilization metric in either direction has an adverse impact on your organization, in either operational costs (degraded performance due to over-utilization), or wasted AWS expenditures (due to over-provisioning).

Best practices

- [COST09-BP01 Perform an analysis on the workload demand](#)
- [COST09-BP02 Implement a buffer or throttle to manage demand](#)
- [COST09-BP03 Supply resources dynamically](#)

COST09-BP01 Perform an analysis on the workload demand

Analyze the demand of the workload over time. Verify that the analysis covers seasonal trends and accurately represents operating conditions over the full workload lifetime. Analysis effort should reflect the potential benefit, for example, time spent is proportional to the workload cost.

Level of risk exposed if this best practice is not established: High

Implementation guidance

Analyzing workload demand for cloud computing involves understanding the patterns and characteristics of computing tasks that are initiated in the cloud environment. This analysis helps users optimize resource allocation, manage costs, and verify that performance meets required levels.

Know the requirements of the workload. Your organization's requirements should indicate the workload response times for requests. The response time can be used to determine if the demand is managed, or if the supply of resources should change to meet the demand.

The analysis should include the predictability and repeatability of the demand, the rate of change in demand, and the amount of change in demand. Perform the analysis over a long enough period to incorporate any seasonal variance, such as end-of-month processing or holiday peaks.

Analysis effort should reflect the potential benefits of implementing scaling. Look at the expected total cost of the component and any increases or decreases in usage and cost over the workload's lifetime.

The following are some key aspects to consider when performing workload demand analysis for cloud computing:

1. **Resource utilization and performance metrics:** Analyze how AWS resources are being used over time. Determine peak and off-peak usage patterns to optimize resource allocation and scaling strategies. Monitor performance metrics such as response times, latency, throughput, and error rates. These metrics help assess the overall health and efficiency of the cloud infrastructure.
2. **User and application scaling behaviour:** Understand user behavior and how it affects workload demand. Examining the patterns of user traffic assists in enhancing the delivery of content and the responsiveness of applications. Analyze how workloads scale with increasing demand. Determine whether auto-scaling parameters are configured correctly and effectively for handling load fluctuations.
3. **Workload types:** Identify the different types of workloads running in the cloud, such as batch processing, real-time data processing, web applications, databases, or machine learning. Each type of workload may have different resource requirements and performance profiles.
4. **Service-level agreements (SLAs):** Compare actual performance with SLAs to ensure compliance and identify areas that need improvement.

You can use [Amazon CloudWatch](#) to collect and track metrics, monitor log files, set alarms, and automatically react to changes in your AWS resources. You can also use Amazon CloudWatch to gain system-wide visibility into resource utilization, application performance, and operational health.

With [AWS Trusted Advisor](#), you can provision your resources following best practices to improve system performance and reliability, increase security, and look for opportunities to save money. You can also turn off non-production instances and use Amazon CloudWatch and Auto Scaling to match increases or reductions in demand.

Finally, you can use [AWS Cost Explorer](#) or [Quick Suite](#) with the AWS Cost and Usage Report (CUR) file or your application logs to perform advanced analysis of workload demand.

Overall, a comprehensive workload demand analysis allows organizations to make informed decisions about resource provisioning, scaling, and optimization, leading to better performance, cost efficiency, and user satisfaction.

Implementation steps

- **Analyze existing workload data:** Analyze data from the existing workload, previous versions of the workload, or predicted usage patterns. Use Amazon CloudWatch, log files and monitoring data to gain insight on how workload was used. Analyze a full cycle of the workload, and collect data for any seasonal changes such as end-of-month or end-of-year events. The effort reflected in the analysis should reflect the workload characteristics. The largest effort should be placed on high-value workloads that have the largest changes in demand. The least effort should be placed on low-value workloads that have minimal changes in demand.
- **Forecast outside influence:** Meet with team members from across the organization that can influence or change the demand in the workload. Common teams would be sales, marketing, or business development. Work with them to know the cycles they operate within, and if there are any events that would change the demand of the workload. Forecast the workload demand with this data.

Resources

Related documents:

- [Amazon CloudWatch](#)
- [AWS Trusted Advisor](#)

- [AWS X-Ray](#)
- [AWS Auto Scaling](#)
- [AWS Instance Scheduler](#)
- [Getting started with Amazon SQS](#)
- [AWS Cost Explorer](#)
- [Quick Suite](#)

Related examples:

- [Monitor, Track and Analyze for cost optimization](#)
- [Searching and analyzing logs in CloudWatch](#)

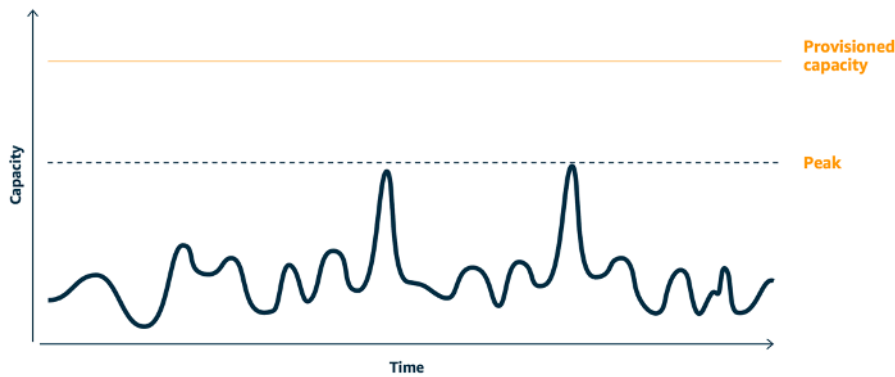
COST09-BP02 Implement a buffer or throttle to manage demand

Buffering and throttling modify the demand on your workload, smoothing out any peaks. Implement throttling when your clients perform retries. Implement buffering to store the request and defer processing until a later time. Verify that your throttles and buffers are designed so clients receive a response in the required time.

Level of risk exposed if this best practice is not established: Medium

Implementation guidance

Implementing a buffer or throttle is crucial in cloud computing in order to manage demand and reduce the provisioned capacity required for your workload. For optimal performance, it's essential to gauge the total demand, including peaks, the pace of change in requests, and the necessary response time. When clients have the ability to resend their requests, it becomes practical to apply throttling. Conversely, for clients lacking retry functionalities, the ideal approach is implementing a buffer solution. Such buffers streamline the influx of requests and optimize the interaction of applications with varied operational speeds.



Demand curve with two distinct peaks that require high provisioned capacity

Assume a workload with the demand curve shown in preceding image. This workload has two peaks, and to handle those peaks, the resource capacity as shown by orange line is provisioned. The resources and energy used for this workload are not indicated by the area under the demand curve, but the area under the provisioned capacity line, as provisioned capacity is needed to handle those two peaks. Flattening the workload demand curve can help you to reduce the provisioned capacity for a workload and reduce its environmental impact. To smooth out the peak, consider to implement throttling or buffering solution.

To understand them better, let's explore throttling and buffering.

Throttling: If the source of the demand has retry capability, then you can implement throttling. Throttling tells the source that if it cannot service the request at the current time, it should try again later. The source waits for a period of time, and then retries the request. Implementing throttling has the advantage of limiting the maximum amount of resources and costs of the workload. In AWS, you can use [Amazon API Gateway](#) to implement throttling.

Buffer based: A buffer-based approach uses *producers* (components that send messages to the queue), *consumers* (components that receive messages from the queue), and a *queue* (which holds messages) to store the messages. Messages are read by consumers and processed, allowing the messages to run at the rate that meets the consumers' business requirements. By using a buffer-centric methodology, messages from producers are housed in queues or streams, ready to be accessed by consumers at a pace that aligns with their operational demands.

In AWS, you can choose from multiple services to implement a buffering approach. [Amazon Simple Queue Service \(Amazon SQS\)](#) is a managed service that provides queues that allow a single consumer to read individual messages. [Amazon Kinesis](#) provides a stream that allows many consumers to read the same messages.

Buffering and throttling can smooth out any peaks by modifying the demand on your workload. Use throttling when clients retry actions and use buffering to hold the request and process it later. When working with a buffer-based approach, architect your workload to service the request in the required time, verify that you are able to handle duplicate requests for work. Analyze the overall demand, rate of change, and required response time to right size the throttle or buffer required.

Implementation steps

- **Analyze the client requirements:** Analyze the client requests to determine if they are capable of performing retries. For clients that cannot perform retries, buffers need to be implemented. Analyze the overall demand, rate of change, and required response time to determine the size of throttle or buffer required.
- **Implement a buffer or throttle:** Implement a buffer or throttle in the workload. A queue such as Amazon Simple Queue Service (Amazon SQS) can provide a buffer to your workload components. Amazon API Gateway can provide throttling for your workload components.

Resources

Related best practices:

- [SUS02-BP06 Implement buffering or throttling to flatten the demand curve](#)
- [REL05-BP02 Throttle requests](#)

Related documents:

- [AWS Auto Scaling](#)
- [AWS Instance Scheduler](#)
- [Amazon API Gateway](#)
- [Amazon Simple Queue Service](#)
- [Getting started with Amazon SQS](#)
- [Amazon Kinesis](#)

Related videos:

- [Choosing the Right Messaging Service for Your Distributed App](#)

Related examples:

- [Managing and monitoring API throttling in your workloads](#)
- [Throttling a tiered, multi-tenant REST API at scale using API Gateway](#)
- [Enabling Tiering and Throttling in a Multi-Tenant Amazon EKS SaaS Solution Using Amazon API Gateway](#)
- [Application integration Using Queues and Messages](#)

COST09-BP03 Supply resources dynamically

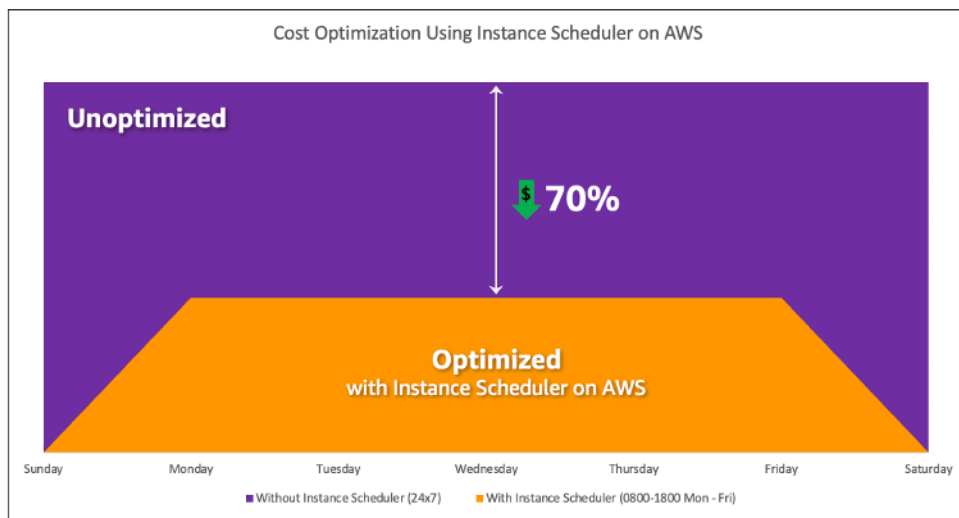
Resources are provisioned in a planned manner. This can be demand-based, such as through automatic scaling, or time-based, where demand is predictable and resources are provided based on time. These methods result in the least amount of over-provisioning or under-provisioning.

Level of risk exposed if this best practice is not established: Low

Implementation guidance

There are several ways for AWS customers to increase the resources available to their applications and supply resources to meet the demand. One of these options is to use AWS Instance Scheduler, which automates the starting and stopping of Amazon Elastic Compute Cloud (Amazon EC2) and Amazon Relational Database Service (Amazon RDS) instances. The other option is to use AWS Auto Scaling, which allows you to automatically scale your computing resources based on the demand of your application or service. Supplying resources based on demand will allow you to pay for the resources you use only, reduce cost by launching resources when they are needed, and terminate them when they aren't.

[AWS Instance Scheduler](#) allows you to configure the stop and start of your Amazon EC2 and Amazon RDS instances at defined times so that you can meet the demand for the same resources within a consistent time pattern such as every day user access Amazon EC2 instances at eight in the morning that they don't need after six at night. This solution helps reduce operational cost by stopping resources that are not in use and starting them when they are needed.



Cost optimization with AWS Instance Scheduler.

You can also easily configure schedules for your Amazon EC2 instances across your accounts and Regions with a simple user interface (UI) using AWS Systems Manager Quick Setup. You can schedule Amazon EC2 or Amazon RDS instances with AWS Instance Scheduler and you can stop and start existing instances. However, you cannot stop and start instances which are part of your Auto Scaling group (ASG) or that manage services such as Amazon Redshift or Amazon OpenSearch Service. Auto Scaling groups have their own scheduling for the instances in the group and these instances are created.

[AWS Auto Scaling](#) helps you adjust your capacity to maintain steady, predictable performance at the lowest possible cost to meet changing demand. It is a fully managed and free service to scale the capacity of your application that integrates with Amazon EC2 instances and Spot Fleets, Amazon ECS, Amazon DynamoDB, and Amazon Aurora. Auto Scaling provides automatic resource discovery to help find resources in your workload that can be configured, it has built-in scaling strategies to optimize performance, costs, or a balance between the two, and provides predictive scaling to assist with regularly occurring spikes.

There are multiple scaling options available to scale your Auto Scaling group:

- Maintain current instance levels at all times
- Scale manually
- Scale based on a schedule
- Scale based on demand
- Use predictive scaling

Auto Scaling policies differ and can be categorized as dynamic and scheduled scaling policies. Dynamic policies are manual or dynamic scaling which, scheduled or predictive scaling. You can use scaling policies for dynamic, scheduled, and predictive scaling. You can also use metrics and alarms from [Amazon CloudWatch](#) to trigger scaling events for your workload. We recommend you use [launch templates](#), which allow you to access the latest features and improvements. Not all Auto Scaling features are available when you use launch configurations. For example, you cannot create an Auto Scaling group that launches both Spot and On-Demand Instances or that specifies multiple instance types. You must use a launch template to configure these features. When using launch templates, we recommended you version each one. With versioning of launch templates, you can create a subset of the full set of parameters. Then, you can reuse it to create other versions of the same launch template.

You can use AWS Auto Scaling or incorporate scaling in your code with [AWS APIs or SDKs](#). This reduces your overall workload costs by removing the operational cost from manually making changes to your environment, and changes can be performed much faster. This also matches your workload resourcing to your demand at any time. In order to follow this best practice and supply resources dynamically for your organization, you should understand horizontal and vertical scaling in the AWS Cloud, as well as the nature of the applications running on Amazon EC2 instances. It is better for your Cloud Financial Management team to work with technical teams to follow this best practice.

[Elastic Load Balancing \(Elastic Load Balancing\)](#) helps you scale by distributing demand across multiple resources. By using ASG and Elastic Load Balancing, you can manage incoming requests by optimally routing traffic so that no one instance is overwhelmed in an Auto Scaling group. The requests would be distributed among all the targets of a target group in a round-robin fashion without consideration for capacity or utilization.

Typical metrics can be standard Amazon EC2 metrics, such as CPU utilization, network throughput, and Elastic Load Balancing observed request and response latency. When possible, you should use a metric that is indicative of customer experience, typically a custom metric that might originate from application code within your workload. To elaborate how to meet the demand dynamically in this document, we will group Auto Scaling into two categories as demand-based and time-based supply models and deep dive into each.

Demand-based supply: Take advantage of elasticity of the cloud to supply resources to meet changing demand by relying on near real-time demand state. For demand-based supply, use APIs or service features to programmatically vary the amount of cloud resources in your architecture. This allows you to scale components in your architecture and increase the number of resources

during demand spikes to maintain performance and decrease capacity when demand subsides to reduce costs.

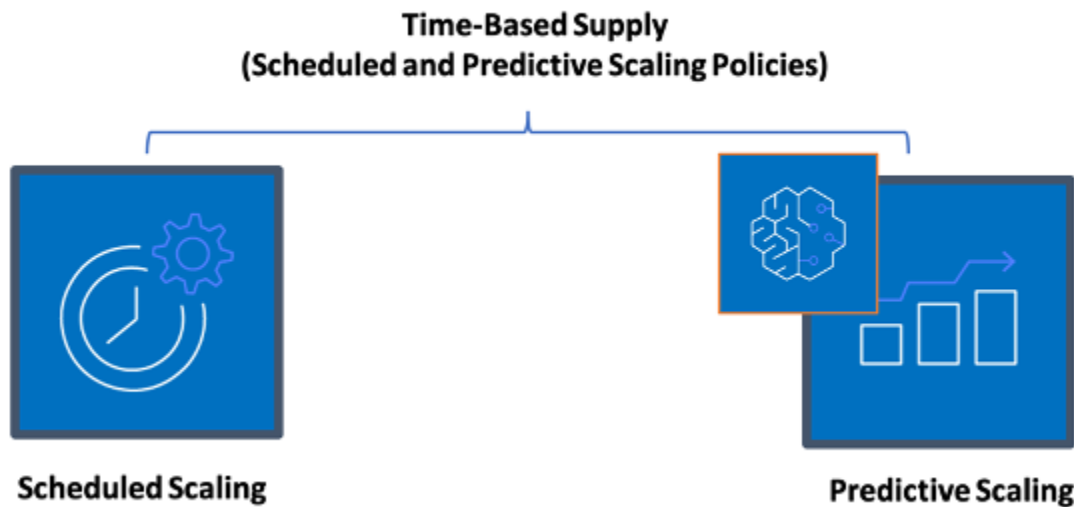


Demand-based dynamic scaling policies

- **Simple/Step Scaling:** Monitors metrics and adds/removes instances as per steps defined by the customers manually.
- **Target Tracking:** Thermostat-like control mechanism that automatically adds or removes instances to maintain metrics at a customer defined target.

When architecting with a demand-based approach keep in mind two key considerations. First, understand how quickly you must provision new resources. Second, understand that the size of margin between supply and demand will shift. You must be ready to cope with the rate of change in demand and also be ready for resource failures.

Time-based supply: A time-based approach aligns resource capacity to demand that is predictable or well-defined by time. This approach is typically not dependent upon utilization levels of the resources. A time-based approach ensures that resources are available at the specific time they are required and can be provided without any delays due to start-up procedures and system or consistency checks. Using a time-based approach, you can provide additional resources or increase capacity during busy periods.



Time-based scaling policies

You can use scheduled or predictive auto scaling to implement a time-based approach. Workloads can be scheduled to scale out or in at defined times (for example, the start of business hours), making resources available when users arrive or demand increases. Predictive scaling uses patterns to scale out while scheduled scaling uses pre-defined times to scale out. You can also use [attribute-based instance type selection \(ABS\) strategy](#) in Auto Scaling groups, which lets you express your instance requirements as a set of attributes, such as vCPU, memory, and storage. This also allows you to automatically use newer generation instance types when they are released and access a broader range of capacity with Amazon EC2 Spot Instances. Amazon EC2 Fleet and Amazon EC2 Auto Scaling select and launch instances that fit the specified attributes, removing the need to manually pick instance types.

You can also leverage the [AWS APIs and SDKs](#) and [AWS CloudFormation](#) to automatically provision and decommission entire environments as you need them. This approach is well suited for development or test environments that run only in defined business hours or periods of time. You can use APIs to scale the size of resources within an environment (vertical scaling). For example, you could scale up a production workload by changing the instance size or class. This can be achieved by stopping and starting the instance and selecting the different instance size or class. This technique can also be applied to other resources, such as Amazon EBS Elastic Volumes, which can be modified to increase size, adjust performance (IOPS) or change the volume type while in use.

When architecting with a time-based approach keep in mind two key considerations. First, how consistent is the usage pattern? Second, what is the impact if the pattern changes? You can increase the accuracy of predictions by monitoring your workloads and by using business intelligence. If you see significant changes in the usage pattern, you can adjust the times to ensure that coverage is provided.

Implementation steps

- **Configure scheduled scaling:** For predictable changes in demand, time-based scaling can provide the correct number of resources in a timely manner. It is also useful if resource creation and configuration is not fast enough to respond to changes on demand. Using the workload analysis configure scheduled scaling using AWS Auto Scaling. To configure time-based scheduling, you can use predictive scaling of scheduled scaling to increase the number of Amazon EC2 instances in your Auto Scaling groups in advance according to expected or predictable load changes.
- **Configure predictive scaling:** Predictive scaling allows you to increase the number of Amazon EC2 instances in your Auto Scaling group in advance of daily and weekly patterns in traffic flows. If you have regular traffic spikes and applications that take a long time to start, you should consider using predictive scaling. Predictive scaling can help you scale faster by initializing capacity before projected load compared to dynamic scaling alone, which is reactive in nature. For example, if users start using your workload with the start of the business hours and don't use after hours, then predictive scaling can add capacity before the business hours which eliminates delay of dynamic scaling to react to changing traffic.
- **Configure dynamic automatic scaling:** To configure scaling based on active workload metrics, use Auto Scaling. Use the analysis and configure Auto Scaling to launch on the correct resource levels, and verify that the workload scales in the required time. You can launch and automatically scale a fleet of On-Demand Instances and Spot Instances within a single Auto Scaling group. In addition to receiving discounts for using Spot Instances, you can use Reserved Instances or a Savings Plan to receive discounted rates of the regular On-Demand Instance pricing. All of these factors combined help you to optimize your cost savings for Amazon EC2 instances and help you get the desired scale and performance for your application.

Resources

Related documents:

- [AWS Auto Scaling](#)

- [AWS Instance Scheduler](#)
- Scale the size of your Auto Scaling group
- [Getting Started with Amazon EC2 Auto Scaling](#)
- [Getting started with Amazon SQS](#)
- [Scheduled Scaling for Amazon EC2 Auto Scaling](#)
- [Predictive scaling for Amazon EC2 Auto Scaling](#)

Related videos:

- [Target Tracking Scaling Policies for Auto Scaling](#)
- [AWS Instance Scheduler](#)

Related examples:

- [Attribute based Instance Type Selection for Auto Scaling for Amazon EC2 Fleet](#)
- [Optimizing Amazon Elastic Container Service for cost using scheduled scaling](#)
- [Predictive Scaling with Amazon EC2 Auto Scaling](#)
- [How do I use Instance Scheduler with AWS CloudFormation to schedule Amazon EC2 instances?](#)

Optimize over time

Questions

- [COST 10. How do you evaluate new services?](#)
- [COST 11. How do you evaluate the cost of effort?](#)

COST 10. How do you evaluate new services?

As AWS releases new services and features, it's a best practice to review your existing architectural decisions to verify they continue to be the most cost effective.

Best practices

- [COST10-BP01 Develop a workload review process](#)
- [COST10-BP02 Review and analyze this workload regularly](#)

COST10-BP01 Develop a workload review process

Develop a process that defines the criteria and process for workload review. The review effort should reflect potential benefit. For example, core workloads or workloads with a value of over ten percent of the bill are reviewed quarterly or every six months, while workloads below ten percent are reviewed annually.

Level of risk exposed if this best practice is not established: High

Implementation guidance

To have the most cost-efficient workload, you must regularly review the workload to know if there are opportunities to implement new services, features, and components. To achieve overall lower costs the process must be proportional to the potential amount of savings. For example, workloads that are 50% of your overall spend should be reviewed more regularly, and more thoroughly, than workloads that are five percent of your overall spend. Factor in any external factors or volatility. If the workload services a specific geography or market segment, and change in that area is predicted, more frequent reviews could lead to cost savings. Another factor in review is the effort to implement changes. If there are significant costs in testing and validating changes, reviews should be less frequent.

Factor in the long-term cost of maintaining outdated and legacy, components and resources and the inability to implement new features into them. The current cost of testing and validation may exceed the proposed benefit. However, over time, the cost of making the change may significantly increase as the gap between the workload and the current technologies increases, resulting in even larger costs. For example, the cost of moving to a new programming language may not currently be cost effective. However, in five years time, the cost of people skilled in that language may increase, and due to workload growth, you would be moving an even larger system to the new language, requiring even more effort than previously.

Break down your workload into components, assign the cost of the component (an estimate is sufficient), and then list the factors (for example, effort and external markets) next to each component. Use these indicators to determine a review frequency for each workload. For example, you may have web servers as a high cost, low change effort, and high external factors, resulting in high frequency of review. A central database may be medium cost, high change effort, and low external factors, resulting in a medium frequency of review.

Define a process to evaluate new services, design patterns, resource types, and configurations to optimize your workload cost as they become available. Similar to [performance pillar review](#) and

[reliability pillar review](#) processes, identify, validate, and prioritize optimization and improvement activities and issue remediation and incorporate this into your backlog.

Implementation steps

- **Define review frequency:** Define how frequently the workload and its components should be reviewed. Allocate time and resources to continual improvement and review frequency to improve the efficiency and optimization of your workload. This is a combination of factors and may differ from workload to workload within your organization and between components in the workload. Common factors include the importance to the organization measured in terms of revenue or brand, the total cost of running the workload (including operation and resource costs), the complexity of the workload, how easy is it to implement a change, any software licensing agreements, and if a change would incur significant increases in licensing costs due to punitive licensing. Components can be defined functionally or technically, such as web servers and databases, or compute and storage resources. Balance the factors accordingly and develop a period for the workload and its components. You may decide to review the full workload every 18 months, review the web servers every six months, the database every 12 months, compute and short-term storage every six months, and long-term storage every 12 months.
- **Define review thoroughness:** Define how much effort is spent on the review of the workload or workload components. Similar to the review frequency, this is a balance of multiple factors. Evaluate and prioritize opportunities for improvement to focus efforts where they provide the greatest benefits while estimating how much effort is required for these activities. If the expected outcomes do not satisfy the goals, and required effort costs more, then iterate using alternative courses of action. Your review processes should include dedicated time and resources to make continuous incremental improvements possible. As an example, you may decide to spend one week of analysis on the database component, one week of analysis for compute resources, and four hours for storage reviews.

Resources

Related documents:

- [AWS News Blog](#)
- [Types of Cloud Computing](#)
- [What's New with AWS](#)

Related examples:

- [AWS Support Proactive Services](#)
- [Regular workload reviews for SAP workloads](#)

COST10-BP02 Review and analyze this workload regularly

Existing workloads are regularly reviewed based on each defined process to find out if new services can be adopted, existing services can be replaced, or workloads can be re-architected.

Level of risk exposed if this best practice is not established: Medium

Implementation guidance

AWS is constantly adding new features so you can experiment and innovate faster with the latest technology. [AWS What's New](#) details how AWS is doing this and provides a quick overview of AWS services, features, and Regional expansion announcements as they are released. You can dive deeper into the launches that have been announced and use them for your review and analyze of your existing workloads. To realize the benefits of new AWS services and features, you review on your workloads and implement new services and features as required. This means you may need to replace existing services you use for your workload, or modernize your workload to adopt these new AWS services. For example, you might review your workloads and replace the messaging component with Amazon Simple Email Service. This removes the cost of operating and maintaining a fleet of instances, while providing all the functionality at a reduced cost.

To analyze your workload and highlight potential opportunities, you should consider not only new services but also new ways of building solutions. Review the [This is My Architecture](#) videos on AWS to learn about other customers' architecture designs, their challenges and their solutions. Check the [All-In series](#) to find out real world applications of AWS services and customer stories. You can also watch the [Back to Basics](#) video series that explains, examines, and breaks down basic cloud architecture pattern best practices. Another source is [How to Build This](#) videos, which are designed to assist people with big ideas on how to bring their minimum viable product (MVP) to life using AWS services. It is a way for builders from all over the world who have a strong idea to gain architectural guidance from experienced AWS Solutions Architects. Finally, you can review the [Getting Started](#) resource materials, which has step by step tutorials.

Before starting your review process, follow your business' requirements for the workload, security and data privacy requirements in order to use specific service or Region and performance requirements while following your agreed review process.

Implementation steps

- **Regularly review the workload:** Using your defined process, perform reviews with the frequency specified. Verify that you spend the correct amount of effort on each component. This process would be similar to the initial design process where you selected services for cost optimization. Analyze the services and the benefits they would bring, this time factor in the cost of making the change, not just the long-term benefits.
- **Implement new services:** If the outcome of the analysis is to implement changes, first perform a baseline of the workload to know the current cost for each output. Implement the changes, then perform an analysis to confirm the new cost for each output.

Resources

Related documents:

- [AWS News Blog](#)
- [What's New with AWS](#)
- [AWS Documentation](#)
- [AWS Getting Started](#)
- [AWS General Resources](#)

Related videos:

- [AWS - This is My Architecture](#)
- [AWS - Back to Basics](#)
- [AWS - All-In series](#)
- [How to Build This](#)

COST 11. How do you evaluate the cost of effort?

Best practices

- [COST11-BP01 Perform automation for operations](#)

COST11-BP01 Perform automation for operations

Evaluate the operational costs on the cloud, focusing on quantifying the time and effort savings in administrative tasks, deployments, mitigating the risk of human errors, compliance, and other

operations through automation. Assess the time and associated costs required for operational efforts and implement automation for administrative tasks to minimize manual effort wherever feasible.

Level of risk exposed if this best practice is not established: Low

Implementation guidance

Automating operations reduces the frequency of manual tasks, improves efficiency, and benefits customers by delivering a consistent and reliable experience when deploying, administering, or operating workloads. You can free up infrastructure resources from manual operational tasks and use them for higher value tasks and innovations, which improves business value. Enterprises require a proven, tested way to manage their workloads in the cloud. That solution must be secure, fast, and cost effective, with minimum risk and maximum reliability.

Start by prioritizing your operational activities based on required effort by looking at overall operations cost. For example, how long does it take to deploy new resources in the cloud, make optimization changes to existing ones, or implement necessary configurations? Look at the total cost of human actions by factoring in cost of operations and management. Prioritize automations for admin tasks to reduce the human effort.

Review effort should reflect the potential benefit. For example, examine time spent performing tasks manually as opposed to automatically. Prioritize automating repetitive, high value, time consuming and complex activities. Activities that pose a high value or high risk of human error are typically the better place to start automating as the risk often poses an unwanted additional operational cost (like operations team working extra hours).

Use automation tools like AWS Systems Manager or AWS Config to streamline operations, compliance, monitoring, lifecycle, and termination processes. With AWS services, tools, and third-party products, you can customize the automations you implement to meet your specific requirement. Following table shows some of the core operation functions and capabilities you can achieve with AWS services to automate administration and operation:

- [AWS Audit Manager](#): Continually audit your AWS usage to simplify risk and compliance assessment
- [AWS Backup](#): Centrally manage and automate data protection.
- [AWS Config](#): Configure compute resources, assess, audit, evaluate configurations and resource inventory.
- [AWS CloudFormation](#): Launch highly available resources with Infrastructure as Code.

- [AWS CloudTrail](#): IT change management, compliance, and control.
- [Amazon EventBridge](#): Schedule events and trigger AWS Lambda to take action.
- [AWS Lambda](#): Automate repetitive processes by triggering them with events or by running them on a fixed schedule with AWS EventBridge.
- [AWS Systems Manager](#): Start and stop workloads, patch operating systems, automate configuration, and ongoing management.
- [AWS Step Functions](#): Schedule jobs and automate workflows.
- [AWS Service Catalog](#): Template consumption, infrastructure as code with compliance and control.

If you would like to adopt automations immediately by using AWS products and service and if don't have skills in your organization, reach out to [AWS Managed Services \(AMS\)](#), [AWS Professional Services](#), or [AWS Partners](#) to increase adoption of automation and improve your operational excellence in the cloud.

AWS Managed Services (AMS) is a service that operates AWS infrastructure on behalf of enterprise customers and partners. It provides a secure and compliant environment that you can deploy your workloads onto. AMS uses enterprise cloud operating models with automation to allow you to meet your organization requirements, move into the cloud faster, and reduce your on-going management costs.

AWS Professional Services can also help you achieve your desired business outcomes and automate operations with AWS. They help customers to deploy automated, robust, agile IT operations, and governance capabilities optimized for the cloud. For detailed monitoring examples and recommended best practices, see Operational Excellence Pillar whitepaper.

Implementation steps

- **Build once and deploy many:** Use infrastructure-as-code such as CloudFormation, AWS SDK, or AWS CLI to deploy once and use many times for similar environments or for disaster recovery scenarios. Tag while deploying to track your consumption as defined in other best practices. Use [AWS Launch Wizard](#) to reduce the time to deploy many popular enterprise workloads. AWS Launch Wizard guides you through the sizing, configuration, and deployment of enterprise workloads following AWS best practices. You can also use the [Service Catalog](#), which helps you create and manage infrastructure-as-code approved templates for use on AWS so anyone can discover approved, self-service cloud resources.
- **Automate continuous compliance:** Consider automating assessment and remediation of recorded configurations against predefined standards. When you combine AWS Organizations

with the capabilities of AWS Config and [AWS CloudFormation](#), you can efficiently manage and automate configuration compliance at scale for hundreds of member accounts. You can review changes in configurations and relationships between AWS resources and dive into the history of a resource configuration.

- **Automate monitoring tasks** AWS provides various tools that you can use to monitor services. You can configure these tools to automate monitoring tasks. Create and implement a monitoring plan that collects monitoring data from all the parts in your workload so that you can more easily debug a multi-point failure if one occurs. For example, you can use the automated monitoring tools to observe Amazon EC2 and report back to you when something is wrong for system status checks, instance status checks, and Amazon CloudWatch alarms.
- **Automate maintenance and operations:** Run routine operations automatically without human intervention. Using AWS services and tools, you can choose which AWS automations to implement and customize for your specific requirements. For example, use [EC2 Image Builder](#) for building, testing, and deployment of virtual machine and container images for use on AWS or on-premises or patching your EC2 instances with AWS SSM. If your desired action cannot be done with AWS services or you need more complex actions with filtering resources, then automate your operations by using [AWS Command Line Interface](#) (AWS CLI) or AWS SDK tools. AWS CLI provides the ability to automate the entire process of controlling and managing AWS services with scripts without using the AWS Management Console. Select your preferred AWS SDKs to interact with AWS services. For other code examples, see AWS SDK Code [examples repository](#).
- **Create a continual lifecycle with automations:** It is important that you establish and preserve mature lifecycle policies not only for regulations or redundancy but also for cost optimization. You can use AWS Backup to centrally manage and automate data protection of data stores, such as your buckets, volumes, databases, and file systems. You can also use Amazon Data Lifecycle Manager to automate the creation, retention, and deletion of EBS snapshots and EBS-backed AMIs.
- **Delete unnecessary resources:** It's quite common to accumulate unused resources in sandbox or development AWS accounts. Developers create and experiment with various services and resources as part of the normal development cycle, and then they don't delete those resources when they're no longer needed. Unused resources can incur unnecessary and sometimes high costs for the organization. Deleting these resources can reduce the costs of operating these environments. Make sure your data is not needed or backed up if you are not sure. You can use AWS CloudFormation to clean up deployed stacks, which automatically deletes most resources defined in the template. Alternatively, you can create an automation for the deletion of AWS resources using tools like [aws-nuke](#).

Resources

Related documents:

- [Modernizing operations in the AWS Cloud](#)
- [AWS Services for Automation](#)
- [Infrastructure and automation](#)
- [AWS Systems Manager Automation](#)
- [Automated and manual monitoring](#)
- [AWS automations for SAP administration and operations](#)
- [AWS Managed Services](#)
- [AWS Professional Services](#)

Related videos:

- [Automate Continuous Compliance at Scale in AWS](#)
- [AWS Backup Demo: Cross-Account & Cross-Region Backup](#)
- [Patching for your Amazon EC2 Instances](#)

Related examples:

- [Reinventing automated operations \(Part I\)](#)
- [Reinventing automated operations \(Part II\)](#)
- [Automate deletion of AWS resources by using aws-nuke](#)
- [Delete unused Amazon EBS volumes by using AWS Config and AWS SSM](#)
- [Automate continuous compliance at scale in AWS](#)
- [IT Automations with AWS Lambda](#)

Sustainability

The Sustainability pillar includes understanding the impacts of the services used, quantifying impacts through the entire workload lifecycle, and applying design principles and best practices to reduce these impacts when building cloud workloads. You can find prescriptive guidance on implementation in the [Sustainability Pillar whitepaper](#).

Best practice areas

- [Region selection](#)
- [Alignment to demand](#)
- [Software and architecture](#)
- [Data](#)
- [Hardware and services](#)
- [Process and culture](#)

Region selection

Question

- [SUS 1 How do you select Regions for your workload?](#)

SUS 1 How do you select Regions for your workload?

The choice of Region for your workload significantly affects its KPIs, including performance, cost, and carbon footprint. To effectively improve these KPIs, you should choose Regions for your workloads based on both business requirements and sustainability goals.

Best practices

- [SUS01-BP01 Choose Region based on both business requirements and sustainability goals](#)

SUS01-BP01 Choose Region based on both business requirements and sustainability goals

Choose a Region for your workload based on both your business requirements and sustainability goals to optimize its KPIs, including performance, cost, and carbon footprint.

Common anti-patterns:

- You select the workload's Region based on your own location.
- You consolidate all workload resources into one geographic location.

Benefits of establishing this best practice: Placing a workload close to Amazon renewable energy projects or Regions with low published carbon intensity can help to lower the carbon footprint of a cloud workload.

Level of risk exposed if this best practice is not established: Medium**Implementation guidance**

The AWS Cloud is a constantly expanding network of Regions and points of presence (PoP), with a global network infrastructure linking them together. The choice of Region for your workload significantly affects its KPIs, including performance, cost, and carbon footprint. To effectively improve these KPIs, you should choose Regions for your workload based on both your business requirements and sustainability goals.

Implementation steps

- **Shortlist potential Regions:** Follow these steps to assess and shortlist potential Regions for your workload based on your business requirements, including compliance, available features, cost, and latency:
 - Confirm that these Regions are compliant, based on your required local regulations (for example, data sovereignty).
 - Use the [AWS Regional Services Lists](#) to check if the Regions have the services and features you need to run your workload.
 - Calculate the cost of the workload on each Region using the [AWS Pricing Calculator](#).
 - Test the network latency between your end user locations and each AWS Region.
- **Choose Regions:** Choose Regions near Amazon renewable energy projects and Regions where the grid has a published carbon intensity that is lower than other locations (or Regions).
 - Identify your relevant sustainability guidelines to track and compare year-to-year carbon emissions based on [Greenhouse Gas Protocol](#) (market-based and location based methods).
 - Choose region based on method you use to track carbon emissions. For more detail on choosing a Region based on your sustainability guidelines, see [How to select a Region for your workload based on sustainability goals](#).

Resources**Related documents:**

- [Understanding your carbon emission estimations](#)
- [Amazon Around the Globe](#)
- [Renewable Energy Methodology](#)
- [What to Consider when Selecting a Region for your Workloads](#)

Related videos:

- [AWS re:Invent 2023 - Sustainability innovation in AWS Global Infrastructure](#)
- [AWS re:Invent 2023 - Sustainable architecture: Past, present, and future](#)
- [AWS re:Invent 2022 - Delivering sustainable, high-performing architectures](#)
- [AWS re:Invent 2022 - Architecting sustainably and reducing your AWS carbon footprint](#)
- [AWS re:Invent 2022 - Sustainability in AWS global infrastructure](#)

Alignment to demand

Question

- [SUS 2 How do you align cloud resources to your demand?](#)

SUS 2 How do you align cloud resources to your demand?

The way users and applications consume your workloads and other resources can help you identify improvements to meet sustainability goals. Scale infrastructure to continually match demand and verify that you use only the minimum resources required to support your users. Align service levels to customer needs. Position resources to limit the network required for users and applications to consume them. Remove unused assets. Provide your team members with devices that support their needs and minimize their sustainability impact.

Best practices

- [SUS02-BP01 Scale workload infrastructure dynamically](#)
- [SUS02-BP02 Align SLAs with sustainability goals](#)
- [SUS02-BP03 Stop the creation and maintenance of unused assets](#)
- [SUS02-BP04 Optimize geographic placement of workloads based on their networking requirements](#)
- [SUS02-BP05 Optimize team member resources for activities performed](#)
- [SUS02-BP06 Implement buffering or throttling to flatten the demand curve](#)

SUS02-BP01 Scale workload infrastructure dynamically

Use elasticity of the cloud and scale your infrastructure dynamically to match supply of cloud resources to demand and avoid overprovisioned capacity in your workload.

Common anti-patterns:

- You do not scale your infrastructure with user load.
- You manually scale your infrastructure all the time.
- You leave increased capacity after a scaling event instead of scaling back down.

Benefits of establishing this best practice: Configuring and testing workload elasticity help to efficiently match supply of cloud resources to demand and avoid overprovisioned capacity. You can take advantage of elasticity in the cloud to automatically scale capacity during and after demand spikes to make sure you are only using the right number of resources needed to meet your business requirements.

Level of risk exposed if this best practice is not established: Medium

Implementation guidance

The cloud provides the flexibility to expand or reduce your resources dynamically through a variety of mechanisms to meet changes in demand. Optimally matching supply to demand delivers the lowest environmental impact for a workload.

Demand can be fixed or variable, requiring metrics and automation to make sure that management does not become burdensome. Applications can scale vertically (up or down) by modifying the instance size, horizontally (in or out) by modifying the number of instances, or a combination of both.

You can use a number of different approaches to match supply of resources with demand.

- **Target-tracking approach:** Monitor your scaling metric and automatically increase or decrease capacity as you need it.
- **Predictive scaling:** Scale in anticipation of daily and weekly trends.
- **Schedule-based approach:** Set your own scaling schedule according to predictable load changes.
- **Service scaling:** Pick services (like serverless) that are natively scaling by design or provide auto scaling as a feature.

Identify periods of low or no utilization and scale resources to remove excess capacity and improve efficiency.

Implementation steps

- Elasticity matches the supply of resources you have against the demand for those resources. Instances, containers, and functions provide mechanisms for elasticity, either in combination with automatic scaling or as a feature of the service. AWS provides a range of auto scaling mechanisms to ensure that workloads can scale down quickly and easily during periods of low user load. Here are some examples of auto scaling mechanisms:

Auto scaling mechanism	Where to use
Amazon EC2 Auto Scaling	Use to verify you have the correct number of Amazon EC2 instances available to handle the user load for your application.
Application Auto Scaling	Use to automatically scale the resources for individual AWS services beyond Amazon EC2, such as Lambda functions or Amazon Elastic Container Service (Amazon ECS) services.
Kubernetes Cluster Autoscaler	Use to automatically scale Kubernetes clusters on AWS.

- Scaling is often discussed related to compute services like Amazon EC2 instances or AWS Lambda functions. Consider the configuration of non-compute services like [Amazon DynamoDB](#) read and write capacity units or [Amazon Kinesis Data Streams](#) shards to match the demand.
- Verify that the metrics for scaling up or down are validated against the type of workload being deployed. If you are deploying a video transcoding application, 100% CPU utilization is expected and should not be your primary metric. You can use a [customized metric](#) (such as memory utilization) for your scaling policy if required. To choose the right metrics, consider the following guidance for Amazon EC2:
 - The metric should be a valid utilization metric and describe how busy an instance is.
 - The metric value must increase or decrease proportionally to the number of instances in the Auto Scaling group.
- Use [dynamic scaling](#) instead of [manual scaling](#) for your Auto Scaling group. We also recommend that you use [target tracking scaling policies](#) in your dynamic scaling.
- Verify that workload deployments can handle both scale-out and scale-in events. Create test scenarios for scale-in events to verify that the workload behaves as expected and does not affect

the user experience (like losing sticky sessions). You can use [Activity history](#) to verify a scaling activity for an Auto Scaling group.

- Evaluate your workload for predictable patterns and proactively scale as you anticipate predicted and planned changes in demand. With predictive scaling, you can eliminate the need to overprovision capacity. For more detail, see [Predictive Scaling with Amazon EC2 Auto Scaling](#).

Resources

Related documents:

- [Getting Started with Amazon EC2 Auto Scaling](#)
- [Predictive Scaling for EC2, Powered by Machine Learning](#)
- [Analyze user behavior using Amazon OpenSearch Service, Amazon Data Firehose and Kibana](#)
- [What is Amazon CloudWatch?](#)
- [Monitoring DB load with Performance Insights on Amazon RDS](#)
- [Introducing Native Support for Predictive Scaling with Amazon EC2 Auto Scaling](#)
- [Introducing Karpenter - An Open-Source, High-Performance Kubernetes Cluster Autoscaler](#)
- [Deep Dive on Amazon ECS Cluster Auto Scaling](#)

Related videos:

- [AWS re:Invent 2023 - Scaling on AWS for the first 10 million users](#)
- [AWS re:Invent 2023 - Sustainable architecture: Past, present, and future](#)
- [AWS re:Invent 2022 - Build a cost-, energy-, and resource-efficient compute environment](#)
- [AWS re:Invent 2022 - Scaling containers from one user to millions](#)
- [AWS re:Invent 2023 - Scaling FM inference to hundreds of models with Amazon SageMaker AI](#)
- [AWS re:Invent 2023 - Harness the power of Karpenter to scale, optimize & upgrade Kubernetes](#)

Related examples:

- [Autoscaling](#)

SUS02-BP02 Align SLAs with sustainability goals

Review and optimize workload service-level agreements (SLA) based on your sustainability goals to minimize the resources required to support your workload while continuing to meet business needs.

Common anti-patterns:

- Workload SLAs are unknown or ambiguous.
- You define your SLA just for availability and performance.
- You use the same design pattern (like Multi-AZ architecture) for all your workloads.

Benefits of establishing this best practice: Aligning SLAs with sustainability goals leads to optimal resource usage while meeting business needs.

Level of risk exposed if this best practice is not established: Low

Implementation guidance

SLAs define the level of service expected from a cloud workload, such as response time, availability, and data retention. They influence the architecture, resource usage, and environmental impact of a cloud workload. At a regular cadence, review SLAs and make trade-offs that significantly reduce resource usage in exchange for acceptable decreases in service levels.

Implementation steps

- **Understand sustainability goals:** Identify sustainability goals in your organization, such as carbon reduction or improving resource utilization.
- **Review SLAs:** Evaluate your SLAs to assess if they support your business requirements. If you are exceeding SLAs, perform further review.
- **Understand trade-offs:** Understand the trade-offs across your workload's complexity (like high volume of concurrent users), performance (like latency), and sustainability impact (like required resources). Typically, prioritizing two of the factors comes at the expense of the third.
- **Adjust SLAs:** Adjust your SLAs by making trade-offs that significantly reduce sustainability impacts in exchange for acceptable decreases in service levels.
 - **Sustainability and reliability:** Highly available workloads tend to consume more resources.
 - **Sustainability and performance:** Using more resources to boost performance could have a higher environmental impact.

- **Sustainability and security:** Overly secure workloads could have a higher environmental impact.
- **Define sustainability SLAs if possible:** Include sustainability SLAs for your workload. For example, define a minimum utilization level as a sustainability SLA for your compute instances.
- **Use efficient design patterns:** Use design patterns such as microservices on AWS that prioritize business-critical functions and allow lower service levels (such as response time or recovery time objectives) for non-critical functions.
- **Communicate and establish accountability:** Share the SLAs with all relevant stakeholders, including your development team and your customers. Use reporting to track and monitor the SLAs. Assign accountability to meet the sustainability targets for your SLAs.
- **Use incentives and rewards:** Use incentives and rewards to achieve or exceed SLAs aligned with sustainability goals.
- **Review and iterate:** Regularly review and adjust your SLAs to make sure they are aligned with evolving sustainability and performance goals.

Resources

Related documents:

- [Understand resiliency patterns and trade-offs to architect efficiently in the cloud](#)
- [Importance of Service Level Agreement for SaaS Providers](#)

Related videos:

- [AWS re:Invent 2023 - Capacity, availability, cost efficiency: Pick three](#)
- [AWS re:Invent 2023 - Sustainable architecture: Past, present, and future](#)
- [AWS re:Invent 2023 - Advanced integration patterns & trade-offs for loosely coupled systems](#)
- [AWS re:Invent 2022 - Delivering sustainable, high-performing architectures](#)
- [AWS re:Invent 2022 - Build a cost-, energy-, and resource-efficient compute environment](#)

SUS02-BP03 Stop the creation and maintenance of unused assets

Decommission unused assets in your workload to reduce the number of cloud resources required to support your demand and minimize waste.

Common anti-patterns:

- You do not analyze your application for assets that are redundant or no longer required.
- You do not remove assets that are redundant or no longer required.

Benefits of establishing this best practice: Removing unused assets frees resources and improves the overall efficiency of the workload.

Level of risk exposed if this best practice is not established: Low

Implementation guidance

Unused assets consume cloud resources like storage space and compute power. By identifying and eliminating these assets, you can free up these resources, resulting in a more efficient cloud architecture. Perform regular analysis on application assets such as pre-compiled reports, datasets, static images, and asset access patterns to identify redundancy, underutilization, and potential decommission targets. Remove those redundant assets to reduce the resource waste in your workload.

Implementation steps

- **Conduct an inventory:** Conduct a comprehensive inventory to identify all assets within your workload.
- **Analyze usage:** Use continuous monitoring to identify static assets that are no longer required.
- **Remove unused assets:** Develop a plan to remove assets that are no longer required.
 - Before removing any asset, evaluate the impact of removing it on the architecture.
 - Consolidate overlapping generated assets to remove redundant processing.
 - Update your applications to no longer produce and store assets that are not required.
- **Communicate with third parties:** Instruct third parties to stop producing and storing assets managed on your behalf that are no longer required. Ask to consolidate redundant assets.
- **Use lifecycle policies:** Use lifecycle policies to automatically delete unused assets.
 - You can use [Amazon S3 Lifecycle](#) to manage your objects throughout their lifecycle.
 - You can use [Amazon Data Lifecycle Manager](#) to automate the creation, retention, and deletion of Amazon EBS snapshots and Amazon EBS-backed AMIs.
- **Review and optimize:** Regularly review your workload to identify and remove any unused assets.

Resources

Related documents:

- [Optimizing your AWS Infrastructure for Sustainability, Part II: Storage](#)
- [How do I terminate active resources that I no longer need on my AWS account?](#)

Related videos:

- [AWS re:Invent 2023 - Sustainable architecture: Past, present, and future](#)
- [AWS re:Invent 2022 - Preserving and maximizing the value of digital media assets using Amazon S3](#)
- [AWS re:Invent 2023 - Optimize costs in your multi-account environments](#)

SUS02-BP04 Optimize geographic placement of workloads based on their networking requirements

Select cloud location and services for your workload that reduce the distance network traffic must travel and decrease the total network resources required to support your workload.

Common anti-patterns:

- You select the workload's Region based on your own location.
- You consolidate all workload resources into one geographic location.
- All traffic flows through your existing data centers.

Benefits of establishing this best practice: Placing a workload close to its users provides the lowest latency while decreasing data movement across the network and reducing environmental impact.

Level of risk exposed if this best practice is not established: Medium

Implementation guidance

The AWS Cloud infrastructure is built around location options such as Regions, Availability Zones, placement groups, and edge locations such as [AWS Outposts](#) and [AWS Local Zones](#). These location options are responsible for maintaining connectivity between application components, cloud services, edge networks and on-premises data centers.

Analyze the network access patterns in your workload to identify how to use these cloud location options and reduce the distance network traffic must travel.

Implementation steps

- Analyze network access patterns in your workload to identify how users use your application.
 - Use monitoring tools, such as [Amazon CloudWatch](#) and [AWS CloudTrail](#), to gather data on network activities.
 - Analyze the data to identify the network access pattern.
- Select the Regions for your workload deployment based on the following key elements:
 - **Your Sustainability goal:** as explained in [Region selection](#).
 - **Where your data is located:** For data-heavy applications (such as big data and machine learning), application code should run as close to the data as possible.
 - **Where your users are located:** For user-facing applications, choose a Region (or Regions) close to your workload's users.
 - **Other constraints:** Consider constraints such as cost and compliance as explained in [What to Consider when Selecting a Region for your Workloads](#).
- Use local caching or [AWS Caching Solutions](#) for frequently used assets to improve performance, reduce data movement, and lower environmental impact.

Service	When to use
Amazon CloudFront	Use to cache static content such as images, scripts, and videos, as well as dynamic content such as API responses or web applications.
Amazon ElastiCache	Use to cache content for web applications.
DynamoDB Accelerator	Use to add in-memory acceleration to your DynamoDB tables.

- Use services that can help you run code closer to users of your workload:

Service	When to use
Lambda@Edge	Use for compute-heavy operations that are initiated when objects are not in the cache.
Amazon CloudFront Functions	Use for simple use cases like HTTP(s) request or response manipulations that can be initiated by short-lived functions.
AWS IoT Greengrass	Use to run local compute, messaging, and data caching for connected devices.

- Use connection pooling to allow for connection reuse and reduce required resources.
- Use distributed data stores that don't rely on persistent connections and synchronous updates for consistency to serve regional populations.
- Replace pre-provisioned static network capacity with shared dynamic capacity, and share the sustainability impact of network capacity with other subscribers.

Resources

Related documents:

- [Optimizing your AWS Infrastructure for Sustainability, Part III: Networking](#)
- [Amazon ElastiCache Documentation](#)
- [What is Amazon CloudFront?](#)
- [Amazon CloudFront Key Features](#)
- [AWS Global Infrastructure](#)
- [AWS Local Zones and AWS Outposts, choosing the right technology for your edge workload](#)
- [Placement groups](#)
- [AWS Local Zones](#)
- [AWS Outposts](#)

Related videos:

- [Demystifying data transfer on AWS](#)

- [Scaling network performance on next-gen Amazon EC2 instances](#)
- [AWS Local Zones Explainer Video](#)
- [AWS Outposts: Overview and How it Works](#)
- [AWS re:Invent 2023 - A migration strategy for edge and on-premises workloads](#)
- [AWS re:Invent 2021 - AWS Outposts: Bringing the AWS experience on premises](#)
- [AWS re:Invent 2020 - AWS Wavelength: Run apps with ultra-low latency at 5G edge](#)
- [AWS re:Invent 2022 - AWS Local Zones: Building applications for a distributed edge](#)
- [AWS re:Invent 2021 - Building low-latency websites with Amazon CloudFront](#)
- [AWS re:Invent 2022 - Improve performance and availability with AWS Global Accelerator](#)
- [AWS re:Invent 2022 - Build your global wide area network using AWS](#)
- [AWS re:Invent 2020: Global traffic management with Amazon Route 53](#)

Related examples:

- [AWS Networking Workshops](#)
- [Architecting for sustainability - Minimize data movement across networks](#)

SUS02-BP05 Optimize team member resources for activities performed

Optimize resources provided to team members to minimize the environmental sustainability impact while supporting their needs.

Common anti-patterns:

- You ignore the impact of devices used by your team members on the overall efficiency of your cloud application.
- You manually manage and update resources used by team members.

Benefits of establishing this best practice: Optimizing team member resources improves the overall efficiency of cloud-enabled applications.

Level of risk exposed if this best practice is not established: Low

Implementation guidance

Understand the resources your team members use to consume your services, their expected lifecycle, and the financial and sustainability impact. Implement strategies to optimize these resources. For example, perform complex operations, such as rendering and compilation, on highly utilized scalable infrastructure instead of on underutilized high-powered single-user systems.

Implementation steps

- **Use energy-efficient workstations:** Provide team members with energy-efficient workstations and peripherals. Use efficient power management features (like low power mode) in these devices to reduce their energy usage
- **use virtualization:** Use virtual desktops and application streaming to limit upgrade and device requirements.
- **Encourage remote collaboration:** Encourage team members to use remote collaboration tools such as [Amazon Chime](#) or [AWS Wickr](#) to reduce the need for travel and associated carbon emissions.
- **Use energy-efficient software:** Provide team members with energy-efficient software by removing or turning off unnecessary features and processes.
- **Manage lifecycles:** Evaluate the impact of processes and systems on your device lifecycle, and select solutions that minimize the requirement for device replacement while satisfying business requirements. Regularly maintain and update workstations or software to maintain and improve efficiency.
- **Remote device management:** Implement remote management for devices to reduce required business travel.
 - [AWS Systems Manager Fleet Manager](#) is a unified user interface (UI) experience that helps you remotely manage your nodes running on AWS or on premises.

Resources

Related documents:

- [What is Amazon WorkSpaces?](#)
- [Cost Optimizer for Amazon WorkSpaces](#)
- [Amazon AppStream 2.0 Documentation](#)
- [NICE DCV](#)

Related videos:

- [Managing cost for Amazon WorkSpaces on AWS](#)

SUS02-BP06 Implement buffering or throttling to flatten the demand curve

Buffering and throttling flatten the demand curve and reduce the provisioned capacity required for your workload.

Common anti-patterns:

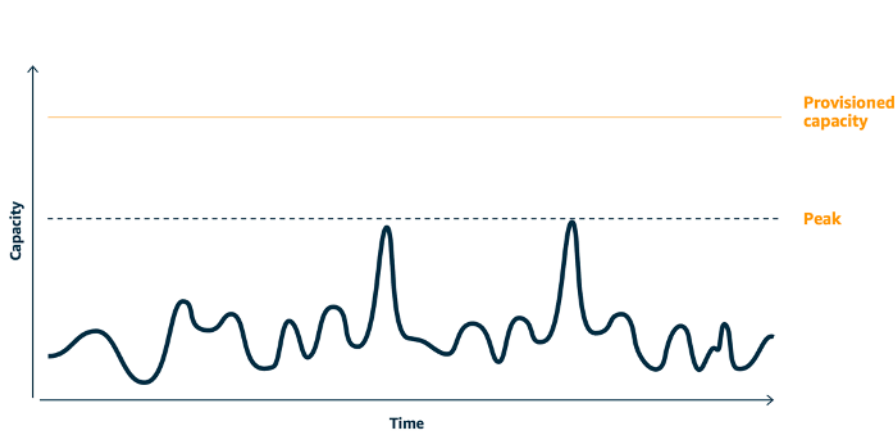
- You process the client requests immediately while it is not needed.
- You do not analyze the requirements for client requests.

Benefits of establishing this best practice: Flattening the demand curve reduce the required provisioned capacity for the workload. Reducing the provisioned capacity means less energy consumption and less environmental impact.

Level of risk exposed if this best practice is not established: Low

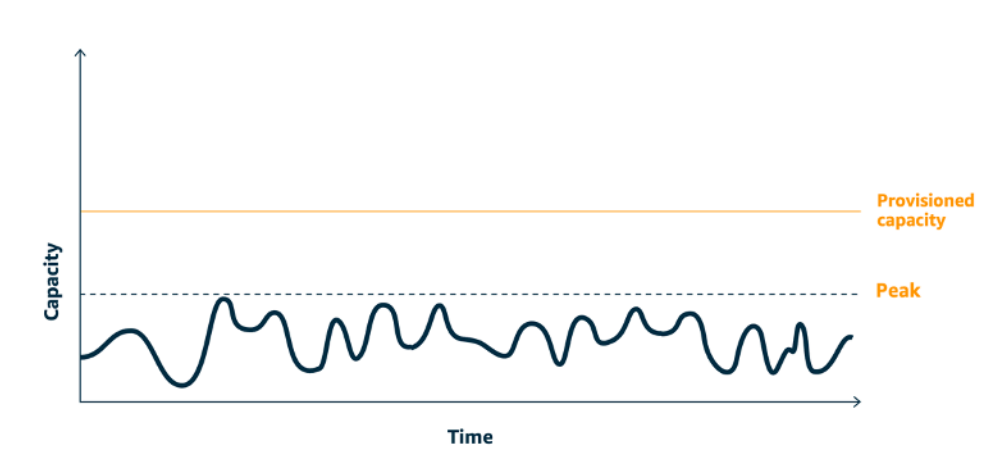
Implementation guidance

Flattening the workload demand curve can help you to reduce the provisioned capacity for a workload and reduce its environmental impact. Assume a workload with the demand curve shown in below figure. This workload has two peaks, and to handle those peaks, the resource capacity as shown by orange line is provisioned. The resources and energy used for this workload is not indicated by the area under the demand curve, but the area under the provisioned capacity line, as provisioned capacity is needed to handle those two peaks.



Demand curve with two distinct peaks that require high provisioned capacity.

You can use buffering or throttling to modify the demand curve and smooth out the peaks, which means less provisioned capacity and less energy consumed. Implement throttling when your clients can perform retries. Implement buffering to store the request and defer processing until a later time.



Throttling's effect on the demand curve and provisioned capacity.

Implementation steps

- Analyze the client requests to determine how to respond to them. Questions to consider include:
 - Can this request be processed asynchronously?
 - Does the client have retry capability?
- If the client has retry capability, then you can implement throttling, which tells the source that if it cannot service the request at the current time, it should try again later.
 - You can use [Amazon API Gateway](#) to implement throttling.
- For clients that cannot perform retries, a buffer needs to be implemented to flatten the demand curve. A buffer defers request processing, allowing applications that run at different rates to communicate effectively. A buffer-based approach uses a queue or a stream to accept messages from producers. Messages are read by consumers and processed, allowing the messages to run at the rate that meets the consumers' business requirements.
 - [Amazon Simple Queue Service \(Amazon SQS\)](#) is a managed service that provides queues that allow a single consumer to read individual messages.
 - [Amazon Kinesis](#) provides a stream that allows many consumers to read the same messages.
- Analyze the overall demand, rate of change, and required response time to right size the throttle or buffer required.

Resources

Related documents:

- [Getting started with Amazon SQS](#)
- [Application integration Using Queues and Messages](#)
- [Managing and monitoring API throttling in your workloads](#)
- [Throttling a tiered, multi-tenant REST API at scale using API Gateway](#)
- [Application integration Using Queues and Messages](#)

Related videos:

- [AWS re:Invent 2022 - Application integration patterns for microservices](#)
- [AWS re:Invent 2023 - Smart savings: Amazon EC2 cost-optimization strategies](#)
- [AWS re:Invent 2023 - Advanced integration patterns & trade-offs for loosely coupled systems](#)

Software and architecture

Question

- [SUS 3 How do you take advantage of software and architecture patterns to support your sustainability goals?](#)

SUS 3 How do you take advantage of software and architecture patterns to support your sustainability goals?

Implement patterns for performing load smoothing and maintaining consistent high utilization of deployed resources to minimize the resources consumed. Components might become idle from lack of use because of changes in user behavior over time. Revise patterns and architecture to consolidate under-utilized components to increase overall utilization. Retire components that are no longer required. Understand the performance of your workload components, and optimize the components that consume the most resources. Be aware of the devices that your customers use to access your services, and implement patterns to minimize the need for device upgrades.

Best practices

- [SUS03-BP01 Optimize software and architecture for asynchronous and scheduled jobs](#)

- [SUS03-BP02 Remove or refactor workload components with low or no use](#)
- [SUS03-BP03 Optimize areas of code that consume the most time or resources](#)
- [SUS03-BP04 Optimize impact on devices and equipment](#)
- [SUS03-BP05 Use software patterns and architectures that best support data access and storage patterns](#)

SUS03-BP01 Optimize software and architecture for asynchronous and scheduled jobs

Use efficient software and architecture patterns such as queue-driven to maintain consistent high utilization of deployed resources.

Common anti-patterns:

- You overprovision the resources in your cloud workload to meet unforeseen spikes in demand.
- Your architecture does not decouple senders and receivers of asynchronous messages by a messaging component.

Benefits of establishing this best practice:

- Efficient software and architecture patterns minimize the unused resources in your workload and improve the overall efficiency.
- You can scale the processing independently of the receiving of asynchronous messages.
- Through a messaging component, you have relaxed availability requirements that you can meet with fewer resources.

Level of risk exposed if this best practice is not established: Medium

Implementation guidance

Use efficient architecture patterns such as [event-driven architecture](#) that result in even utilization of components and minimize overprovisioning in your workload. Using efficient architecture patterns minimizes idle resources from lack of use due to changes in demand over time.

Understand the requirements of your workload components and adopt architecture patterns that increase overall utilization of resources. Retire components that are no longer required.

Implementation steps

- Analyze the demand for your workload to determine how to respond to those.
- For requests or jobs that don't require synchronous responses, use queue-driven architectures and auto scaling workers to maximize utilization. Here are some examples of when you might consider queue-driven architecture:

Queuing mechanism	Description
AWS Batch job queues	AWS Batch jobs are submitted to a job queue where they reside until they can be scheduled to run in a compute environment.
Amazon Simple Queue Service and Amazon EC2 Spot Instances	Pairing Amazon SQS and Spot Instances to build fault tolerant and efficient architecture.

- For requests or jobs that can be processed anytime, use scheduling mechanisms to process jobs in batch for more efficiency. Here are some examples of scheduling mechanisms on AWS:

Scheduling mechanism	Description
Amazon EventBridge Scheduler	A capability from Amazon EventBridge that allows you to create, run, and manage scheduled tasks at scale.
AWS Glue time-based schedule	Define a time-based schedule for your crawlers and jobs in AWS Glue.
Amazon Elastic Container Service (Amazon ECS) scheduled tasks	Amazon ECS supports creating scheduled tasks. Scheduled tasks use Amazon EventBridge rules to run tasks either on a schedule or in a response to an EventBridge event.
Instance Scheduler	Configure start and stop schedules for your Amazon EC2 and Amazon Relational Database Service instances.

- If you use polling and webhooks mechanisms in your architecture, replace those with events. Use [event-driven architectures](#) to build highly efficient workloads.

- Leverage [serverless on AWS](#) to eliminate over-provisioned infrastructure.
- Right size individual components of your architecture to prevent idling resources waiting for input.
 - You can use the [Rightsizing Recommendations in AWS Cost Explorer](#) or [AWS Compute Optimizer](#) to identify rightsizing opportunities.
 - For more detail, see [Right Sizing: Provisioning Instances to Match Workloads](#).

Resources

Related documents:

- [What is Amazon Simple Queue Service?](#)
- [What is Amazon MQ?](#)
- [Scaling based on Amazon SQS](#)
- [What is AWS Step Functions?](#)
- [What is AWS Lambda?](#)
- [Using AWS Lambda with Amazon SQS](#)
- [What is Amazon EventBridge?](#)
- [Managing Asynchronous Workflows with a REST API](#)

Related videos:

- [AWS re:Invent 2023 - Navigating the journey to serverless event-driven architecture](#)
- [AWS re:Invent 2023 - Using serverless for event-driven architecture & domain-driven design](#)
- [AWS re:Invent 2023 - Advanced event-driven patterns with Amazon EventBridge](#)
- [AWS re:Invent 2023 - Sustainable architecture: Past, present, and future](#)
- [Asynchronous Message Patterns | AWS Events](#)

Related examples:

- [Event-driven architecture with AWS Graviton Processors and Amazon EC2 Spot Instances](#)

SUS03-BP02 Remove or refactor workload components with low or no use

Remove components that are unused and no longer required, and refactor components with little utilization to minimize waste in your workload.

Common anti-patterns:

- You do not regularly check the utilization level of individual components of your workload.
- You do not check and analyze recommendations from AWS rightsizing tools such as [AWS Compute Optimizer](#).

Benefits of establishing this best practice: Removing unused components minimizes waste and improves the overall efficiency of your cloud workload.

Level of risk exposed if this best practice is not established: Medium

Implementation guidance

Unused or underutilized components in a cloud workload consume unnecessary compute, storage or network resources. Remove or refactor these components to directly reduce waste and improve the overall efficiency of a cloud workload. This is an iterative improvement process which can be initiated by changes in demand or the release of a new cloud service. For example, a significant drop in [AWS Lambda](#) function run time can indicate a need to lower the memory size. Also, as AWS releases new services and features, the optimal services and architecture for your workload may change.

Continually monitor workload activity and look for opportunities to improve the utilization level of individual components. By removing idle components and performing rightsizing activities, you meet your business requirements with the fewest cloud resources.

Implementation steps

- **Inventory your AWS resources:** Create an inventory of your AWS resources. In AWS, you can turn on [AWS Resource Explorer](#) to explore and organize your AWS resources. For more details, see [AWS re:Invent 2022 - How to manage resources and applications at scale on AWS](#).
- **Monitor utilization:** Monitor and capture the utilization metrics for critical components of your workload (like CPU utilization, memory utilization, or network throughput in [Amazon CloudWatch metrics](#)).

- **Identify unused components:** Identify unused or under-utilized components in your architecture.
 - For stable workloads, check AWS rightsizing tools such as [AWS Compute Optimizer](#) at regular intervals to identify idle, unused, or underutilized components.
 - For ephemeral workloads, evaluate utilization metrics to identify idle, unused, or underutilized components.
- **Remove unused components:** Retire components and associated assets (like Amazon ECR images) that are no longer needed.
 - [Automated Cleanup of Unused Images in Amazon ECR](#)
 - [Delete unused Amazon Elastic Block Store \(Amazon EBS\) volumes by using AWS Config and AWS Systems Manager](#)
- **Refactor underutilized components:** Refactor or consolidate underutilized components with other resources to improve utilization efficiency. For example, you can provision multiple small databases on a single [Amazon RDS](#) database instance instead of running databases on individual underutilized instances.
- **Evaluate improvements:** Understand the [resources provisioned by your workload to complete a unit of work](#). Use this information to evaluate improvements achieved by removing or refactoring components.
 - [Measure and track cloud efficiency with sustainability proxy metrics, Part I: What are proxy metrics?](#)
 - [Measure and track cloud efficiency with sustainability proxy metrics, Part II: Establish a metrics pipeline](#)

Resources

Related documents:

- [AWS Trusted Advisor](#)
- [What is Amazon CloudWatch?](#)
- [Right Sizing: Provisioning Instances to Match Workloads](#)
- [Optimizing your cost with Rightsizing Recommendations](#)

Related videos:

- [AWS re:Invent 2023 - Capacity, availability, cost efficiency: Pick three](#)

SUS03-BP03 Optimize areas of code that consume the most time or resources

Optimize your code that runs within different components of your architecture to minimize resource usage while maximizing performance.

Common anti-patterns:

- You ignore optimizing your code for resource usage.
- You usually respond to performance issues by increasing the resources.
- Your code review and development process does not track performance changes.

Benefits of establishing this best practice: Using efficient code minimizes resource usage and improves performance.

Level of risk exposed if this best practice is not established: Medium

Implementation guidance

It is crucial to examine every functional area, including the code for a cloud architected application, to optimize its resource usage and performance. Continually monitor your workload's performance in build environments and production and identify opportunities to improve code snippets that have particularly high resource usage. Adopt a regular review process to identify bugs or anti-patterns within your code that use resources inefficiently. Leverage simple and efficient algorithms that produce the same results for your use case.

Implementation steps

- **Use efficient programming language:** Use an efficient operating system and programming language for the workload. For details on energy efficient programming languages (including Rust), see [Sustainability with Rust](#).
- **Use an AI coding companion:** Consider using an AI coding companion such as [Amazon Q Developer](#) to efficiently write code.
- **Automate code reviews:** While developing your workloads, adopt an automated code review process to improve quality and identify bugs and anti-patterns.
 - [Automate code reviews with Amazon CodeGuru Reviewer](#)
 - [Detecting concurrency bugs with Amazon CodeGuru](#)
 - [Raising code quality for Python applications using Amazon CodeGuru](#)

- **Use a code profiler:** Use a code profiler to identify the areas of code that use the most time or resources as targets for optimization.
 - [Reducing your organization's carbon footprint with Amazon CodeGuru Profiler](#)
 - [Understanding memory usage in your Java application with Amazon CodeGuru Profiler](#)
 - [Improving customer experience and reducing cost with Amazon CodeGuru Profiler](#)
- **Monitor and optimize:** Use continuous monitoring resources to identify components with high resource requirements or suboptimal configuration.
 - Replace computationally intensive algorithms with simpler and more efficient version that produce the same result.
 - Remove unnecessary code such as sorting and formatting.
- **Use code refactoring or transformation:** Explore the possibility of [Amazon Q code transformation](#) for application maintenance and upgrades.
 - [Upgrade language versions with Amazon Q Code Transformation](#)
 - [AWS re:Invent 2023 - Automate app upgrades & maintenance using Amazon Q Code Transformation](#)

Resources

Related documents:

- [What is Amazon CodeGuru Profiler?](#)
- [FPGA instances](#)
- [The AWS SDKs on Tools to Build on AWS](#)

Related videos:

- [Improve Code Efficiency Using Amazon CodeGuru Profiler](#)
- [Automate Code Reviews and Application Performance Recommendations with Amazon CodeGuru](#)

SUS03-BP04 Optimize impact on devices and equipment

Understand the devices and equipment used in your architecture and use strategies to reduce their usage. This can minimize the overall environmental impact of your cloud workload.

Common anti-patterns:

- You ignore the environmental impact of devices used by your customers.
- You manually manage and update resources used by customers.

Benefits of establishing this best practice: Implementing software patterns and features that are optimized for customer device can reduce the overall environmental impact of cloud workload.

Level of risk exposed if this best practice is not established: Medium

Implementation guidance

Implementing software patterns and features that are optimized for customer devices can reduce the environmental impact in several ways:

- Implementing new features that are backward compatible can reduce the number of hardware replacements.
- Optimizing an application to run efficiently on devices can help to reduce their energy consumption and extend their battery life (if they are powered by battery).
- Optimizing an application for devices can also reduce the data transfer over the network.

Understand the devices and equipment used in your architecture, their expected lifecycle, and the impact of replacing those components. Implement software patterns and features that can help to minimize the device energy consumption, the need for customers to replace the device and also upgrade it manually.

Implementation steps

- **Conduct an inventory:** Inventory the devices used in your architecture. Devices can be mobile, tablet, IOT devices, smart light, or even smart devices in a factory.
- **Use energy-efficient devices:** Consider using energy-efficient devices in your architecture. Use power management configurations on devices to enter low power mode when not in use.
- **Run efficient applications:** Optimize the application running on the devices:
 - Use strategies such as running tasks in the background to reduce their energy consumption.
 - Account for network bandwidth and latency when building payloads, and implement capabilities that help your applications work well on low bandwidth, high latency links.

- Convert payloads and files into optimized formats required by devices. For example, you can use [Amazon Elastic Transcoder](#) or [AWS Elemental MediaConvert](#) to convert large, high quality digital media files into formats that users can play back on mobile devices, tablets, web browsers, and connected televisions.
- Perform computationally intense activities server-side (such as image rendering), or use application streaming to improve the user experience on older devices.
- Segment and paginate output, especially for interactive sessions, to manage payloads and limit local storage requirements.
- **Engage suppliers:** Work with device suppliers who use sustainable materials and provide transparency in their supply chains and environmental certifications.
- **Use over-the-air (OTA) updates:** Use automated over-the-air (OTA) mechanism to deploy updates to one or more devices.
 - You can use a [CI/CD pipeline](#) to update mobile applications.
 - You can use [AWS IoT Device Management](#) to remotely manage connected devices at scale.
- **Use managed device farms:** To test new features and updates, use managed device farms with representative sets of hardware and iterate development to maximize the devices supported. For more details, see [SUS06-BP05 Use managed device farms for testing](#).
- **Continue to monitor and improve:** Track the energy usage of devices to identify areas for improvement. Use new technologies or best practices to enhance environmental impacts of these devices.

Resources

Related documents:

- [What is AWS Device Farm?](#)
- [WorkSpaces Applications Documentation](#)
- [NICE DCV](#)
- [OTA tutorial for updating firmware on devices running FreeRTOS](#)
- [Optimizing Your IoT Devices for Environmental Sustainability](#)

Related videos:

- [AWS re:Invent 2023 - Improve your mobile and web app quality using AWS Device Farm](#)

SUS03-BP05 Use software patterns and architectures that best support data access and storage patterns

Understand how data is used within your workload, consumed by your users, transferred, and stored. Use software patterns and architectures that best support data access and storage to minimize the compute, networking, and storage resources required to support the workload.

Common anti-patterns:

- You assume that all workloads have similar data storage and access patterns.
- You only use one tier of storage, assuming all workloads fit within that tier.
- You assume that data access patterns will stay consistent over time.
- Your architecture supports a potential high data access burst, which results in the resources remaining idle most of the time.

Benefits of establishing this best practice: Selecting and optimizing your architecture based on data access and storage patterns will help decrease development complexity and increase overall utilization. Understanding when to use global tables, data partitioning, and caching will help you decrease operational overhead and scale based on your workload needs.

Level of risk exposed if this best practice is not established: Medium

Implementation guidance

To improve long-term workload sustainability, use architecture patterns that support data access and storage characteristics for your workload. These patterns help you efficiently retrieve and process data. For example, you can use [modern data architecture on AWS](#) with purpose-built services optimized for your unique analytics use cases. These architecture patterns allow for efficient data processing and reduce the resource usage.

Implementation steps

- **Understand data characteristics:** Analyze your data characteristics and access patterns to identify the correct configuration for your cloud resources. Key characteristics to consider include:
 - **Data type:** structured, semi-structured, unstructured
 - **Data growth:** bounded, unbounded
 - **Data durability:** persistent, ephemeral, transient

- **Access patterns** reads or writes, update frequency, spiky, or consistent
- **Use optimal architecture patterns:** Use architecture patterns that best support data access and storage patterns.
 - [Patterns for enabling data persistence](#)
 - [Let's Architect! Modern data architectures](#)
 - [Databases on AWS: The Right Tool for the Right Job](#)
- **Use purpose-built services:** Use technologies that are fit-for-purpose.
 - Use technologies that work natively with compressed data.
 - [Athena Compression Support file formats](#)
 - [Format Options for ETL Inputs and Outputs in AWS Glue](#)
 - [Loading compressed data files from Amazon S3 with Amazon Redshift](#)
 - Use purpose-built [analytics services](#) for data processing in your architecture. For detail on AWS purpose-built analytics services, see [AWS re:Invent 2022 - Building modern data architectures on AWS](#).
 - Use the database engine that best supports your dominant query pattern. Manage your database indexes for efficient querying. For further details, see [AWS Databases](#) and [AWS re:Invent 2022 - Modernize apps with purpose-built databases](#).
- **Minimize data transfer:** Select network protocols that reduce the amount of network capacity consumed in your architecture.

Resources

Related documents:

- [COPY from columnar data formats with Amazon Redshift](#)
- [Converting Your Input Record Format in Firehose](#)
- [Improve query performance on Amazon Athena by Converting to Columnar Formats](#)
- [Monitoring DB load with Performance Insights on Amazon Aurora](#)
- [Monitoring DB load with Performance Insights on Amazon RDS](#)
- [Amazon S3 Intelligent-Tiering storage class](#)
- [Build a CQRS event store with Amazon DynamoDB](#)

Related videos:

- [AWS re:Invent 2022 - Building data mesh architectures on AWS](#)
- [AWS re:Invent 2023 - Deep dive into Amazon Aurora and its innovations](#)
- [AWS re:Invent 2023 - Improve Amazon EBS efficiency and be more cost-efficient](#)
- [AWS re:Invent 2023 - Optimizing storage price and performance with Amazon S3](#)
- [AWS re:Invent 2023 - Building and optimizing a data lake on Amazon S3](#)
- [AWS re:Invent 2023 - Advanced event-driven patterns with Amazon EventBridge](#)

Related examples:

- [AWS Purpose Built Databases Workshop](#)
- [AWS Modern Data Architecture Immersion Day](#)
- [Build a Data Mesh on AWS](#)

Data

Question

- [SUS 4 How do you take advantage of data management policies and patterns to support your sustainability goals?](#)

SUS 4 How do you take advantage of data management policies and patterns to support your sustainability goals?

Implement data management practices to reduce the provisioned storage required to support your workload, and the resources required to use it. Understand your data, and use storage technologies and configurations that more effectively support the business value of the data and how it's used. Lifecycle data to more efficient, less performant storage when requirements decrease, and delete data that's no longer required.

Best practices

- [SUS04-BP01 Implement a data classification policy](#)
- [SUS04-BP02 Use technologies that support data access and storage patterns](#)
- [SUS04-BP03 Use policies to manage the lifecycle of your datasets](#)
- [SUS04-BP04 Use elasticity and automation to expand block storage or file system](#)

- [SUS04-BP05 Remove unneeded or redundant data](#)
- [SUS04-BP06 Use shared file systems or storage to access common data](#)
- [SUS04-BP07 Minimize data movement across networks](#)
- [SUS04-BP08 Back up data only when difficult to recreate](#)

SUS04-BP01 Implement a data classification policy

Classify data to understand its criticality to business outcomes and choose the right energy-efficient storage tier to store the data.

Common anti-patterns:

- You do not identify data assets with similar characteristics (such as sensitivity, business criticality, or regulatory requirements) that are being processed or stored.
- You have not implemented a data catalog to inventory your data assets.

Benefits of establishing this best practice: Implementing a data classification policy allows you to determine the most energy-efficient storage tier for data.

Level of risk exposed if this best practice is not established: Medium

Implementation guidance

Data classification involves identifying the types of data that are being processed and stored in an information system owned or operated by an organization. It also involves making a determination on the criticality of the data and the likely impact of a data compromise, loss, or misuse.

Implement data classification policy by working backwards from the contextual use of the data and creating a categorization scheme that takes into account the level of criticality of a given dataset to an organization's operations.

Implementation steps

- **Perform data inventory:** Conduct an inventory of the various data types that exist for your workload.
- **Group data:** Determine criticality, confidentiality, integrity, and availability of data based on risk to the organization. Use these requirements to group data into one of the data classification

tiers that you adopt. As an example, see [Four simple steps to classify your data and secure your startup](#).

- **Define data classification levels and policies:** For each data group, define data classification level (for example, public or confidential) and handling policies. Tag data accordingly. For more detail on data classification categories, see Data Classification whitepaper.
- **Periodically review:** Periodically review and audit your environment for untagged and unclassified data. Use automation to identify this data, and classify and tag the data appropriately. As an example, see [Data Catalog and crawlers in AWS Glue](#).
- **Establish a data catalog:** Establish a data catalog that provides audit and governance capabilities.
- **Documentation:** Document data classification policies and handling procedures for each data class.

Resources

Related documents:

- [Leveraging AWS Cloud to Support Data Classification](#)
- [Tag policies from AWS Organizations](#)

Related videos:

- [AWS re:Invent 2022 - Enabling agility with data governance on AWS](#)
- [AWS re:Invent 2023 - Data protection and resilience with AWS storage](#)

SUS04-BP02 Use technologies that support data access and storage patterns

Use storage technologies that best support how your data is accessed and stored to minimize the resources provisioned while supporting your workload.

Common anti-patterns:

- You assume that all workloads have similar data storage and access patterns.
- You only use one tier of storage, assuming all workloads fit within that tier.
- You assume that data access patterns will stay consistent over time.

Benefits of establishing this best practice: Selecting and optimizing your storage technologies based on data access and storage patterns will help you reduce the required cloud resources to meet your business needs and improve the overall efficiency of cloud workload.

Level of risk exposed if this best practice is not established: Low

Implementation guidance

Select the storage solution that aligns best to your access patterns, or consider changing your access patterns to align with the storage solution to maximize performance efficiency.

Implementation steps

- **Evaluate data and access characteristics:** Evaluate your data characteristics and access pattern to collect the key characteristics of your storage needs. Key characteristics to consider include:
 - **Data type:** structured, semi-structured, unstructured
 - **Data growth:** bounded, unbounded
 - **Data durability:** persistent, ephemeral, transient
 - **Access patterns:** reads or writes, frequency, spiky, or consistent
- **Choose the right storage technology:** Migrate data to the appropriate storage technology that supports your data characteristics and access pattern. Here are some examples of AWS storage technologies and their key characteristics:

Type	Technology	Key characteristics
Object storage	Amazon S3	An object storage service with unlimited scalability, high availability, and multiple options for accessibility. Transferring and accessing objects in and out of Amazon S3 can use a service, such as Transfer Acceleration or Access Points , to support your location, security needs, and access patterns.

Type	Technology	Key characteristics
Archiving storage	Amazon Glacier	Storage class of Amazon S3 built for data-archiving.
Shared file system	Amazon Elastic File System (Amazon EFS)	Mountable file system that can be accessed by multiple types of compute solutions . Amazon EFS automatically grows and shrinks storage and is performance-optimized to deliver consistent low latencies.
Shared file system	Amazon FSx	Built on the latest AWS compute solutions to support four commonly used file systems: NetApp ONTAP, OpenZFS, Windows File Server, and Lustre. Amazon FSx latency, throughput, and IOPS vary per file system and should be considered when selecting the right file system for your workload needs.
Block storage	Amazon Elastic Block Store (Amazon EBS)	Scalable, high-performance block-storage service designed for Amazon Elastic Compute Cloud (Amazon EC2). Amazon EBS includes SSD-backed storage for transactional, IOPS-intensive workloads and HDD-backed storage for throughput-intensive workloads.

Type	Technology	Key characteristics
Relational database	Amazon Aurora , Amazon RDS , Amazon Redshift	Designed to support ACID (atomicity, consistency, isolation, durability) transactions and maintain referential integrity and strong data consistency. Many traditional applications, enterprise resource planning (ERP), customer relationship management (CRM), and ecommerce systems use relational databases to store their data.
Key-value database	Amazon DynamoDB	Optimized for common access patterns, typically to store and retrieve large volumes of data. High-traffic web apps, ecommerce systems, and gaming applications are typical use-cases for key-value databases.

- **Automate storage allocation:** For storage systems that are a fixed size, such as Amazon EBS or Amazon FSx, monitor the available storage space and automate storage allocation on reaching a threshold. You can leverage Amazon CloudWatch to collect and analyze different metrics for [Amazon EBS](#) and [Amazon FSx](#).
- **Choose the right storage class:** Choose the appropriate storage class for your data.
 - Amazon S3 storage classes can be configured at the object level. A single bucket can contain objects stored across all of the storage classes.
 - You can use [Amazon S3 Lifecycle policies](#) to automatically transition objects between storage classes or remove data without any application changes. In general, you have to make a trade-off between resource efficiency, access latency, and reliability when considering these storage mechanisms.

Resources

Related documents:

- [Amazon EBS volume types](#)
- [Amazon EC2 instance store](#)
- [Amazon S3 Intelligent-Tiering](#)
- [Amazon EBS I/O Characteristics](#)
- [Using Amazon S3 storage classes](#)
- [What is Amazon Glacier?](#)

Related videos:

- [AWS re:Invent 2023 - Improve Amazon EBS efficiency and be more cost-efficient](#)
- [AWS re:Invent 2023 - Optimizing storage price and performance with Amazon S3](#)
- [AWS re:Invent 2023 - Building and optimizing a data lake on Amazon S3](#)
- [AWS re:Invent 2022 - Building modern data architectures on AWS](#)
- [AWS re:Invent 2022 - Modernize apps with purpose-built databases](#)
- [AWS re:Invent 2022 - Building data mesh architectures on AWS](#)
- [AWS re:Invent 2023 - Deep dive into Amazon Aurora and its innovations](#)
- [AWS re:Invent 2023 - Advanced data modeling with Amazon DynamoDB](#)

Related examples:

- [Amazon S3 Examples](#)
- [AWS Purpose Built Databases Workshop](#)
- [Databases for Developers](#)
- [AWS Modern Data Architecture Immersion Day](#)
- [Build a Data Mesh on AWS](#)

SUS04-BP03 Use policies to manage the lifecycle of your datasets

Manage the lifecycle of all of your data and automatically enforce deletion to minimize the total storage required for your workload.

Common anti-patterns:

- You manually delete data.
- You do not delete any of your workload data.
- You do not move data to more energy-efficient storage tiers based on its retention and access requirements.

Benefits of establishing this best practice: Using data lifecycle policies ensures efficient data access and retention in a workload.

Level of risk exposed if this best practice is not established: Medium

Implementation guidance

Datasets usually have different retention and access requirements during their lifecycle. For example, your application may need frequent access to some datasets for a limited period of time. After that, those datasets are infrequently accessed. To improve the efficiency of data storage and computation over time, implement lifecycle policies, which are rules that define how data is handled over time.

With lifecycle configuration rules, you can tell the specific storage service to transition a dataset to more energy-efficient storage tiers, archive it, or delete it. This practice minimizes active data storage and retrieval, which leads to lower energy consumption. In addition, practices such as archiving or deleting obsolete data support regulatory compliance and data governance.

Implementation steps

- **Use data classification:** [Classify datasets in your workload.](#)
- **Define handling rules:** Define handling procedures for each data class.
- **Enable automation:** Set automated lifecycle policies to enforce lifecycle rules. Here are some examples of how to set up automated lifecycle policies for different AWS storage services:

Storage service	How to set automated lifecycle policies
Amazon S3	You can use Amazon S3 Lifecycle to manage your objects throughout their lifecycle. If your access patterns are unknown, changing, or unpredictable, you can use Amazon S3

Storage service	How to set automated lifecycle policies
	Intelligent-Tiering , which monitors access patterns and automatically moves objects that have not been accessed to lower-cost access tiers. You can leverage Amazon S3 Storage Lens metrics to identify optimization opportunities and gaps in lifecycle management.
Amazon Elastic Block Store	You can use Amazon Data Lifecycle Manager to automate the creation, retention, and deletion of Amazon EBS snapshots and Amazon EBS-backed AMIs.
Amazon Elastic File System	Amazon EFS lifecycle management automatically manages file storage for your file systems.
Amazon Elastic Container Registry	Amazon ECR lifecycle policies automate the cleanup of your container images by expiring images based on age or count.
AWS Elemental MediaStore	You can use an object lifecycle policy that governs how long objects should be stored in the MediaStore container.

- **Delete unused assets:** Delete unused volumes, snapshots, and data that is out of its retention period. Use native service features like [Amazon DynamoDB Time To Live](#) or [Amazon CloudWatch log retention](#) for deletion.
- **Aggregate and compress:** Aggregate and compress data where applicable based on lifecycle rules.

Resources

Related documents:

- [Optimize your Amazon S3 Lifecycle rules with Amazon S3 Storage Class Analysis](#)
- [Evaluating Resources with AWS Config Rules](#)

Related videos:

- [AWS re:Invent 2021 - Amazon S3 Lifecycle best practices to optimize your storage spend](#)
- [AWS re:Invent 2023 - Optimizing storage price and performance with Amazon S3](#)
- [Simplify Your Data Lifecycle and Optimize Storage Costs With Amazon S3 Lifecycle](#)
- [Reduce Your Storage Costs Using Amazon S3 Storage Lens](#)

SUS04-BP04 Use elasticity and automation to expand block storage or file system

Use elasticity and automation to expand block storage or file system as data grows to minimize the total provisioned storage.

Common anti-patterns:

- You procure large block storage or file system for future need.
- You overprovision the input and output operations per second (IOPS) of your file system.
- You do not monitor the utilization of your data volumes.

Benefits of establishing this best practice: Minimizing over-provisioning for storage system reduces the idle resources and improves the overall efficiency of your workload.

Level of risk exposed if this best practice is not established: Medium

Implementation guidance

Create block storage and file systems with size allocation, throughput, and latency that are appropriate for your workload. Use elasticity and automation to expand block storage or file system as data grows without having to over-provision these storage services.

Implementation steps

- For fixed size storage like [Amazon EBS](#), verify that you are monitoring the amount of storage used versus the overall storage size and create automation, if possible, to increase the storage size when reaching a threshold.
- Use elastic volumes and managed block data services to automate allocation of additional storage as your persistent data grows. As an example, you can use [Amazon EBS Elastic Volumes](#) to change volume size, volume type, or adjust the performance of your Amazon EBS volumes.

- Choose the right storage class, performance mode, and throughput mode for your file system to address your business need, not exceeding that.
 - [Amazon EFS performance](#)
 - [Amazon EBS volume performance on Linux instances](#)
- Set target levels of utilization for your data volumes, and resize volumes outside of expected ranges.
- Right size read-only volumes to fit the data.
- Migrate data to object stores to avoid provisioning the excess capacity from fixed volume sizes on block storage.
- Regularly review elastic volumes and file systems to terminate idle volumes and shrink over-provisioned resources to fit the current data size.

Resources

Related documents:

- [Extend the file system after resizing an EBS volume](#)
- [Modify a volume using Amazon EBS Elastic Volumes](#)
- [Amazon FSx Documentation](#)
- [What is Amazon Elastic File System?](#)

Related videos:

- [Deep Dive on Amazon EBS Elastic Volumes](#)
- [Amazon EBS and Snapshot Optimization Strategies for Better Performance and Cost Savings](#)
- [Optimizing Amazon EFS for cost and performance, using best practices](#)

SUS04-BP05 Remove unneeded or redundant data

Remove unneeded or redundant data to minimize the storage resources required to store your datasets.

Common anti-patterns:

- You duplicate data that can be easily obtained or recreated.

- You back up all data without considering its criticality.
- You only delete data irregularly, on operational events, or not at all.
- You store data redundantly irrespective of the storage service's durability.
- You turn on Amazon S3 versioning without any business justification.

Benefits of establishing this best practice: Removing unneeded data reduces the storage size required for your workload and the workload environmental impact.

Level of risk exposed if this best practice is not established: Medium

Implementation guidance

When you remove unneeded and redundant datasets, you can reduce storage cost and environmental footprint. This practice may also make computing more efficient, as compute resources only process important data instead of unneeded data. Automate the deletion of unneeded data. Use technologies that deduplicate data at the file and block level. Use service features for native data replication and redundancy.

Implementation steps

- **Evaluate public datasets:** Evaluate if you can avoid storing data by using existing publicly available datasets in [AWS Data Exchange](#) and [Open Data on AWS](#).
- **De-duplicate data:** Use mechanisms that can deduplicate data at the block and object level. Here are some examples of how to deduplicate data on AWS:

Storage service	Deduplication mechanism
Amazon S3	Use AWS Lake Formation FindMatches to find matching records across a dataset (including ones without identifiers) by using the new FindMatches ML Transform.
Amazon FSx	Use data deduplication on Amazon FSx for Windows.
Amazon Elastic Block Store snapshots	Snapshots are incremental backups, which means that only the blocks on the device

Storage service	Deduplication mechanism
	that have changed after your most recent snapshot are saved.

- **Use lifecycle policies:** Use lifecycle policies to automate unneeded data deletion. Use native service features like [Amazon DynamoDB Time To Live](#), [Amazon S3 Lifecycle](#), or [Amazon CloudWatch log retention](#) for deletion.
- **Use data virtualization:** Use data virtualization capabilities on AWS to maintain data at its source and avoid data duplication.
 - [Cloud Native Data Virtualization on AWS](#)
 - [Optimize Data Pattern Using Amazon Redshift Data Sharing](#)
- **Use incremental backup:** Use backup technology that can make incremental backups.
- **Use native durability:** Leverage the durability of [Amazon S3](#) and [replication of Amazon EBS](#) to meet your durability goals instead of self-managed technologies (such as a redundant array of independent disks (RAID)).
- **Use efficient logging:** Centralize log and trace data, deduplicate identical log entries, and establish mechanisms to tune verbosity when needed.
- **Use efficient caching:** Pre-populate caches only where justified.
- Establish cache monitoring and automation to resize the cache accordingly.
- **Remove old version assets:** Remove out-of-date deployments and assets from object stores and edge caches when pushing new versions of your workload.

Resources

Related documents:

- [Change log data retention in CloudWatch Logs](#)
- [Data deduplication on Amazon FSx for Windows File Server](#)
- [Features of Amazon FSx for ONTAP including data deduplication](#)
- [Invalidating Files on Amazon CloudFront](#)
- [Using AWS Backup to back up and restore Amazon EFS file systems](#)
- [What is Amazon CloudWatch Logs?](#)
- [Working with backups on Amazon RDS](#)

- [Integrate and deduplicate datasets using AWS Lake Formation](#)

Related videos:

- [Amazon Redshift Data Sharing Use Cases](#)

Related examples:

- [How do I analyze my Amazon S3 server access logs using Amazon Athena?](#)

SUS04-BP06 Use shared file systems or storage to access common data

Adopt shared file systems or storage to avoid data duplication and allow for more efficient infrastructure for your workload.

Common anti-patterns:

- You provision storage for each individual client.
- You do not detach data volume from inactive clients.
- You do not provide access to storage across platforms and systems.

Benefits of establishing this best practice: Using shared file systems or storage allows for sharing data to one or more consumers without having to copy the data. This helps to reduce the storage resources required for the workload.

Level of risk exposed if this best practice is not established: Medium

Implementation guidance

If you have multiple users or applications accessing the same datasets, using shared storage technology is crucial to use efficient infrastructure for your workload. Shared storage technology provides a central location to store and manage datasets and avoid data duplication. It also enforces consistency of the data across different systems. Moreover, shared storage technology allows for more efficient use of compute power, as multiple compute resources can access and process data at the same time in parallel.

Fetch data from these shared storage services only as needed and detach unused volumes to free up resources.

Implementation steps

- **Use shared storage:** Migrate data to shared storage when the data has multiple consumers. Here are some examples of shared storage technology on AWS:

Storage option	When to use
Amazon EBS Multi-Attach	Amazon EBS Multi-Attach allows you to attach a single Provisioned IOPS SSD (io1 or io2) volume to multiple instances that are in the same Availability Zone.
Amazon EFS	See When to Choose Amazon EFS .
Amazon FSx	See Choosing an Amazon FSx File System .
Amazon S3	Applications that do not require a file system structure and are designed to work with object storage can use Amazon S3 as a massively scalable, durable, low-cost object storage solution.

- **Fetch data as needed:** Copy data to or fetch data from shared file systems only as needed. As an example, you can create an [Amazon FSx for Lustre file system backed by Amazon S3](#) and only load the subset of data required for processing jobs to Amazon FSx.
- **Delete unneeded data:** Delete data as appropriate for your usage patterns as outlined in [SUS04-BP03 Use policies to manage the lifecycle of your datasets](#).
- **Detach inactive clients:** Detach volumes from clients that are not actively using them.

Resources

Related documents:

- [Linking your file system to an Amazon S3 bucket](#)
- [Using Amazon EFS for AWS Lambda in your serverless applications](#)
- [Amazon EFS Intelligent-Tiering Optimizes Costs for Workloads with Changing Access Patterns](#)
- [Using Amazon FSx with your on-premises data repository](#)

related videos:

- [Storage cost optimization with Amazon EFS](#)
- [AWS re:Invent 2023 - What's new with AWS file storage](#)
- [AWS re:Invent 2023 - File storage for builders and data scientists on Amazon Elastic File System](#)

SUS04-BP07 Minimize data movement across networks

Use shared file systems or object storage to access common data and minimize the total networking resources required to support data movement for your workload.

Common anti-patterns:

- You store all data in the same AWS Region independent of where the data users are.
- You do not optimize data size and format before moving it over the network.

Benefits of establishing this best practice: Optimizing data movement across the network reduces the total networking resources required for the workload and lowers its environmental impact.

Level of risk exposed if this best practice is not established: Medium

Implementation guidance

Moving data around your organization requires compute, networking, and storage resources. Use techniques to minimize data movement and improve the overall efficiency of your workload.

Implementation steps

- **Use proximity:** Consider proximity to the data or users as a decision factor when [selecting a Region for your workload](#).
- **Partition services:** Partition Regionally-consumed services so that their Region-specific data is stored within the Region where it is consumed.
- **Use efficient file formats:** Use efficient file formats (such as Parquet or ORC) and compress data before you move it over the network.
- **Minimize data movement:** Don't move unused data. Some examples that can help you avoid moving unused data:
 - Reduce API responses to only relevant data.

- Aggregate data where detailed (record-level information is not required).
- See [Well-Architected Lab - Optimize Data Pattern Using Amazon Redshift Data Sharing](#).
- Consider [Cross-account data sharing in AWS Lake Formation](#).
- **Use edge services:** Use services that can help you run code closer to users of your workload.

Service	When to use
Lambda@Edge	Use for compute-heavy operations that are run when objects are not in the cache.
CloudFront Functions	Use for simple use cases such as HTTP(s) request/response manipulations that can be initiated by short-lived functions.
AWS IoT Greengrass	Run local compute, messaging, and data caching for connected devices.

Resources

Related documents:

- [Optimizing your AWS Infrastructure for Sustainability, Part III: Networking](#)
- [AWS Global Infrastructure](#)
- [Amazon CloudFront Key Features including the CloudFront Global Edge Network](#)
- [Compressing HTTP requests in Amazon OpenSearch Service](#)
- [Intermediate data compression with Amazon EMR](#)
- [Loading compressed data files from Amazon S3 into Amazon Redshift](#)
- [Serving compressed files with Amazon CloudFront](#)

Related videos:

- [Demystifying data transfer on AWS](#)

SUS04-BP08 Back up data only when difficult to recreate

Avoid backing up data that has no business value to minimize storage resources requirements for your workload.

Common anti-patterns:

- You do not have a backup strategy for your data.
- You back up data that can be easily recreated.

Benefits of establishing this best practice: Avoiding back-up of non-critical data reduces the required storage resources for the workload and lowers its environmental impact.

Level of risk exposed if this best practice is not established: Medium

Implementation guidance

Avoiding the back up of unnecessary data can help lower cost and reduce the storage resources used by the workload. Only back up data that has business value or is needed to satisfy compliance requirements. Examine backup policies and exclude ephemeral storage that doesn't provide value in a recovery scenario.

Implementation steps

- **Classify data:** Implement data classification policy as outlined in [SUS04-BP01 Implement a data classification policy](#).
- **Design a backup strategy:** Use the criticality of your data classification and design backup strategy based on your [recovery time objective \(RTO\)](#) and [recovery point objective \(RPO\)](#). Avoid backing up non-critical data.
 - Exclude data that can be easily recreated.
 - Exclude ephemeral data from your backups.
 - Exclude local copies of data, unless the time required to restore that data from a common location exceeds your service-level agreements (SLAs).
- **Use automated backup:** Use an automated solution or managed service to back up business-critical data.
 - [AWS Backup](#) is a fully-managed service that makes it easy to centralize and automate data protection across AWS services, in the cloud, and on premises. For hands-on guidance on how

to create automated backups using AWS Backup, see [Well-Architected Labs - Testing Backup and Restore of Data](#).

- [Automate backups and optimize backup costs for Amazon EFS using AWS Backup](#).

Resources

Related best practices:

- [REL09-BP01 Identify and back up all data that needs to be backed up, or reproduce the data from sources](#)
- [REL09-BP03 Perform data backup automatically](#)
- [REL13-BP02 Use defined recovery strategies to meet the recovery objectives](#)

Related documents:

- [Using AWS Backup to back up and restore Amazon EFS file systems](#)
- [Amazon EBS snapshots](#)
- [Working with backups on Amazon Relational Database Service](#)
- [APN Partner: partners that can help with backup](#)
- [AWS Marketplace: products that can be used for backup](#)
- [Backing Up Amazon EFS](#)
- [Backing Up Amazon FSx for Windows File Server](#)
- [Backup and Restore for Amazon ElastiCache \(Redis OSS\)](#)

Related videos:

- [AWS re:Invent 2023 - Backup and disaster recovery strategies for increased resilience](#)
- [AWS re:Invent 2023 - What's new with AWS Backup](#)
- [AWS re:Invent 2021 - Backup, disaster recovery, and ransomware protection with AWS](#)

Hardware and services

Question

- [SUS 5 How do you select and use cloud hardware and services in your architecture to support your sustainability goals?](#)

SUS 5 How do you select and use cloud hardware and services in your architecture to support your sustainability goals?

Look for opportunities to reduce workload sustainability impacts by making changes to your hardware management practices. Minimize the amount of hardware needed to provision and deploy, and select the most efficient hardware and services for your individual workload.

Best practices

- [SUS05-BP01 Use the minimum amount of hardware to meet your needs](#)
- [SUS05-BP02 Use instance types with the least impact](#)
- [SUS05-BP03 Use managed services](#)
- [SUS05-BP04 Optimize your use of hardware-based compute accelerators](#)

SUS05-BP01 Use the minimum amount of hardware to meet your needs

Use the minimum amount of hardware for your workload to efficiently meet your business needs.

Common anti-patterns:

- You do not monitor resource utilization.
- You have resources with a low utilization level in your architecture.
- You do not review the utilization of static hardware to determine if it should be resized.
- You do not set hardware utilization goals for your compute infrastructure based on business KPIs.

Benefits of establishing this best practice: Rightsizing your cloud resources helps to reduce a workload's environmental impact, save money, and maintain performance benchmarks.

Level of risk exposed if this best practice is not established: Medium

Implementation guidance

Optimally select the total number of hardware required for your workload to improve its overall efficiency. The AWS Cloud provides the flexibility to expand or reduce the number of resources

dynamically through a variety of mechanisms, such as [AWS Auto Scaling](#), and meet changes in demand. It also provides [APIs and SDKs](#) that allow resources to be modified with minimal effort. Use these capabilities to make frequent changes to your workload implementations. Additionally, use rightsizing guidelines from AWS tools to efficiently operate your cloud resource and meet your business needs.

Implementation steps

- **Choose the instances type:** Choose the right instances type to best fit your needs. To learn about how to choose Amazon Elastic Compute Cloud instances and use mechanisms such as attribute-based instance selection, see the following:
 - [How do I choose the appropriate Amazon EC2 instance type for my workload?](#)
 - [Attribute-based instance type selection for Amazon EC2 Fleet.](#)
 - [Create an Auto Scaling group using attribute-based instance type selection.](#)
- **Scale:** Use small increments to scale variable workloads.
- **Use multiple compute purchase options:** Balance instance flexibility, scalability, and cost savings with multiple compute purchase options.
 - [Amazon EC2 On-Demand Instances](#) are best suited for new, stateful, and spiky workloads which can't be instance type, location, or time flexible.
 - [Amazon EC2 Spot Instances](#) are a great way to supplement the other options for applications that are fault tolerant and flexible.
 - Leverage [Compute Savings Plans](#) for steady state workloads that allow flexibility if your needs (like AZ, Region, instance families, or instance types) change.
- **Use instance and Availability Zone diversity:** Maximize application availability and take advantage of excess capacity by diversifying your instances and Availability Zones.
- **Rightsize instances:** Use the rightsizing recommendations from AWS tools to make adjustments on your workload. For more information, see [Optimizing your cost with Rightsizing Recommendations](#) and [Right Sizing: Provisioning Instances to Match Workloads](#)
 - Use rightsizing recommendations in AWS Cost Explorer or [AWS Compute Optimizer](#) to identify rightsizing opportunities.
- **Negotiate service-level agreements (SLAs):** Negotiate SLAs that permit temporarily reducing capacity while automation deploys replacement resources.

Resources

Related documents:

- [Optimizing your AWS Infrastructure for Sustainability, Part I: Compute](#)
- [Attribute based Instance Type Selection for Auto Scaling for Amazon EC2 Fleet](#)
- [AWS Compute Optimizer Documentation](#)
- [Operating Lambda: Performance optimization](#)
- [Auto Scaling Documentation](#)

Related videos:

- [AWS re:Invent 2023 - What's new with Amazon EC2](#)
- [AWS re:Invent 2023 - Smart savings: Amazon Elastic Compute Cloud cost-optimization strategies](#)
- [AWS re:Invent 2022 - Optimizing Amazon Elastic Kubernetes Service for performance and cost on AWS](#)
- [AWS re:Invent 2023 - Sustainable compute: reducing costs and carbon emissions with AWS](#)

SUS05-BP02 Use instance types with the least impact

Continually monitor and use new instance types to take advantage of energy efficiency improvements.

Common anti-patterns:

- You are only using one family of instances.
- You are only using x86 instances.
- You specify one instance type in your Amazon EC2 Auto Scaling configuration.
- You use AWS instances in a manner that they were not designed for (for example, you use compute-optimized instances for a memory-intensive workload).
- You do not evaluate new instance types regularly.
- You do not check recommendations from AWS rightsizing tools such as [AWS Compute Optimizer](#).

Benefits of establishing this best practice: By using energy-efficient and right-sized instances, you are able to greatly reduce the environmental impact and cost of your workload.

Level of risk exposed if this best practice is not established: Medium

Implementation guidance

Using efficient instances in cloud workload is crucial for lower resource usage and cost-effectiveness. Continually monitor the release of new instance types and take advantage of energy efficiency improvements, including those instance types designed to support specific workloads such as machine learning training and inference, and video transcoding.

Implementation steps

- **Learn and explore instance types:** Find instance types that can lower your workload's environmental impact.
 - Subscribe to [What's New with AWS](#) to stay up-to-date with the latest AWS technologies and instances.
 - Learn about different AWS instance types.
 - Learn about AWS Graviton-based instances which offer the best performance per watt of energy use in Amazon EC2 by watching [re:Invent 2020 - Deep dive on AWS Graviton2 processor-powered Amazon EC2 instances](#) and [Deep dive into AWS Graviton3 and Amazon EC2 C7g instances](#).
- **Use instance types with the least impact:** Plan and transition your workload to instance types with the least impact.
 - Define a process to evaluate new features or instances for your workload. Take advantage of agility in the cloud to quickly test how new instance types can improve your workload environmental sustainability. Use proxy metrics to measure how many resources it takes you to complete a unit of work.
 - If possible, modify your workload to work with different numbers of vCPUs and different amounts of memory to maximize your choice of instance type.
 - Consider transitioning your workload to Graviton-based instances to improve the performance efficiency of your workload. For more information on moving workloads to AWS Graviton, see [AWS Graviton Fast Start](#) and [Considerations when transitioning workloads to AWS Graviton-based Amazon Elastic Compute Cloud instances](#).
 - Consider selecting the AWS Graviton option in your usage of [AWS managed services](#).
 - Migrate your workload to Regions that offer instances with the least sustainability impact and still meet your business requirements.

- For machine learning workloads, take advantage of purpose-built hardware that is specific to your workload such as [AWS Trainium](#), [AWS Inferentia](#), and [Amazon EC2 DL1](#). AWS Inferentia instances such as Inf2 instances offer up to 50% better performance per watt over comparable Amazon EC2 instances.
- Use [Amazon SageMaker AI Inference Recommender](#) to right size ML inference endpoint.
- For spiky workloads (workloads with infrequent requirements for additional capacity), use [burstable performance instances](#).
- For stateless and fault-tolerant workloads, use [Amazon EC2 Spot Instances](#) to increase overall utilization of the cloud, and reduce the sustainability impact of unused resources.
- **Operate and optimize:** Operate and optimize your workload instance.
 - For ephemeral workloads, evaluate [instance Amazon CloudWatch metrics](#) such as CPUUtilization to identify if the instance is idle or under-utilized.
 - For stable workloads, check AWS rightsizing tools such as [AWS Compute Optimizer](#) at regular intervals to identify opportunities to optimize and right-size the instances. For further examples and recommendations, see the following labs:
 - [Well-Architected Lab - Rightsizing Recommendations](#)
 - [Well-Architected Lab - Rightsizing with Compute Optimizer](#)
 - [Well-Architected Lab - Optimize Hardware Patterns and Observe Sustainability KPIs](#)

Resources

Related documents:

- [Optimizing your AWS Infrastructure for Sustainability, Part I: Compute](#)
- [AWS Graviton](#)
- [Amazon EC2 DL1](#)
- [Amazon EC2 Capacity Reservation Fleets](#)
- [Amazon EC2 Spot Fleet](#)
- [Functions: Lambda Function Configuration](#)
- [Attribute-based instance type selection for Amazon EC2 Fleet](#)
- [Building Sustainable, Efficient, and Cost-Optimized Applications on AWS](#)
- [How the Contino Sustainability Dashboard Helps Customers Optimize Their Carbon Footprint](#)

Related videos:

- [AWS re:Invent 2023 - AWS Graviton: The best price performance for your AWS workloads](#)
- [AWS re:Invent 2023 - New Amazon Elastic Compute Cloud generative AI capabilities in AWS Management Console](#)
- [AWS re:Invent 2023 - What's new with Amazon Elastic Compute Cloud](#)
- [AWS re:Invent 2023 - Smart savings: Amazon Elastic Compute Cloud cost-optimization strategies](#)
- [AWS re:Invent 2021 - Deep dive into AWS Graviton3 and Amazon EC2 C7g instances](#)
- [AWS re:Invent 2022 - Build a cost-, energy-, and resource-efficient compute environment](#)

Related examples:

- [Solution: Guidance for Optimizing Deep Learning Workloads for Sustainability on AWS](#)

SUS05-BP03 Use managed services

Use managed services to operate more efficiently in the cloud.

Common anti-patterns:

- You use Amazon EC2 instances with low utilization to run your applications.
- Your in-house team only manages the workload, without time to focus on innovation or simplifications.
- You deploy and maintain technologies for tasks that can run more efficiently on managed services.

Benefits of establishing this best practice:

- Using managed services shifts the responsibility to AWS, which has insights across millions of customers that can help drive new innovations and efficiencies.
- Managed service distributes the environmental impact of the service across many users because of the multi-tenant control planes.

Level of risk exposed if this best practice is not established: Medium

Implementation guidance

Managed services shift responsibility to AWS for maintaining high utilization and sustainability optimization of the deployed hardware. Managed services also remove the operational and administrative burden of maintaining a service, which allows your team to have more time and focus on innovation.

Review your workload to identify the components that can be replaced by AWS managed services. For example, [Amazon RDS](#), [Amazon Redshift](#), and [Amazon ElastiCache](#) provide a managed database service. [Amazon Athena](#), [Amazon EMR](#), and [Amazon OpenSearch Service](#) provide a managed analytics service.

Implementation steps

1. **Inventory your workload:** Inventory your workload for services and components.
2. **Identify candidates:** Assess and identify components that can be replaced by managed services. Here are some examples of when you might consider using a managed service:

Task	What to use on AWS
Hosting a database	Use managed Amazon Relational Database Service (Amazon RDS) instances instead of maintaining your own Amazon RDS instances on Amazon Elastic Compute Cloud (Amazon EC2) .
Hosting a container workload	Use AWS Fargate , instead of implementing your own container infrastructure.
Hosting web apps	Use AWS Amplify Hosting as fully managed CI/CD and hosting service for static websites and server-side rendered web apps.

3. **Create a migration plan:** Identify dependencies and create a migrations plan. Update runbooks and playbooks accordingly.
 - The [AWS Application Discovery Service](#) automatically collects and presents detailed information about application dependencies and utilization to help you make more informed decisions as you plan your migration
4. **Perform tests** Test the service before migrating to the managed service.

5. **Replace self-hosted services:** Use your migration plan to replace self-hosted services with managed service.
6. **Monitor and adjust:** Continually monitor the service after the migration is complete to make adjustments as required and optimize the service.

Resources

Related documents:

- [AWS Cloud Products](#)
- [AWS Total Cost of Ownership \(TCO\) Calculator](#)
- [Amazon DocumentDB](#)
- [Amazon Elastic Kubernetes Service \(EKS\)](#)
- [Amazon Managed Streaming for Apache Kafka \(Amazon MSK\)](#)

Related videos:

- [AWS re:Invent 2021 - Cloud operations at scale with AWS Managed Services](#)
- [AWS re:Invent 2023 - Best practices for operating on AWS](#)

SUS05-BP04 Optimize your use of hardware-based compute accelerators

Optimize your use of accelerated computing instances to reduce the physical infrastructure demands of your workload.

Common anti-patterns:

- You are not monitoring GPU usage.
- You are using a general-purpose instance for workload while a purpose-built instance can deliver higher performance, lower cost, and better performance per watt.
- You are using hardware-based compute accelerators for tasks where they're more efficient using CPU-based alternatives.

Benefits of establishing this best practice: By optimizing the use of hardware-based accelerators, you can reduce the physical-infrastructure demands of your workload.

Level of risk exposed if this best practice is not established: Medium**Implementation guidance**

If you require high processing capability, you can benefit from using accelerated computing instances, which provide access to hardware-based compute accelerators such as graphics processing units (GPUs) and field programmable gate arrays (FPGAs). These hardware accelerators perform certain functions like graphic processing or data pattern matching more efficiently than CPU-based alternatives. Many accelerated workloads, such as rendering, transcoding, and machine learning, are highly variable in terms of resource usage. Only run this hardware for the time needed, and decommission them with automation when not required to minimize resources consumed.

Implementation steps

- **Explore compute accelerators:** Identify which [accelerated computing instances](#) can address your requirements.
- **Use purpose-built hardware:** For machine learning workloads, take advantage of purpose-built hardware that is specific to your workload, such as [AWS Trainium](#), [AWS Inferentia](#), and [Amazon EC2 DL1](#). AWS Inferentia instances such as Inf2 instances offer up to [50% better performance per watt over comparable Amazon EC2 instances](#).
- **Monitor usage metrics:** Collect usage metric for your accelerated computing instances. For example, you can use CloudWatch agent to collect metrics such as `utilization_gpu` and `utilization_memory` for your GPUs as shown in [Collect NVIDIA GPU metrics with Amazon CloudWatch](#).
- **Rightsize:** Optimize the code, network operation, and settings of hardware accelerators to make sure that underlying hardware is fully utilized.
 - [Optimize GPU settings](#)
 - [GPU Monitoring and Optimization in the Deep Learning AMI](#)
 - [Optimizing I/O for GPU performance tuning of deep learning training in Amazon SageMaker AI](#)
- **Keep up to date:** Use the latest high performant libraries and GPU drivers.
- **Release unneeded instances:** Use automation to release GPU instances when not in use.

Resources**Related documents:**

- [Accelerated Computing](#)
- [Let's Architect! Architecting with custom chips and accelerators](#)
- [How do I choose the appropriate Amazon EC2 instance type for my workload?](#)
- [Amazon EC2 VT1 Instances](#)
- [Choose the best AI accelerator and model compilation for computer vision inference with Amazon SageMaker AI](#)

Related videos:

- [AWS re:Invent 2021 - How to select Amazon EC2 GPU instances for deep learning](#)
- [AWS Online Tech Talks - Deploying Cost-Effective Deep Learning Inference](#)
- [AWS re:Invent 2023 - Cutting-edge AI with AWS and NVIDIA](#)
- [AWS re:Invent 2022 - \[NEW LAUNCH!\] Introducing AWS Inferentia2-based Amazon EC2 Inf2 instances](#)
- [AWS re:Invent 2022 - Accelerate deep learning and innovate faster with AWS Trainium](#)
- [AWS re:Invent 2022 - Deep learning on AWS with NVIDIA: From training to deployment](#)

Process and culture

Question

- [SUS 6 How do your organizational processes support your sustainability goals?](#)

SUS 6 How do your organizational processes support your sustainability goals?

Look for opportunities to reduce your sustainability impact by making changes to your development, test, and deployment practices.

Best practices

- [SUS06-BP01 Communicate and cascade your sustainability goals](#)
- [SUS06-BP02 Adopt methods that can rapidly introduce sustainability improvements](#)
- [SUS06-BP03 Keep your workload up-to-date](#)
- [SUS06-BP04 Increase utilization of build environments](#)

- [SUS06-BP05 Use managed device farms for testing](#)

SUS06-BP01 Communicate and cascade your sustainability goals

Technology is a key enabler of sustainability. IT teams play a crucial role in driving meaningful change towards your organization's sustainability goals. These teams should clearly understand the company's sustainability targets and work to communicate and cascade those priorities across its operations.

Common anti-patterns:

- You don't know your organization's sustainability goals and how they apply to your team.
- You have insufficient awareness and training about the environmental impact of cloud workloads.
- You are unsure about the specific areas to prioritize.
- You do not involve your employees and customers in your sustainability initiatives.

Benefits of establishing this best practice: From optimization of infrastructure and systems to use of innovative technologies, IT teams can reduce the organization's carbon emissions and minimize resource consumption. Communication of sustainability goals can provide the ability for IT teams continuously improve and adapt to evolving sustainability challenges. Additionally, these sustainable optimizations often translate to cost savings as well, which strengthens the business case.

Level of risk exposed if this best practice is not established: Medium

Implementation guidance

The primary sustainability goals for IT teams should be to optimize systems and solutions to increase resource efficiency and minimize the organization's carbon footprint and overall environmental impact. Shared services and initiatives like training programs and operational dashboards can support organizations as they optimize IT operations and build solutions that can help significantly reduce the carbon footprint. The cloud presents an opportunity not only to move physical infrastructure and energy procurement responsibilities to the shared responsibility of the cloud provider but also to continuously optimize the resource efficiency of cloud-based services.

When teams use the cloud's inherent efficiency and shared responsibility model, they can drive meaningful reductions in the organization's environmental impact. This, in turn, can contribute

to the organization's overall sustainability goals and demonstrate the value of these teams as strategic partners in the journey towards a more sustainable future.

Implementation steps

- **Define goals and objectives:** Establish well-defined goals for your IT program. This involves getting input from responsible stakeholders from different departments such as IT, sustainability, and finance. These teams should define measurable goals that align with your organization's sustainability goals, including areas such carbon reduction and resource optimization.
- **Understand the carbon accounting boundaries of your business:** Understand how carbon accounting methods like the Greenhouse Gas (GHG) Protocol relate to your workloads in the cloud (for more detail, see [Cloud sustainability](#)).
- **Use cloud solutions for carbon accounting:** Use cloud solutions such as [carbon accounting solutions on AWS](#) to track scope one, two, and three for GHG emissions across your operations, portfolios, and value chains. With these solutions, organizations can streamline GHG emission data acquisition, simplify reporting, and derive insights to inform their climate strategies.
- **Monitor the carbon footprint of your IT portfolio:** Track and report carbon emissions of your IT systems. Use the [AWS Customer Carbon Footprint Tool](#) to track, measure, review, and forecast the carbon emissions generated from your AWS usage.
- **Communicate resource usage through proxy metrics to your teams:** Track and report on your [resource usage through proxy metrics](#). In the on-demand pricing models of the cloud, resource usage is related to cost, which is a generally-understandable metric. At a minimum, use cost as a proxy metric to communicate the resource usage and improvements by each team.
- **Enable hourly granularity in your Cost Explorer and create a [Cost and Usage Report \(CUR\)](#):** The CUR provides daily or hourly usage granularity, rates, costs, and usage attributes for all AWS services. Use [the Cloud Intelligence Dashboards](#) and its Sustainability Proxy Metrics Dashboard as a starting point for the processing and visualization of cost and usage based data. For more detail, see the following:
 - [Measure and track cloud efficiency with sustainability proxy metrics, Part I: What are proxy metrics?](#)
 - [Measure and track cloud efficiency with sustainability proxy metrics, Part II: Establish a metrics pipeline](#)
- **Continuously optimize and evaluate:** Use an [improvement process](#) to continuously optimize your IT systems, including cloud workload for efficiency and sustainability. Monitor carbon footprint before and after implementation of optimization strategy. Use the reduction in carbon footprint to assess the effectiveness.

- **Foster a sustainability culture:** Use training programs (like [AWS Skill Builder](#)) to educate your employees about sustainability. Engage them in sustainability initiatives. Share and celebrate their success stories. Use incentives to award them if they achieve sustainability targets.

Resources

Related documents:

- [Understanding your carbon emission estimations](#)

Related videos:

- [AWS re:Invent 2023 - Accelerate data-driven circular economy initiatives with AWS](#)
- [AWS re:Invent 2023 - Sustainability innovation in AWS Global Infrastructure](#)
- [AWS re:Invent 2023 - Sustainable architecture: Past, present, and future](#)
- [AWS re:Invent 2022 - Delivering sustainable, high-performing architectures](#)
- [AWS re:Invent 2022 - Architecting sustainably and reducing your AWS carbon footprint](#)
- [AWS re:Invent 2022 - Sustainability in AWS global infrastructure](#)

Related examples:

- [Well-Architected Lab - Turning cost & usage reports into efficiency reports](#)

Related trainings:

- [Sustainability Transformation on AWS](#)
- [SimuLearn - Sustainability Reporting](#)
- [Decarbonization with AWS](#)

SUS06-BP02 Adopt methods that can rapidly introduce sustainability improvements

Adopt methods and processes to validate potential improvements, minimize testing costs, and deliver small improvements.

Common anti-patterns:

- Reviewing your application for sustainability is a task done only once at the beginning of a project.
- Your workload has become stale, as the release process is too cumbersome to introduce minor changes for resource efficiency.
- You do not have mechanisms to improve your workload for sustainability.

Benefits of establishing this best practice: By establishing a process to introduce and track sustainability improvements, you will be able to continually adopt new features and capabilities, remove issues, and improve workload efficiency.

Level of risk exposed if this best practice is not established: Medium

Implementation guidance

Test and validate potential sustainability improvements before deploying them to production. Account for the cost of testing when calculating potential future benefit of an improvement. Develop low cost testing methods to deliver small improvements.

Implementation steps

- **Understand and communicate your organizational sustainability goals:** Understand your organizational sustainability goals, such carbon reduction or water stewardship. Translate these goals into sustainability requirements for your cloud workloads. Communicate these requirements to key stakeholders.
- **Add sustainability requirements to your backlog:** Add requirements for sustainability improvement to your development backlog.
- **Iterate and improve:** Use an [iterative improvement process](#) to identify, evaluate, prioritize, test, and deploy these improvements.
- **Test using minimum viable product (MVP):** Develop and test potential improvements using the minimum viable representative components to reduce the cost and environmental impact of testing.
- **Streamline the process:** Continually improve and streamline your development processes. As an example, Automate your software delivery process using continuous integration and delivery (CI/CD) pipelines to test and deploy potential improvements to reduce the level of effort and limit errors caused by manual processes.
- **Training and awareness:** Run training programs for your team members to educate them about sustainability and how their activities impact your organizational sustainability goals.

- **Assess and adjust:** Continually assess the impact of improvements and make adjustments as needed.

Resources

Related documents:

- [AWS enables sustainability solutions](#)

Related videos:

- [AWS re:Invent 2023 - Sustainable architecture: Past, present, and future](#)
- [AWS re:Invent 2022 - Delivering sustainable, high-performing architectures](#)
- [AWS re:Invent 2022 - Architecting sustainably and reducing your AWS carbon footprint](#)
- [AWS re:Invent 2022 - Sustainability in AWS global infrastructure](#)
- [AWS re:Invent 2023 - What's new with AWS observability and operations](#)

SUS06-BP03 Keep your workload up-to-date

Keep your workload up-to-date to adopt efficient features, remove issues, and improve the overall efficiency of your workload.

Common anti-patterns:

- You assume your current architecture is static and will not be updated over time.
- You do not have any systems or a regular cadence to evaluate if updated software and packages are compatible with your workload.

Benefits of establishing this best practice: By establishing a process to keep your workload up to date, you can adopt new features and capabilities, resolve issues, and improve workload efficiency.

Level of risk exposed if this best practice is not established: Low

Implementation guidance

Up to date operating systems, runtimes, middlewares, libraries, and applications can improve workload efficiency and make it easier to adopt more efficient technologies. Up to date software

might also include features to measure the sustainability impact of your workload more accurately, as vendors deliver features to meet their own sustainability goals. Adopt a regular cadence to keep your workload up to date with the latest features and releases.

Implementation steps

- **Define a process:** Use a process and schedule to evaluate new features or instances for your workload. Take advantage of agility in the cloud to quickly test how new features can improve your workload to:
 - Reduce sustainability impacts.
 - Gain performance efficiencies.
 - Remove barriers for a planned improvement.
 - Improve your ability to measure and manage sustainability impacts.
- **Conduct an inventory:** Inventory your workload software and architecture and identify components that need to be updated.
 - You can use [AWS Systems Manager Inventory](#) to collect operating system (OS), application, and instance metadata from your Amazon EC2 instances and quickly understand which instances are running the software and configurations required by your software policy and which instances need to be updated.
- **Learn the update procedure:** Understand how to update the components of your workload.

Workload component	How to update
Machine images	Use EC2 Image Builder to manage updates to Amazon Machine Images (AMIs) for Linux or Windows server images.
Container images	Use Amazon Elastic Container Registry (Amazon ECR) with your existing pipeline to manage Amazon Elastic Container Service (Amazon ECS) images .
AWS Lambda	AWS Lambda includes version management features .

- **Use automation:** Automate updates to reduce the level of effort to deploy new features and limit errors caused by manual processes.
 - You can use [CI/CD](#) to automatically update AMIs, container images, and other artifacts related to your cloud application.
 - You can use tools such as [AWS Systems Manager Patch Manager](#) to automate the process of system updates, and schedule the activity using [AWS Systems Manager Maintenance Windows](#).

Resources

Related documents:

- [AWS Architecture Center](#)
- [What's New with AWS](#)
- [AWS Developer Tools](#)

Related videos:

- [AWS re:Invent 2022 - Optimize your AWS workloads with best-practice guidance](#)
- [All Things Patch: AWS Systems Manager](#)

SUS06-BP04 Increase utilization of build environments

Increase the utilization of resources to develop, test, and build your workloads.

Common anti-patterns:

- You manually provision or terminate your build environments.
- You keep your build environments running independent of test, build, or release activities (for example, running an environment outside of the working hours of your development team members).
- You over-provision resources for your build environments.

Benefits of establishing this best practice: By increasing the utilization of build environments, you can improve the overall efficiency of your cloud workload while allocating the resources to builders to develop, test, and build efficiently.

Level of risk exposed if this best practice is not established: Low

Implementation guidance

Use automation and infrastructure-as-code to bring build environments up when needed and take them down when not used. A common pattern is to schedule periods of availability that coincide with the working hours of your development team members. Your test environments should closely resemble the production configuration. However, look for opportunities to use instance types with burst capacity, Amazon EC2 Spot Instances, automatic scaling database services, containers, and serverless technologies to align development and test capacity with use. Limit data volume to just meet the test requirements. If using production data in test, explore possibilities of sharing data from production and not moving data across.

Implementation steps

- **Use infrastructure as code:** Use infrastructure as code to provision your build environments.
- **Use automation:** Use automation to manage the lifecycle of your development and test environments and maximize the efficiency of your build resources.
- **Maximize utilization:** Use strategies to maximize the utilization of development and test environments.
 - Use minimum viable representative environments to develop and test potential improvements.
 - Use serverless technologies if possible.
 - Use On-Demand Instances to supplement your developer devices.
 - Use instance types with burst capacity, Spot Instances, and other technologies to align build capacity with use.
 - Adopt native cloud services for secure instance shell access rather than deploying fleets of bastion hosts.
 - Automatically scale your build resources depending on your build jobs.

Resources

Related documents:

- [AWS Systems Manager Session Manager](#)
- [Amazon EC2 Burstable performance instances](#)
- [What is AWS CloudFormation?](#)
- [What is AWS CodeBuild?](#)
- [Instance Scheduler on AWS](#)

Related videos:

- [AWS re:Invent 2023 - Continuous integration and delivery for AWS](#)

SUS06-BP05 Use managed device farms for testing

Use managed device farms to efficiently test a new feature on a representative set of hardware.

Common anti-patterns:

- You manually test and deploy your application on individual physical devices.
- You do not use app testing service to test and interact with your apps (for example, Android, iOS, and web apps) on real, physical devices.

Benefits of establishing this best practice: Using managed device farms for testing cloud-enabled applications provides a number of benefits:

- They include more efficient features to test application on wide range of devices.
- They eliminate the need for in-house infrastructure for testing.
- They offer diverse device types, including older and less popular hardware, which eliminates the need for unnecessary device upgrades.

Level of risk exposed if this best practice is not established: Low

Implementation guidance

Using Managed device farms can help you to streamline the testing process for new features on a representative set of hardware. Managed device farms offer diverse device types including older, less popular hardware, and avoid customer sustainability impact from unnecessary device upgrades.

Implementation steps

- **Define testing requirements:** Define your testing requirements and plan (like test type, operating systems, and test schedule).
 - You can use [Amazon CloudWatch RUM](#) to collect and analyze client-side data and shape your testing plan.

- **Select a managed device farm:** Select a managed device farm that can support your testing requirements. For example, you can use [AWS Device Farm](#) to test and understand the impact of your changes on a representative set of hardware.
- **Use automation:** Use automation and continuous integration/continuous deployment (CI/CD) to schedule and run your tests.
 - [Integrating AWS Device Farm with your CI/CD pipeline to run cross-browser Selenium tests](#)
 - [Building and testing iOS and iPadOS apps with AWS DevOps and mobile services](#)
- **Review and adjust:** Continually review your testing results and make necessary improvements.

Resources

Related documents:

- [AWS Device Farm device list](#)
- [Viewing the CloudWatch RUM dashboard](#)

Related videos:

- [AWS re:Invent 2023 - Improve your mobile and web app quality using AWS Device Farm](#)
- [AWS re:Invent 2021 - Optimize applications through end user insights with Amazon CloudWatch RUM](#)

Related examples:

- [AWS Device Farm Sample App for Android](#)
- [AWS Device Farm Sample App for iOS](#)
- [Appium Web tests for AWS Device Farm](#)

Notices

Customers are responsible for making their own independent assessment of the information in this document. This document: (a) is for informational purposes only, (b) represents current AWS product offerings and practices, which are subject to change without notice, and (c) does not create any commitments or assurances from AWS and its affiliates, suppliers or licensors. AWS products or services are provided “as is” without warranties, representations, or conditions of any kind, whether express or implied. The responsibilities and liabilities of AWS to its customers are controlled by AWS agreements, and this document is not part of, nor does it modify, any agreement between AWS and its customers.

Copyright © 2024 Amazon Web Services, Inc. or its affiliates.

AWS Glossary

For the latest AWS terminology, see the [AWS glossary](#) in the *AWS Glossary Reference*.